

Maestro Scripting Language (MSL)

CONTENTS

1	INTRODUCTION.....	5
2	BASIC CONCEPTS.....	6
3	PROJECT	7
3.1	PROJECT MANAGEMENT	8
3.1.1	<i>Creating programs (.xcs)</i>	8
	<i>definitions for the program header</i>	9
3.1.2	<i>Project creation from command line</i>	10
3.1.2.1	Create projects from .XCS files.....	10
3.1.2.2	Create project from other .pgmx	11
3.1.2.3	Create projects optimised from other .pgmx	12
3.1.2.4	Create projects optimised from .XCS files	13
3.1.2.5	Create projects from .xml files.....	14
3.1.2.6	Create projects from .pgm files	15
3.1.2.7	Create project complete with supports from .XCS	16
3.2	WORKPIECE	17
3.2.1	<i>Raw workpiece</i>	17
3.2.2	<i>Finished workpiece</i>	18
3.2.2.1	Parallelepipedal shaped workpiece	18
3.2.2.2	Workpiece from extrusion	19
3.3	WORKPLANES	20
3.3.1	<i>Standard Face</i>	20
3.3.2	<i>Workplane Parallel</i>	21
3.3.3	<i>Tilting Plane</i>	22
3.3.3.1	Workplane with three points.....	22
3.3.3.2	Workplane, with origin and rotation in relation to X and Y.....	23
3.3.4	<i>Selecting workplane</i>	24
3.4	2D GEOMETRY.....	25
3.4.1	<i>Insert of new geometry</i>	25
3.4.1.1	Segment	26
3.4.1.2	Arc	27
3.4.1.3	Circle.....	29
3.4.1.4	2D polyline (segments and circle arcs)	31
3.4.1.5	Ellipse	35
3.5	MACHINING OPERATIONS.....	36
3.5.1	<i>Drilling (CreateDrill)</i>	37
3.5.2	<i>Slanted drilling (CreateSlantedDrill)</i>	39
3.5.3	<i>Routing (CreateRoughFinish)</i>	41
3.5.4	<i>Slanted routing (CreateSlantedRoughFinish)</i>	43
3.5.5	<i>Chamfer (CreateChamfer)</i>	46
3.5.6	<i>Routing 3D (Create3DRoughFinish)</i>	48
3.5.7	<i>Slot (CreateSlot)</i>	50
3.5.8	<i>Blade cut (CreateBladeCut)</i>	52
3.5.9	<i>Contour (CreateContour)</i>	54
3.5.10	<i>Pocket (CreateContourPocket)</i>	56
3.5.11	<i>Machining mode</i>	59
3.5.11.1	Drilling strategies	59
3.5.11.2	Routing strategies	61
3.5.11.3	3D routing strategies.....	66
3.5.11.4	Blade cut strategies	68
3.5.11.5	Pocket strategies	70
3.5.11.6	Approach/Retraction	72
3.5.11.7	Machining inversion	77
3.5.12	<i>Machine functions</i>	79
3.5.12.1	Jerk control functions	79
3.5.12.2	Hood control functions	81
3.5.12.3	Dustpan control functions	84
3.5.12.4	Rotating axes control functions.....	86
3.5.13	<i>Tool path</i>	88
3.5.14	<i>Machining technologies</i>	92

3.5.14.1	EdgeBanding work technology	93
3.5.14.1.1	Technology	93
3.5.14.1.2	Geometry Definition	93
3.5.14.1.3	Technology Selection.....	94
3.5.14.1.4	Geometry Technology Selection.....	94
3.5.14.1.5	Technology Application.....	95
3.5.15	<i>Machining attributes</i>	96
3.5.15.1	Attributes FEED,ROT,DEPTH	96
3.5.15.2	TAB Attributes (Bridges)	99
3.5.16	<i>Maintain Technological Sequence (SetPriority)</i>	101
3.6	MACROS AND SUBPROGRAMS.....	103
3.6.1	<i>Creating machining-macros (.xsp)</i>	103
3.6.1.1	Creation of the macro basic structure (wizard);	103
3.6.1.2	Implementation of the functions in the macro.	103
3.6.1.3	Debug of a macro	107
3.6.1.4	Structure of a file .xsp	107
3.6.2	<i>Use of .XSP macros (CreateMacro)</i>	109
3.6.2.1	Use of macros. XSP using SetMacroParam	110
3.6.3	<i>Import Script</i>	111
3.7	NC FUNCTIONS.....	111
3.7.1	<i>Enter an ISO instruction</i>	112
3.7.2	<i>Machine parking</i>	113
3.7.3	<i>Null operation</i>	114
3.7.4	<i>Workpiece probing</i>	115
3.7.4.1	Probing with side position correction.....	115
3.7.4.2	Probing with tool length correction.....	116
3.7.4.3	Probing with workpiece roto-translation	117
3.7.5	<i>Display an operator message</i>	118
3.8	WORKPLANS	119
3.9	WORKPIECE SETUP IN MACHINE.....	121
3.9.1	<i>The SetMachiningParameters instruction</i>	121
3.9.1.1	Reading machine parameters with GetMachiningParameters	122
3.9.1.1.1	Continuous cycle.....	122
3.9.1.1.2	Work area	123
3.9.1.1.3	Mechanical options	123
3.9.1.1.4	Locking options	123
3.9.1.1.5	Technology on mirrored areas	123
3.9.1.2	Reading of Work area parameter with GetWorkingField.....	124
3.9.1.2.1	Work Area symmetry.....	124
3.9.1.2.2	Work area origin.....	124
3.9.1.2.3	Work Area Dimensions	124
3.9.1.2.4	Number of the first bar of the Area.....	125
3.9.1.2.5	Work area bar number	125
3.9.1.2.6	Area quadrant	125
3.9.1.2.7	Reference bar code.....	125
3.9.2	<i>Workpieces positioning on machine plane</i>	126
3.10	DEFINITION OF THE BLOCKING SYSTEM.....	128
3.10.1	<i>Clamps blocking</i>	128
3.10.1.1	Clamps positioning on the workpiece.....	128
3.10.1.1.1	Manual clamps positioning.....	128
3.10.1.1.2	Position clamps with Blocking profile	133
3.10.2	<i>Workpiece blocking with suction cups</i>	137
3.11	DEFINITION OF WORKPIECES MACHINED CONTEMPORANEOUSLY	140
3.11.1	<i>Number of workpieces machined contemporaneously</i>	140
3.12	OPTIMIZATION PROJECT	142
3.12.1	<i>Creating pgmx with various optimized workpieces</i>	142
3.13	MATHEMATICAL FUNCTIONS.....	143
3.14	ACCESSORIAL FUNCTIONS.....	144
4	NESTING PROJECT MANAGEMENT.....	145
4.1	CREATE NEW NESTING PROJECT	145
4.1.1	<i>Nesting parts</i>	145
4.1.1.1	.CSV file structure (for Nesting)	145
5	.MIXX PROJECT CREATION FROM COMMAND LINE	146

5.1	.CSV FILE STRUCTURE (FOR MIX).....	146
5.2	CREATE .MIXX PROJECT FROM .CSV FILE	147
6	CREATION OF NESTING PROJECT FROM THE COMMAND LINE.....	148
6.1	NESTING PROJECT CREATION EXECUTION FROM THE COMMAND LINE	148
6.2	FILE STRUCTURE ,CSV OF THE OPTIONS.....	149
6.3	FILE STRUCTURE ,CSV OF THE SHEETS	153
6.4	FILE STRUCTURE ,.CSV OF THE PARTS	154
6.5	NESTING REPORT	155
7	EXECUTION OF THE CYCLE TIME ESTIMATE FROM COMMAND LINE	159
8	TOOLS DATA-BASE	162
9	POST-PROCESSOR FROM COMMAND LINE.....	163
9.1	CREATE .XXL FILE FROM A .PGMX FILE.....	163

1 Introduction

This manual provides a guide in using the Maestro Scripting Language.

Maestro scripting language (MSL) is a programming language that, through the Maestro platform primitives, is used to create scripting in textual format to create:

1. Machining-macros
2. Machining programs

MSL is based on Microsoft's C# programming language; therefore all its instructions and constructs can be used. For the language syntax see the C# programming language reference manuals (an extensive literature on the subject can be found in the internet).

Programming via scripting provides an extremely powerful textual programming language and provides instructions that allow access to all the operational functions that can be performed interactively with Xilog-Maestro: definition of the shape and position of the workpieces, machining assignment, entering machine functions Furthermore it allows the use of parametrics, flow control (conditional instructions, cycles, jumps,...) and algebraic expressions.

Maestro scripting language (MSL) is aimed at:

1. Process technologists, advanced users and SCM branches/agents, that need a flexible and powerful instrument to produce machining-macros or parametric machining programs autonomously.
2. External software houses (operational software, CAD/CAM, etc...) that must produce post-processors for machining programs in PGMX format for Maestro. MSL in facts can be used to construct a complete project, with a description of the workpieces to machine, the geometries and machining and the phases required for the machining sequence. In short it is the Xilog-Maestro input point for external applications.

In particular, the second case applies to situations where the woodworking workshop already has a software to manage the production of door and window frames, stairs, furniture or other products, and it needs to transfer the information to Maestro.

This manual describes how to transfer the geometric data and the data that describes the entire machining process for a specific workpiece to Maestro.

Various instruments or methods available in the Maestro environment will be used to transfer the machining process.

2 Basic concepts

The main basic objects that can be managed with MSL and that can be defined and created in a generic script are:

1. Workpiece
2. Workplane
3. Geometry
4. Machining operation
5. Workplan

The instructions of a generic scripting file are interpreted and executed in sequence.

During the scripting execution phase, when a new object part of one of the basic types defined in the list is created, the object created automatically becomes the active object for the specific type.

For every type of object listed, during the scripting execution an updated state is maintained with the active object, the last one created during the interpretation.

The active object is used by the system to automatically associate the information necessary to create new objects.

See below for some application examples:

1. Geometries: when a geometry is created, the plane that it is associated to is the active plane (current plane).
2. Machining operation: when a machining operation is created it is associated to the geometry and the active workpiece. Furthermore, the operation is associated to the current work phase or, if no work phase has been defined, it is associated to the default phase.

For the complete list of the instructions available to create scripting files in .XCS format, see the XilogMaestroScripting.chm manual in the Languages folder found in the folder that Xilog Maestro is installed in.

Many of the instructions require the Name and Description string type of parameters, in order to associate a name and description to the instruction. The system checks that a script does not contain more than one instruction with the same name. It is therefore important to use different names for the Name parameter. The user can also choose not to specify any value either for the Name or the Description, by entering an empty string as the parameter value (e.g. ""). In this case the system will associate a correct default name to the object created.

3 Project

Project refers an object that can contain the definition of one or more workpieces with 2D geometries and associated machining.

The project contains all the information needed to define a machining process. This process will be illustrated in the following chapters:

- 1) Project management
- 2) Workpiece
- 3) Worktables
- 4) 2D geometries
- 5) Machining operations
- 6) Macro and Subprograms
- 7) NC functions
- 8) Work phases
- 9) Workpiece setup in machine
- 10)Blocking system
- 11)Workpieces to be machined contemporaneously
- 12)Projects optimisation
- 13)Mathematical functions
- 14)Accessorial functions

3.1 Project management

The project is managed in MSL by writing .xcs files.

3.1.1 Creating programs (.xcs)

The programs (XCS) are used to define a complete part program to machine one or more workpieces in textual mode. They can be imported interactively in the Maestro editor (see script import function) or they can be converted in pgmx programs, in batch mode, with the XConverter tool that is distributed with the Maestro software.

The definition of machining programs with the scripting language is usually used so that from an external environment to Maestro or with a generic post-processor, a complete workpiece can be defined without using the interactive environment.

To create a program in .xcs format there is no specific help function. The program must be created on a text file with .xcs extension and it must contain only instructions recognized by Maestro, described in this manual and the basic instructions of the language C#.

For the complete list of the instructions available to create scripting files in .XCS format, see the XilogMaestroScripting.chm manual in the Languages folder found in the folder that Xilog Maestro is installed in.

The program as well as containing recognized instructions and with the correct syntax, it must also have a set of instructions compatible with each other and have the definition of the finished workpiece.

See below for an example of a workpiece program defined with the scripting language.

```
// definitions for the program header
SetMachiningParameters("AB", 1, 21, 1114112, false);

// definition of the finished workpiece
CreateFinishedWorkpieceBox("Frame_Right",937.464,80,68);

// definition of the raw workpiece
CreateRawWorkpiece("Frame_Right",2,2,2,0,0);

// definition of the workpiece position
SetWorkpieceSetupPosition(4.25,423,0, 0);

//definition of the external blocking profile
CreateBlockingProfile("Frame_Right_2",939.46,82,-15);
AddSegmentToBlockingProfile(35.46,82, "");

// definition of blade cut
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, 1.5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_4", 937,464, 0, 937,464, 80);
SetRetractStrategy(true, true, 1.5, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_5",0,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting , "E025", "3",2,-1,-1.5,0);

// definition of blade cut
ResetApproachStrategy();
ResetRetractStrategy();
ResetDustpan();
CreateSegment("Blade Cut at 5 mm_8", 30.29, 79,789, -8,667, -3,398);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 5 mm_9",0,"Blade Cut at 5 mm", TypeOfProcess.GeneralRouting , "E025", "3",2,-1,-1.5,0);

// head routing definition
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetDustpanOffset(10, 0, false);
CreatePolyline("Frame External Routing_10",37,464,80);
AddArc2PointRadiusToPolyline(0,0, 850,false,false);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(3);
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing1",0,"Frame External Routing", TypeOfProcess.GeneralRouting , "E023", "3",2,-1,-1.5,0);

// Definition of the Swap phase
CreateWorkplan("Swap");

// definition of the workpiece position
SetWorkpieceSetupPosition(4.25,423,0, 0);

//definition of the internal blocking profile
CreateBlockingProfile("Frame_Right_3",0,0, -42);
AddSegmentToBlockingProfile(937.464280,0, "");

// definition of routing with the grain
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetDustpanOffset(10, 0, false);
CreatePolyline("Frame External Routing_6",937,464,80);
AddSegmentToPolyline(37,464,80);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(3);
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing2",0,"Frame External Routing", TypeOfProcess.GeneralRouting , "E023", "3",2,-1,-1.5,0);
```

3.1.2 Project creation from command line

A program (.pgmx) can be created from a command line with the XConverter command.

The XConverter command is launched from the command line with the PC "commands prompt" function.

The most effective way is to create a batch file (.bat) where the complete instruction to be executed is written.

This creates output files with the suffix "(n)" where n is a progressive.

When wanting to maintain the original name overwriting the output file at each execution use the "-ow" option.

The "XConverter.exe" application is in the Maestro installation folder.

Double click on the "XConverter.exe" file when it is executed, but it displays the options that can be used for various types of conversions.

3.1.2.1 Create projects from .XCS files

This mode lets you convert one or more .XCS file in .PGMX.

To do this use the Xconverter command with the m 0 option.

Example:

Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^

-s -m 0 ^

-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MSX.xcs" ^

-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MDX.xcs" ^

-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx"

With this example the following files are created:

2_MSX_Output.pgm

2_MDX_Output.pgm

Starting from the input files:

2_MSX.xcs

2_MDX.xcs

And the fixturing file SerrSCM_R1.tlgx

3.1.2.2 Create project from other .pgmx

This mode lets you import in a single .PGMX all the workpieces in the input .PGMX. To do this use the Xconverter command with the m 1 option.

Example:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^  
-s -m 1 ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MSX.pgm" ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MDX.pgm" ^  
-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx" ^  
-o "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MSX-  
MDX.pgm"
```

With this example the following file is created: MSX-MDX.pgm

Importing the workpieces in the input file:

```
2_MSX.xcs  
2_MDX.xcs
```

And using the fixturing file SerrSCM_R1.tlgx

Even if the fixturing file is not used during the workpiece import, this file must be present because it is needed during saving.

3.1.2.3 Create projects optimised from other .pgmx

This mode lets you create .PGMX by minimising the tool path from one machining operation to another.

To do this use the Xconverter command with the m 2 option.

Example:

Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^

-s -m 2 ^

-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MSX.pgm" ^

-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MDX.pgm" ^

-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx"

With this example the following files are created:

2_MSX_Output.pgm

2_MDX_Output.pgm

Optimising the input files:

1_MSX.pgm

1_MDX.pgm

And using the fixturing file SerrSCM_R1.tlgx

3.1.2.4 Create projects optimised from .XCS files

This mode is used to import one or more workpieces described in the XCS files in a single project .PGMX minimising the number of tool changes and the tool path between machining operations.

To do this use the Xconverter command with the m 3 option.

Example:

Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^

-s -m 3 ^

-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MSX.xcs" ^

"D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MDX.xcs" ^

"D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_TRUP_1.xcs" ^

-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx" ^

-o "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MSX-MDX-
TRSUP.pgm"

With this example the following file is created and optimised: MSX-MDX-TRSUP.pgm

Optimising the input files:

1_MSX.xcs

1_MDX.xcs

1_TRSUP_1.xcs

And using the fixturing file SerrSCM_R1.tlgx

3.1.2.5 Create projects from .xml files

This mode is used to create .PGMX from .xml files.
To do this use the Xconverter command with the m 4 option.

Example:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^  
-s -m 4 ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MSX.xml" ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MDX.xml" ^  
-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx"
```

With this example the following files are created:

MSX_Output.pgm

MDX_Output.pgm

Using the input files:

MSX.xml

MDX.xml

And using the fixturing file SerrSCM_R1.tlgx

3.1.2.6 Create projects from .pgm files

This mode is used to create .PGMX from .pgm files.
To do this use the Xconverter command with the m 5 option.

Example:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^  
-s -m 5 ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MSX.pgm" ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MDX.pgm" ^  
-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx"
```

With this example the following files are created:

MSX_Output.pgmx
MDX_Output.pgmx

Optimising the input files:

MSX.pgm
MDX.pgm

And using the fixturing file SerrSCM_R1.tlgx

3.1.2.7 Create project complete with supports from .XCS

This mode is used to import one or more workpieces described in the .XCS files in a single project .PGMX minimising the number of tool changes and the tool path between machining operations.

The project also includes the clamps and any loading assistance. The project is completed of all the phases necessary for the machining operations in the machine with the automatic setup function and the cycles generation.

To do this use the Xconverter command with the m 12 option.

This function requires the previous maestro configuration with the necessary machine configuration and the selection of the clamp to fit on all the supports.

Example:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^
-s -m 12 ^
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MSX.xcs" ^
  "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MDX.xcs" ^
-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx" ^
-ca 200 300 150 250 ^
-o "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\1_MSX-
MDX.pgm"
```

With this example the following file is created: MSX-MDX.pgm

Optimising the input files:

MSX.xcs

MDX.xcs

And using the fixturing file SerrSCM_R1.tlgx

The clamps used to block the workpieces will be those configured in maestro.

The “-ca” function is used to define the loading assistance that includes a movement in positive Y in the loading phase and a movement in negative Y in the unloading phase. Therefore in the example in the loading phase MSX and MDX rise in Y respectively by 200 and 150 mm, whilst in the unloading phase they drop in Y respectively by 300 and 250 mm.

At the end of this function the MSX-MDX.pgm project is ready to be executed in the machine.

3.2 Workpiece

The workpiece must be defined as either “finished workpiece” or “raw workpiece”. More than one workpiece can be entered in a project.

3.2.1 Raw workpiece

The dimensions and the shape of the raw workpiece can be specified with various primitives.

The `CreateRawWorkpiece` instruction is used to specify the raw status of a workpiece with the machining allowances.

Name	Type	Description
Name	string	Workpiece name
bx1	double	Machining allowance in x-
bx2	double	Machining allowance in x+
by1	double	Machining allowance in y-
by2	double	Machining allowance in y+
bz1	double	Machining allowance in z-
bz2	double	Machining allowance in z+

Table 1: CreateRawWorkpiece Parameters

In the example below a workpiece called “Example workpiece” is created with dimensions 1600x800x18 and associated to a raw with machining allowance in x- and x+ equal to 10mm, in y- 6mm, in y+ 2 mm, in z 0mm.

```
CreateFinishedWorkpieceBox("Example workpiece",1106 ,81 ,68);  
CreateRawWorkpiece("Example workpiece",10 ,10 ,6 ,2 ,0 ,0);
```

To specify the geometry of the raw for a complex shaped workpiece obtained via extrusion, the following instruction is used `CreateRawWorkpieceFromExtrusion`.

Name	Type	Description
Name	string	Workpiece name
Dz	double	Workpiece thickness
Bx	double (opt def. 0)	Distance in x of the raw geometry from the origin of the finished workpiece
By	double (opt def. 0)	Distance in y of the raw geometry from the origin of the finished workpiece
Bz	double (opt def. 0)	Distance in z of the raw geometry from the origin of the finished workpiece
internalProfiles	string[] (opt)	List of internal profiles geometries names

Table 2: CreateRawWorkpieceFromExtrusion Parameters

3.2.2 Finished workpiece

MSL can be used to define two workpieces with the following geometric shapes:

1. Parallelepipedal shaped workpiece
2. Workpiece obtained from extrusion along axis Z of a geometry defined on the XY plane with the option of defining "holes" internally (e.g.: door for a window or door frame)

3.2.2.1 Parallelepipedal shaped workpiece

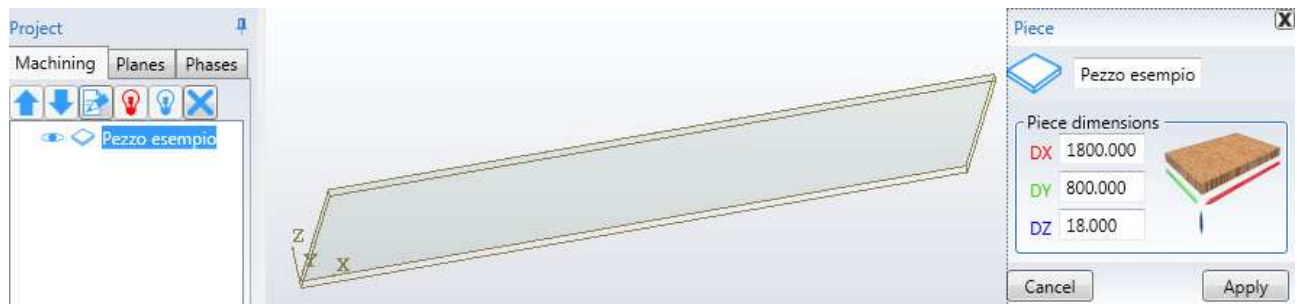
Defined with the following instruction: `CreateFinishedWorkpieceBox`.

Name	Type	Description
name	string	Workpiece name
dx	double	Workpiece length
dy	double	Workpiece width
dz	double	Workpiece thickness

Table 3: CreateFinishedWorkpieceBox parameters

In the example below a workpiece called "Example workpiece" with dimensions 1600x800x18 is created:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
```



3.2.2.2 Workpiece from extrusion

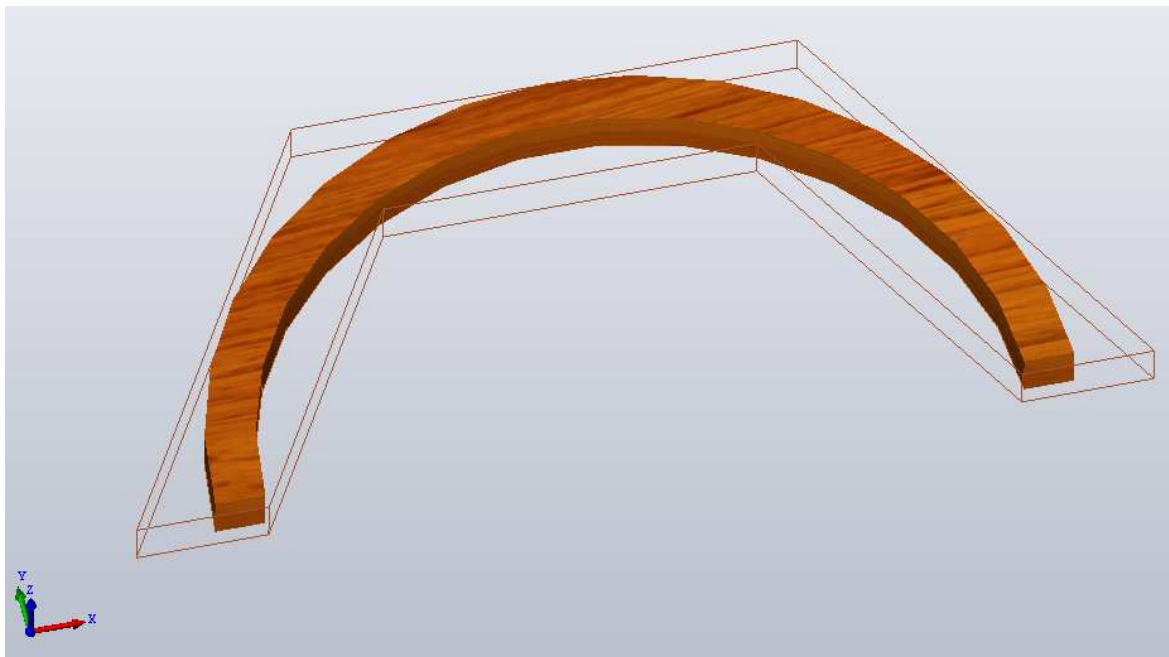
Defined with the following instruction: `CreateFinishedWorkpieceFromExtrusion`. The instruction created a workpiece from extrusion along axis Z starting from a closed geometry defined on the XY plane. The reference geometry for the workpiece “boundary” is the active geometry defined in the script before the call at the instruction `CreateFinishedWorkpieceFromExtrusion`.

Name	Type	Description
Name	string	Workpiece name
Dz	double	Workpiece thickness
internalProfiles	string[] (opt)	List of internal profiles geometries names

Table 4: CreateFinishedWorkpieceFromExtrusion parameters

See the example below for the definition of a finished and raw of an arch workpiece

```
// Create finished workpiece
CreatePolyline("",70,600);
AddArc2PointRadiusToPolyline(1130,600,530,true,false);
AddSegmentToPolyline(1200,600);
AddArc2PointRadiusToPolyline(0,600,600,false,false);
AddSegmentToPolyline(70,600);
CreateFinishedWorkpieceFromExtrusion("Telaio_Up",63);
// Define raw workpiece
CreatePolyline("",74,593);
AddSegmentToPolyline(339,1052);
AddSegmentToPolyline(861,1052);
AddSegmentToPolyline(1126,593);
AddSegmentToPolyline(1310.7,593);
AddSegmentToPolyline(953.3,1211);
AddSegmentToPolyline(246.6,1211);
AddSegmentToPolyline(-110.7,593);
AddSegmentToPolyline(74,593);
CreateRawWorkpieceFromExtrusion("Telaio_Up",63);
```



3.3 Workplanes

To create a workplane, MSL provides three `CreateWorkplane` instructions.

These instructions differ from each other in terms of number and type of parameters that they accept in input. Once created, the workplane becomes an “active workplane”, that is, it becomes the plane on which the subsequent geometries and machining operations will be performed. MSL provides also the `SelectWorkplane` instruction to change the active plane in the script, changing therefore the plane on which the geometries and operations will be performed.

3.3.1 Standard Face

Generating a finished workpiece requires the automatic creation of standard workplanes. In particular, for a parallelepipedal shaped workpiece six planes will be automatically generated :

- “Top” = upper side
- “Bottom” = bottom side
- “Right” = right side
- “Left” = left side
- “Front” = front side
- “Back” = back side)

Whilst for a single plane extrusion workpiece, the upper plane.

3.3.2 Workplane Parallel

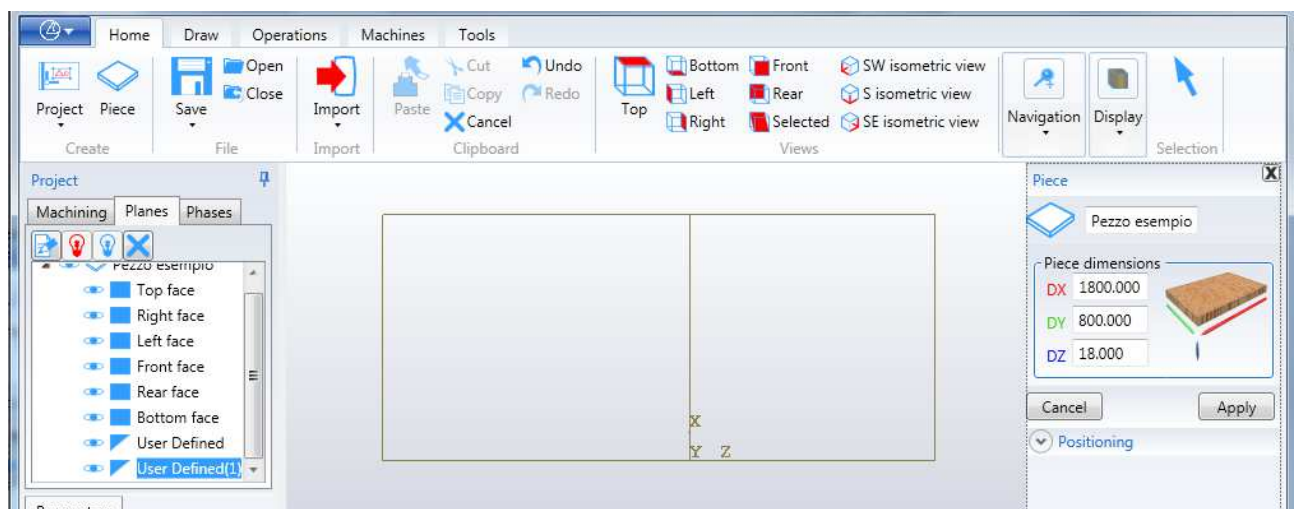
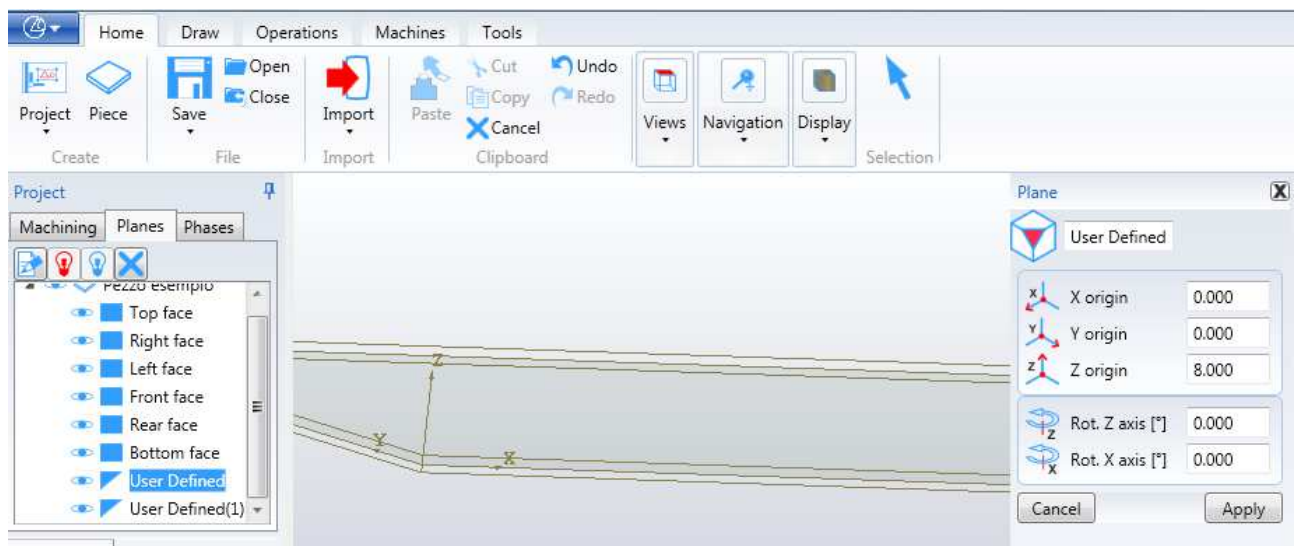
Used to create a workplane parallel to the active plane. Remember that the exit versor from the active plane is always the versor corresponding to axis Z, the new workplane is created by performing a translation of the active plane by an offset along Z.

Name	Type	Description
name	String	Workplane name
offsetZ	Double	Plane translation value along Z

Table 5: CreateWorkplane parameters

In the example below a workplane called "Workplane" is created, parallel to the upper side and a plane "Workplane2" parallel to the right side:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreateWorkplane("Workplane", -10);
SelectWorkplane("Right");
CreateWorkplane("Workplane2", -800);
```



3.3.3 Tilting Plane

The tilting plane can be defined in two ways:

- 1) Workplane with three points
- 2) Workplane with origin and rotation X Y

3.3.3.1 Workplane with three points

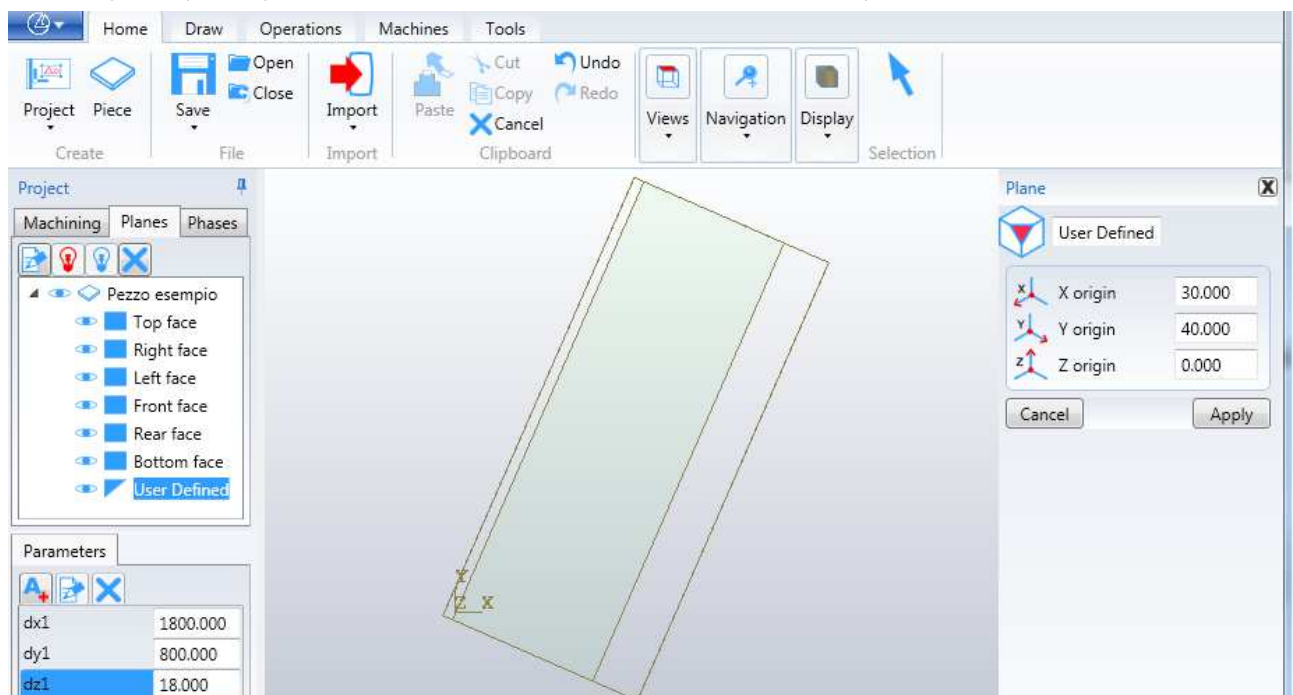
Used to create a workplane starting from the identification of three points on the plane.

Name	Type	Description
Name	String	Workplane name
p1X	Double	Coordinate X of point 1
p1Y	Double	Coordinate Y of point 1
p1Z	Double	Coordinate Z of point 1
p2X	Double	Coordinate X of point 2
p2Y	Double	Coordinate Y of point 2
p2Z	Double	Coordinate Z of point 2
p3X	Double	Coordinate X of point 3
p3Y	Double	Coordinate Y of point 3
p3Z	Double	Coordinate Z of point 3

Table 6: CreateWorkplane parameters

In the example below a workplane called "Workplane" is created, starting from points (30, 40, 0), (100, 200, 5), (1000, 40, 0):

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);  
CreateWorkplane("Workplane", 30, 40, 0, 100, 200, 5, 1000, 40, 0);
```



3.3.3.2 Workplane, with origin and rotation in relation to X and Y

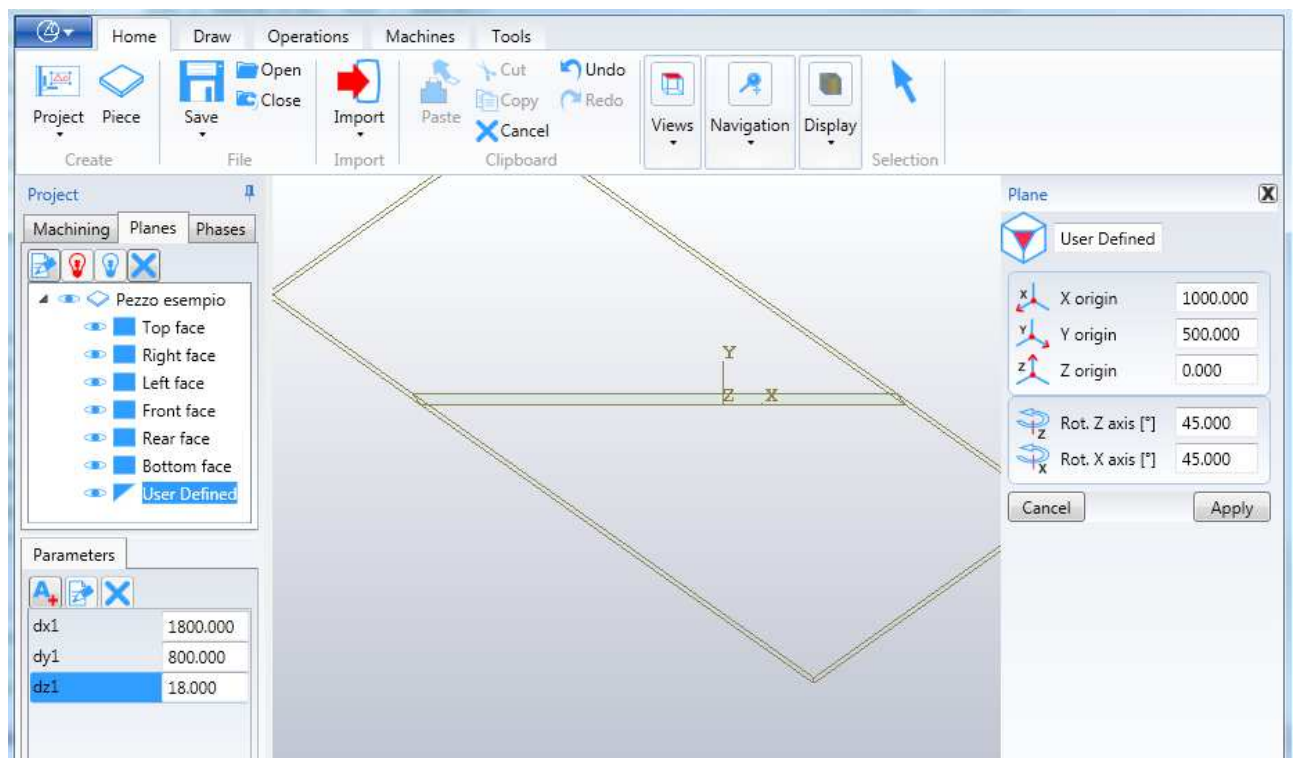
Used to create a workplane starting from the origin of the plane and its rotation in relation to the axes X and Y.

Name	Type	Description
Name	String	Nome del piano di lavoro
X0	Double	Coordinate X of the plane origin
Y0	Double	Coordinate Y of the plane origin
Z0	Double	Coordinate Z of the plane origin
ZRotation	Double	Plane rotation in relation to axis Z
XRotation	Double	Plane rotation in relation to axis X

Table 7: CreateWorkplane parameters

In the example below a workplane called "Workplane " is created, with origin in point (1000, 500, 0), rotation of 45° in relation to axis X and 45° in relation to axis Y:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);  
CreateWorkplane("Workplane", 1000, 500, 0, 45, 45);
```



3.3.4 Selecting workplane

To specify the plane that will be used for the geometries and the machining operations, the active plane must be specified.

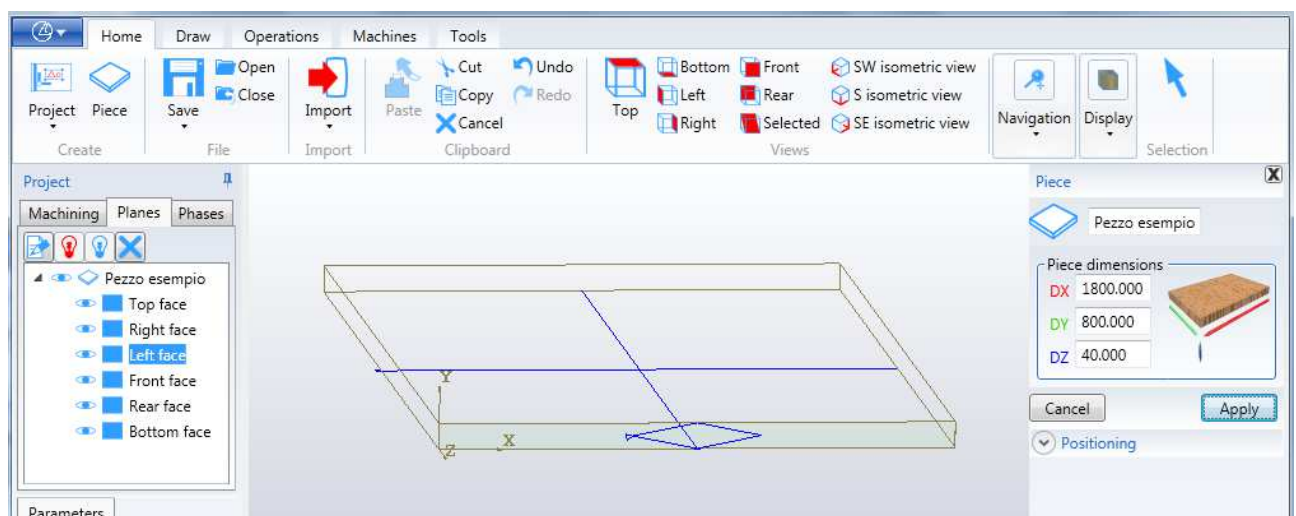
SelectWorkplane is used to select a workplane as “active plane”. To do this the name of the workplane being activated is required. The workplane can be a plane created by the user with one of the previously described instructions or one of the standard machining sides (top, bottom, right, left, front, back). If the user does not activate a plane, the top plane is selected by default as the active workplane.

Name	Type	Description
Name	String	Workplane name “Top” = upper side; “Bottom” = bottom side “Right” = right side; “Left” = left side “Front” = front side; “Back” = back side

Table 8: SelectWorkplane parameters

In the example below the left side of the workpiece is selected as active workplane, on which the geometry of a rhombus is performed, then the bottom side is selected on which a cross is performed:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 40);
SelectWorkplane("Left");
CreatePolyline("Rhombus", 300, 20);
AddSegmentToPolyline( 400, 40);
AddSegmentToPolyline( 500, 20);
AddSegmentToPolyline( 400, 0);
ClosePolyline("Rhombus");
SelectWorkplane("Bottom");
CreateSegment("Segment1", 900, 0, 900, 800);
CreateSegment("Segment2", 0, 400, 1800, 400);
```



3.4 2D geometry

Remember that, as set out in chapter 2, the creation of a 2D geometry is always associated to the active workplane (current plane) and that, as it is associated to a plane, the geometry created is always two-dimensional (2D). In particular the active workplane is always associated to a local reference system, consisting of a right handed Cartesian triad, integral with the plane and with origin in the left bottom end of the plane (axis X along the horizontal component of the plane, axis Y along the vertical component, axis Z exit from the plane). The geometries that the user can create are therefore always dependent on the Cartesian coordinates X and Y, related to the local reference system associated to the current plane. It is important to note that after being created the geometry becomes the “active geometry”, that is, it will be the geometry on which the subsequent machining instructions will be applied.

3.4.1 Insert of new geometry

LIST OF GEOMETRIES:

1. Segment
2. Arc
3. Circle
4. 2D Polyline
5. Ellipse

See below for the instructions available to create geometries available with MSL.

3.4.1.1 Segment

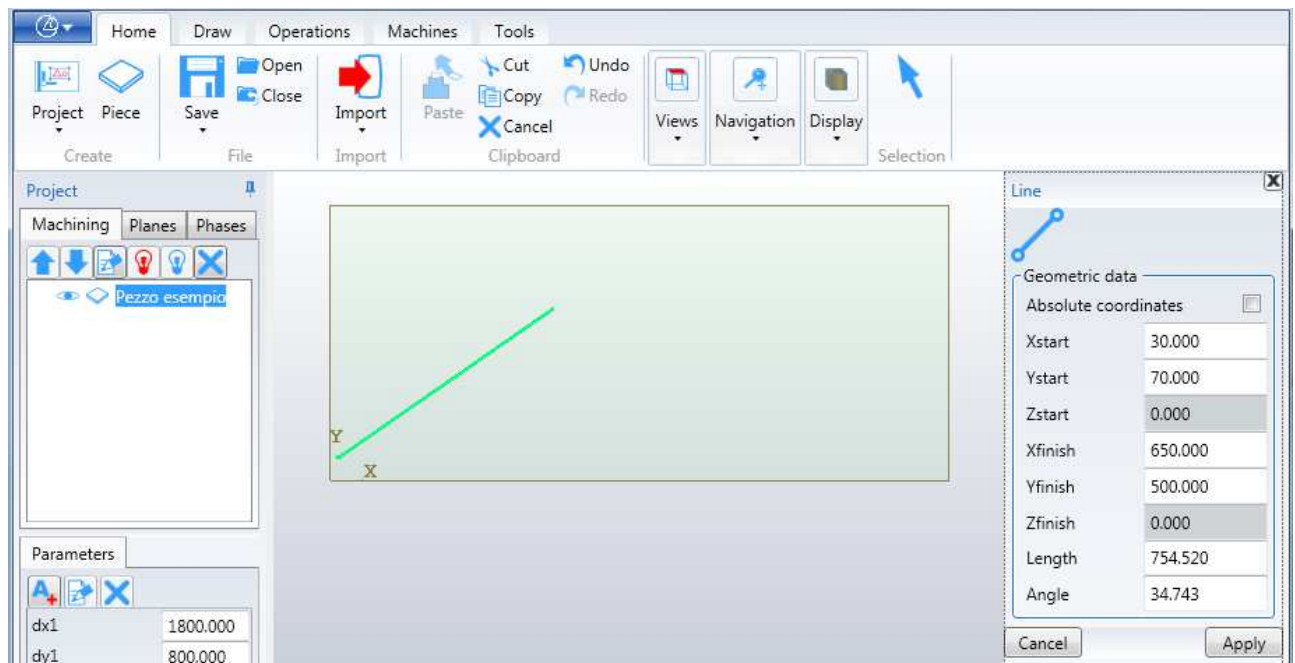
The CreateSegment instructions is used to create a segment. The parameters defined for the function are described out below.

Name	Type	Description
name	string	Geometry name
startX	double	Coordinate X of start point
startY	double	Coordinate Y of start point
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point

Table 9: CreateSegment parameters

In the example below a segment called "Example segment" is created with start point (30,70) and final point (650,500). Firstly the operation workpiece is defined. Furthermore as the workplane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);  
CreateSegment("Example segment", 30, 70, 650, 500);
```



3.4.1.2Arc

Four functions are defined to perform an arc, that differ in terms of the input parameters provided during the arc creation phase:

- CreateArc3Points: creates an arc with three points of the arc
- CreateArc2PointRadius: creates an arc with two points of the arc and the radius
- CreateArc2PointCenter: creates an arc with two points of the arc and the centre point
- CreateArcCenterAngle: creates an arc with one arc point, the centre and the arc angle

See below for the tables describing the input parameters of the four previous instructions.

Name	Type	Description
name	string	Geometry name
p1X	double	Coordinate X of point 1
p1Y	double	Coordinate Y of point 1
p2X	double	Coordinate X of point 2
p2Y	double	Coordinate Y of point 2
p3X	double	Coordinate X of point 3
p3Y	double	Coordinate Y of point 3

Table 10: CreateArc3Points parameters

Name	Type	Description
name	string	Geometry name
startX	double	Coordinate X of start point
startY	double	Coordinate Y of start point
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
radius	double	Arc radius
isClockwise	bool	Clockwise direction
isOver180	bool	Arc angle greater than 180°

Table 11: CreateArc2PointRadius parameters

Name	Type	Description
name	string	Geometry name
startX	double	Coordinate X of start point
startY	double	Coordinate Y of start point
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
isClockwise	bool	Clockwise direction

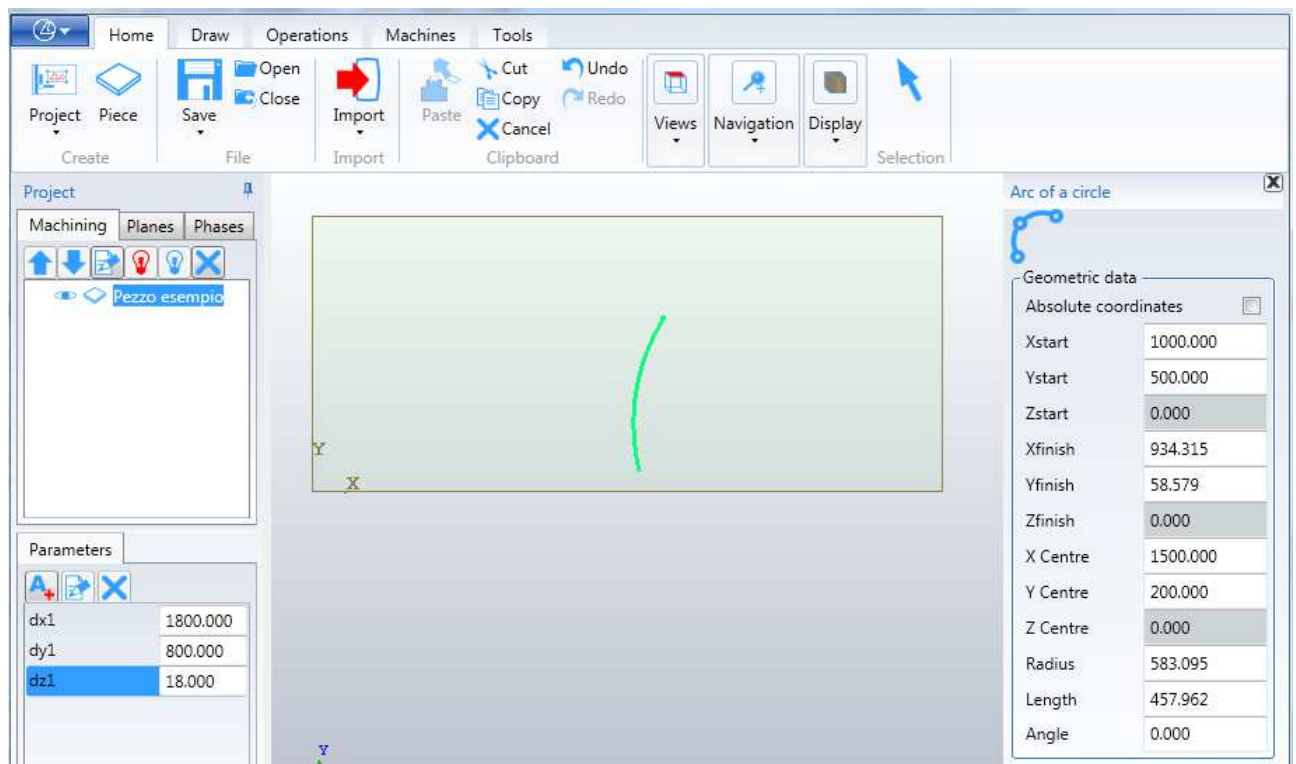
Table 12: CreateArc2PointCenter parameters

Name	Type	Description
name	string	Geometry name
startX	double	Coordinate X of start point
startY	double	Coordinate Y of start point
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
angle	double	Arc angle

Table 13: CreateArcCenterAngle parameters

In the example below an arc called "Example arc" is created with the CreateArcCenterAngle function with start point Pi (1000,500), centre Pc(1500,200) and 45° angle.

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreateArcCenterAngle ("Example arc", 1000, 500, 1500, 200, 45);
```



3.4.1.3Circle

Four instructions are available to perform a circle, that differ in terms of the input parameters provided during the circle creation phase:

- CreateCircleCenterRadius: creates a circle with centre point and radius
- CreateCircleCenterRadius: creates a circle with centre point, radius and rotation direction
- CreateCircleCenterPoint: creates a circle with centre point and a point on the circumference
- CreateCircleCenterPoint: creates a circle with centre point, a point on the circumference and rotation direction

See below for the tables describing the input parameters of the four previous instructions.

Name	Type	Description
name	string	Geometry name
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
radius	double	Circle radius

Table 14: CreateCircleCenterRadius parameters

Name	Type	Description
name	string	Geometry name
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
radius	double	Circle radius
isClockwise	bool	Clockwise direction

Table 15: CreateCircleCenterRadius parameters (with rotation direction)

Name	Type	Description
name	string	Geometry name
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
pointX	double	Coordinate X of the circumference point
pointY	double	Coordinate Y of the circumference point

Table 16: CreateCircleCenterPoint parameters

Name	Type	Description
name	string	Geometry name
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
pointX	double	Coordinate X of the circumference point
pointY	double	Coordinate Y of the circumference point
isClockwise	bool	Clockwise direction

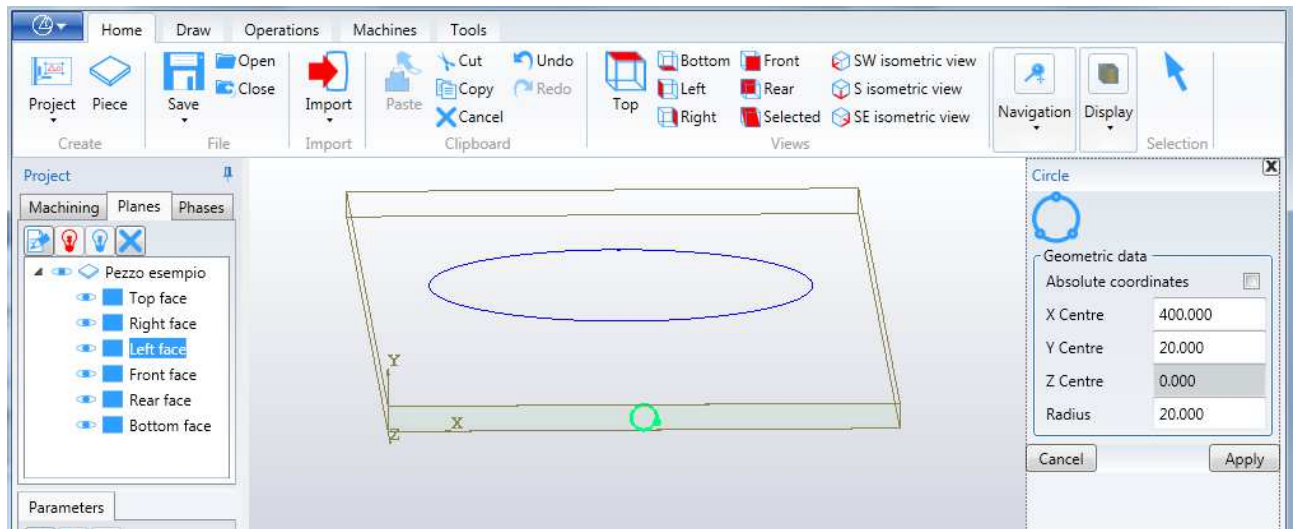
Table 17: CreateCircleCenterPoint parameters (with rotation direction)

In the example below a circle called “Example circle” is created with the CreateCircleCenterRadius function, with centre point Pc (1000,400), radius 300 and counterclockwise direction. This circle is created on the top side of the workpiece, as no

workplane is specified for the geometry. In the same script a second circle called “Example circle2” is then created, using the `CreateCircleCenterPoint` , function with centre `Pc(400,20)` and circumference point `P(420,20)`.

This circle is performed on the left side of the workpiece.

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 40);  
CreateCircleCenterRadius ("Example circle", 1000, 400, 300, false);  
SelectWorkplane("Left");  
CreateCircleCenterPoint("Example circle2", 400, 20, 420, 20);
```



3.4.1.42D polyline (segments and circle arcs)

To create a 2D polyline on the workplane a series of instructions in sequence are used to create and add elements of the polyline. The order of the instructions must be performed correctly as it affects the correct creation of the polyline.

The first instruction in the sequence must always be `CreatePolyline`. This function is used to recognize the start of the procedure and defines the start point of the polyline. The start point is used as the start point of the next instruction in the sequence.

The instructions that follow `CreatePolyline` are functions that add segment or circle arc geometries to the polyline. All these functions are identified with a common prefix (`Add`) and suffix (`ToPolyline`). As a rule these functions use the final point of the geometry defined in the previous instruction as the start point of the geometry to create and they add the geometry they describe to it. The final point of the added geometry will be the start point of the next instruction. The `AddFilletToPolyline` and `AddChamferToPolyline` functions differ from these instructions because even though they have the same prefix and suffix of the previous functions, they do not add further points to the polyline, but they connect adjacent elements already present, with “fillet” and “chamfer” respectively.

The `ClosePolyline` instruction, that joins the last point of the polyline to the first with a segment, can be used to create a closed polyline.

See below for the instructions required to create a polyline:

- `CreatePolyline`: creates a polyline and its start point
- `AddSegmentToPolyline`: adds a segment to the last point of a polyline
- `AddArc3PointsToPolyline`: adds an arc to the last point of a polyline, with three points of the arc
- `AddArc2PointCenterToPolyline`: adds an arc to the last point of a polyline, with two points of the arc and its centre point
- `AddArc2PointRadiusToPolyline`: adds an arc to the last point of a polyline, with two points of the arc and its radius
- `AddArcCenterAngleToPolyline`: adds an arc to the last point of a polyline, with one point of the arc, the centre and the arc angle
- `AddFilletToPolyline`: adds a “fillet” between two adjacent elements of the polyline
- `AddChamferToPolyline`: adds a “chamfer” between two adjacent elements of the polyline
- `ClosePolyline`: closes the geometry by joining the final point and the start point of the polyline with a segment

See below for the input parameters of every instruction described.

Name	Type	Description
name	string	Geometry name
startX	double	Coordinate X of polyline start point
startY	double	Coordinate Y of polyline start point

Table 18: CreatePolyline parameters

Name	Type	Description
endX	double	Coordinate X of polyline final point
endY	double	Coordinate Y of polyline final point
nameElement	string	Name of added element

Table 19: AddSegmentToPolyline parameters

Name	Type	Description
p2X	double	Coordinate X of point 2
p2Y	double	Coordinate Y of point 2
p3X	double	Coordinate X of point 3
p3Y	double	Coordinate Y of point 3
nameElement	string	Name of added element

Table 20: AddArc3PointsToPolyline parameters

Name	Type	Description
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
isClockwise	bool	Clockwise direction
nameElement	string	Name of added element

Table 21: AddArc2PointCenterToPolyline parameters

Name	Type	Description
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
radius	double	Arc radius
isClockwise	bool	Clockwise direction
isOver180	bool	Arc angle greater than 180°
nameElement	string	Name of added element

Table 22: AddArc2PointRadiusToPolyline parameters

Name	Type	Description
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
angle	double	Arc angle
nameElement	string	Name of added element

Table 23: AddArcCenterAngleToPolyline parameters

Name	Type	Description
radius	double	Fillet radius
option	int	0 = straight direct; 1 = indirect fillet 2 = clockwise fillet; 3 = counterclockwise fillet
nameElement	string	Name of added element

Table 24: AddFilletToPolyline parameters

Name	Type	Description
length1	double	Length of first chamfer segment
length2	double	Length of second chamfer segment
option	int	0 = creates chamfer with two lengths
nameElement	string	Name of added element

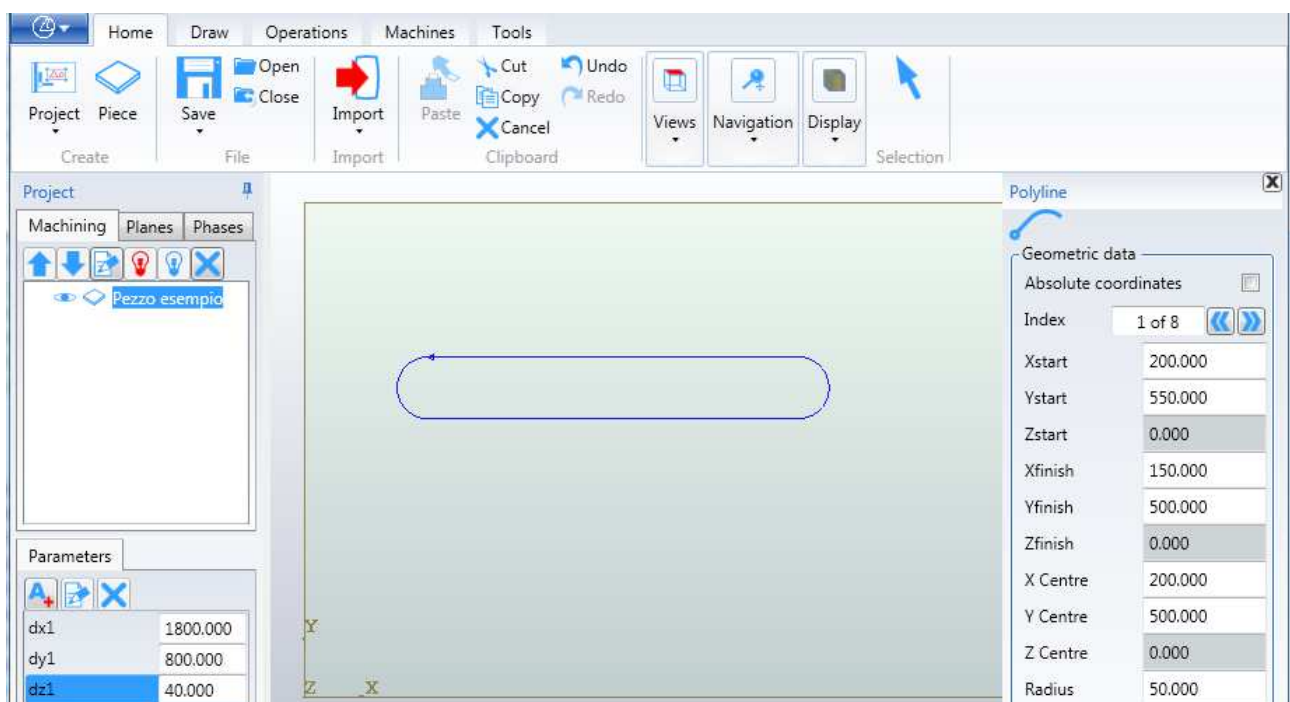
Table 25: AddChamferToPolyline parameters

Name	Type	Description
nameElement	string	Name of added element

Table 26: ClosePolyline parameters

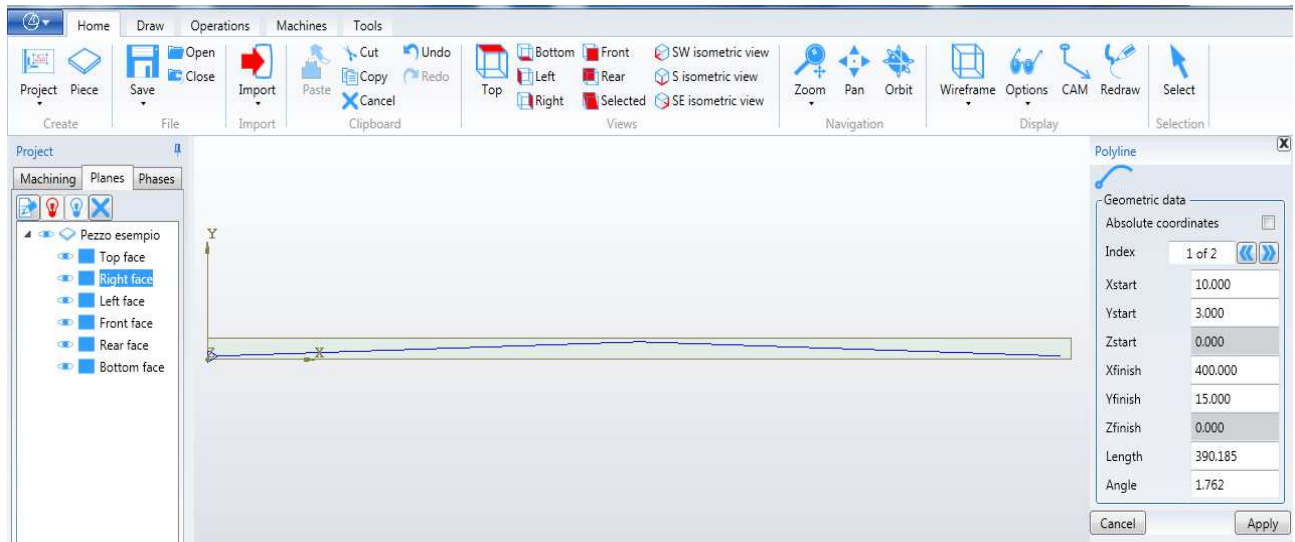
In the example below a closed polyline called “Example polyline” is created, using some of the instructions listed above.

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreatePolyline("Example Polyline", 200, 550);
AddArc2PointRadiusToPolyline(150, 500, 50, false, false);
AddSegmentToPolyline(150, 500);
AddArc2PointRadiusToPolyline(200, 450, 50, false, false);
AddSegmentToPolyline(800, 450);
AddArc2PointRadiusToPolyline(850, 500, 50, false, false);
AddSegmentToPolyline( 850, 500);
AddArc2PointRadiusToPolyline( 800, 550, 50, false, false);
ClosePolyline("Example Polyline");
```



Please note that open polylines can also be created. In this case simply don't specify the ClosePolyline instruction. In the example below an open polyline is added on the right side of the previous workpiece, with the following instructions:

```
// adds open polyline on right side
SelectWorkplane("Right");
CreatePolyline("Example Polyline2", 10, 3);
AddSegmentToPolyline( 400, 18-3);
AddSegmentToPolyline( 800-10, 3);
```



3.4.1.5 Ellipse

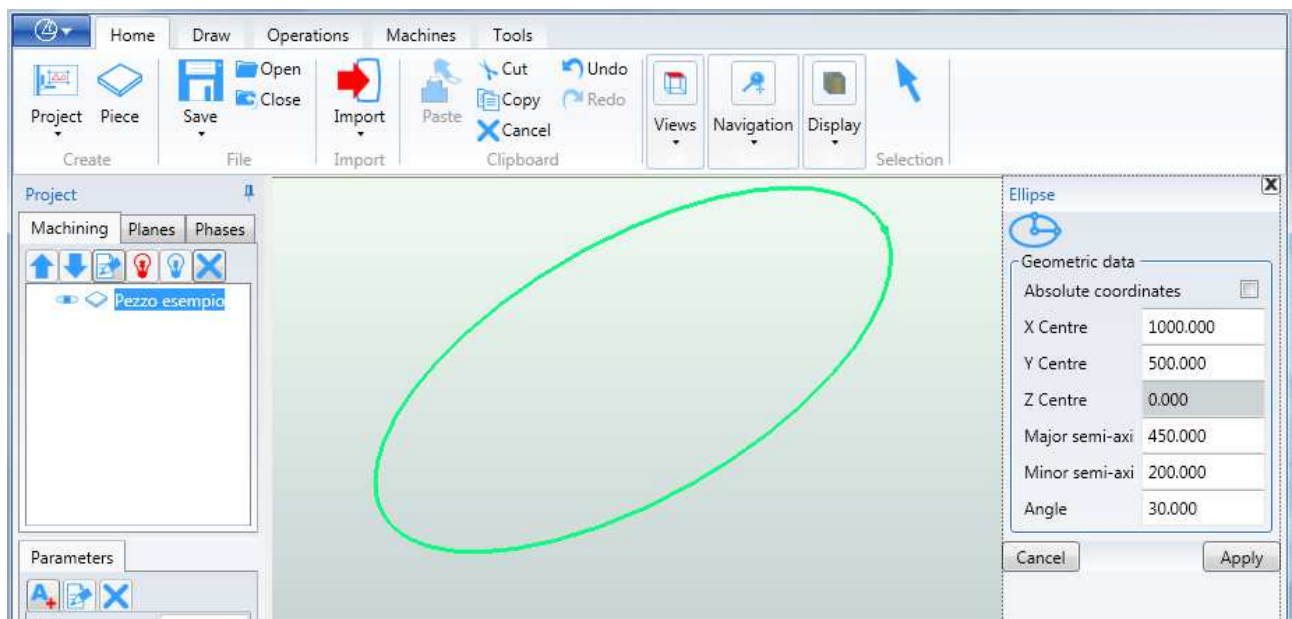
The CreateEllipseCenterAxes instruction is used to create an ellipse. The input parameters defined for the function are set out below.

Name	Type	Description
name	string	Geometry name
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
majorRadius	double	Greater semi-axis
minorRadius	double	Lesser semi-axis
angle	double	Angle of the greater semi-axis in relation to axis X

Table 27: CreateEllipseCenterAxes parameters

In the example below an ellipse called “Example ellipse” is created with centre point (1000,500), greater semi-axis 450, lesser semi-axis 200, angle 30°.

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);  
CreateEllipseCenterAxes ("Example ellipse", 1000, 500, 450, 200, 30);
```



3.5 Machining operations

Type of machining operations that can be defined:

1. Drilling (CreateDrill)
2. Slanted drilling (CreateSlantedDrill)
3. Routing 2.5D (CreateRoughFinish)
4. Routing 3D (Create3DRoughFinish)
5. Slanted routing (CreateSlantedRoughFinish)
6. Slot (CreateSlot)
7. Blade cut (CreateBladeCut)
8. Contour (CreateContour)
9. Pocket (CreateContourPocket)
10. Chamfer (CreateChamfer)
11. Machining-macro defined by the user (CreateMacro)

As a rule for every type of operation a series of parameters can be defined to describe the mode used for the operation. For example the following can be defined: type of tool, tool machining approach and retraction, infeed speed, tool rotation and feed, type of head and other parameters specific to the machining operation selected.

The operation can therefore be customized according to the user's requirements and the type of machining required. If the user does not need to customize all the machining parameters, he can use the default values for some of them, letting the software select the values based on the active fixturing and the specific machine configuration that the operation will be performed on. To recall the "default values" the user can choose not to specify any value for the corresponding input parameter or specify the default value.

See the Maestro manual for the specific meaning of the operations listed above. See below for a description of the most common operations.

3.5.1Drilling (CreateDrill)

To create a drilling use the CreateDrill instruction. The operation is performed at the coordinates assigned to the hole. The coordinates refer to the active workplane.

Name	Type	Description
name	string	Operation name
x	double	Coordinate X of the hole
y	double	Coordinate Y of the hole
depth	double	Hole depth
diameter	double	Hole diameter
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess.Drilling)
tool	string (opt)	Tool name ("-1" for automatic selection)
head	string (opt)	Work head number ("-1" for automatic selection)
dischargeSteps	int (opt def. 0)	Discharge steps number
rotSpeed	double (opt def -1)	Tool rotation speed
boringSpeed	double (opt def -1)	Slow down speed
kindOfHole	double (opt def -1)	Type of hole (tool:P=flat, L=lance, S=taper. The default value is P)
taperHeight	double (opt def 0)	Taper height
securityQuote	double (opt)	Safety quota

Table 28: CreateDrill parameters

See below, by way of an example, the drilling operation in a machining procedure for a door frame.

```
CreateFinishedWorkpieceBox("Internal Frame_Right",1490,92,46.5);
CreateRawWorkpiece("Internal Frame_Right ",0,0,1,1,0,0);

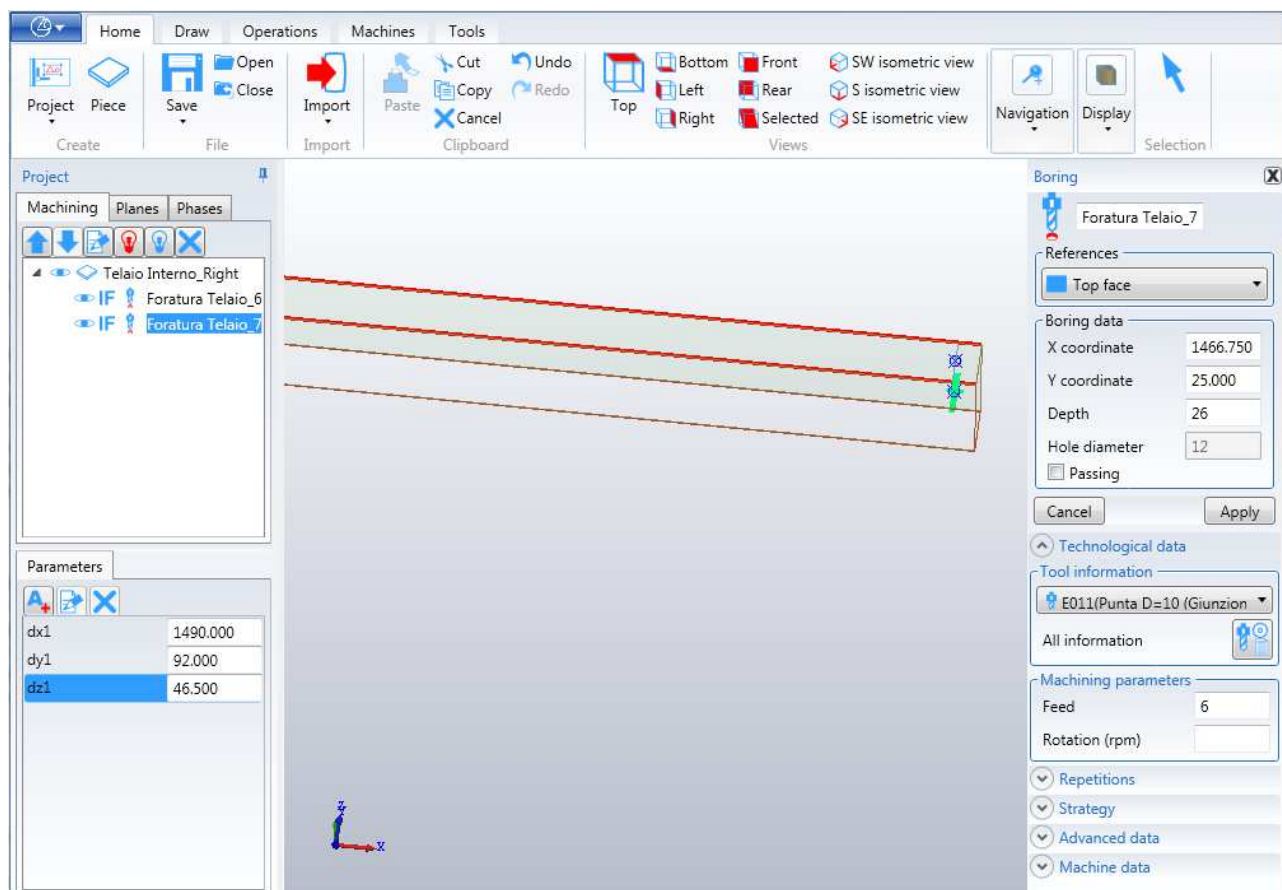
SelectWorkplane("Top");
CreateDrill("Frame Drilling_6",1466.75 ,67 ,26,12, "",TypeOfProcess.Drilling,"E011","3",0,0, 6);

SelectWorkplane("Top");
CreateDrill("Frame Drilling_7",1466.75 ,25 ,26,12, "",TypeOfProcess.Drilling,"E011","3",0,0, 6);
SelectWorkplane("Top");
```

In the first of the script the finished and raw workpiece specifications are configured. The instructions are described in detail in their specific chapters of the manual.

In the second part two drilling operations are performed on the top workplane, with the CreateDrill instruction

In the example the instruction is used without specifying the value of all the input parameters: the parameters not entered use the instruction's default values. For example for the kindOfHole parameter the flat type is used, for taperHeight a zero value height is used, whilst the safety quota is taken from the default configuration.



3.5.2 Slanted drilling (CreateSlantedDrill)

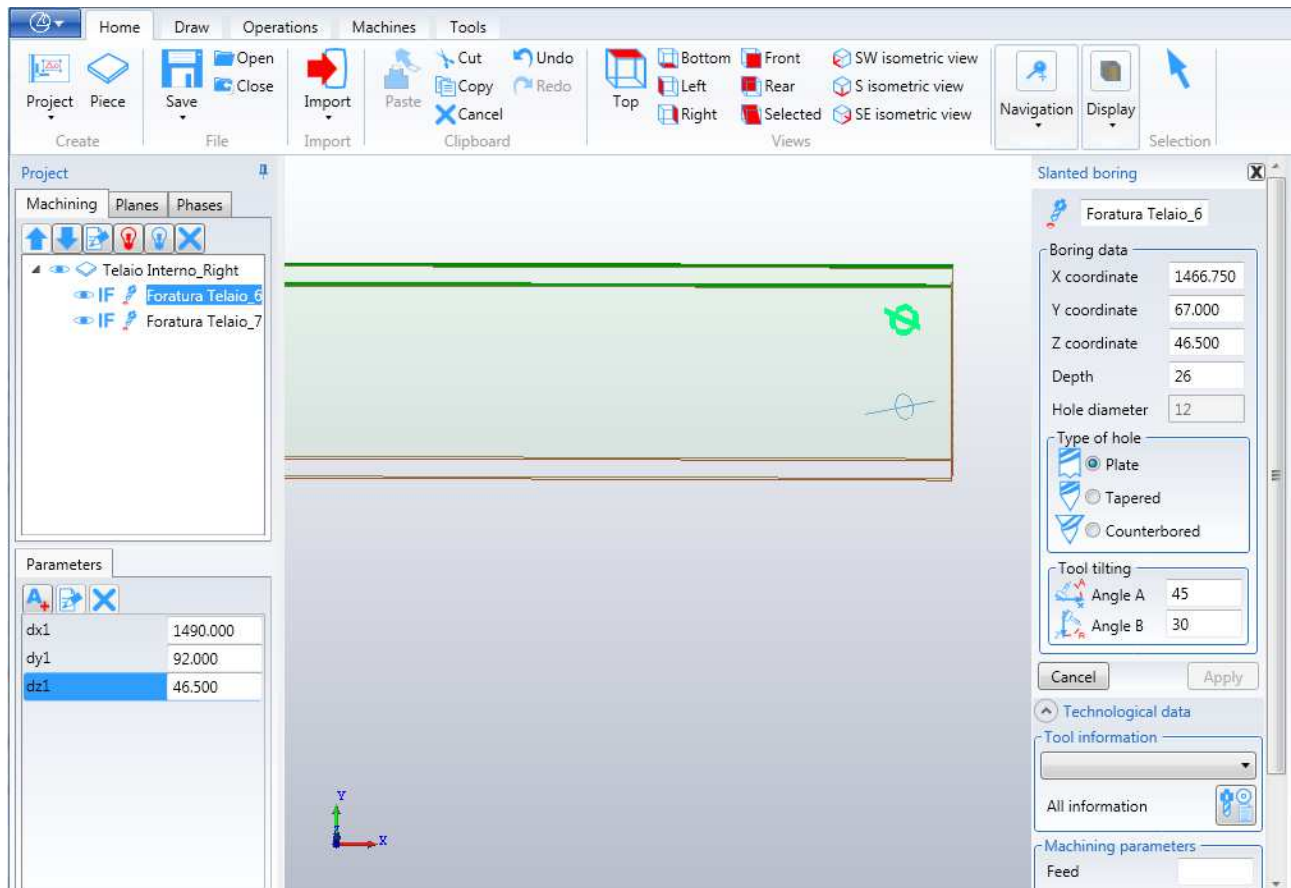
To create a slanted drilling use the `CreateSlantedDrill` instruction. Please note that the coordinates associated to the hole refer to the absolute reference system.

Name	Type	Description
name	string	Operation name
x	double	Coordinate X of the hole
y	double	Coordinate Y of the hole
z	double	Coordinate Z of the hole
angleA	double	Angle (in degrees) of the tool on the horizontal plane, in relation to axis x
angleB	double	Angle (in degrees) of the tool on the horizontal plane, in relation to axis Z
depth	double	Hole depth
diameter	double	Hole diameter
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess.Drilling)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
dischargeSteps	int (opt def. 0)	Discharge steps number
rotSpeed	double (opt def -1)	Tool rotation speed
boringSpeed	double (opt def -1)	Slow down speed
kindOfHole	double (opt def -1)	Type of hole (tool:P=flat, L=lance, S=taper. The default value is P)
taperHeight	double (opt def 0)	Taper height
securityQuote	double (opt)	Safety quota

Table 29: CreateSlantedDrill parameters

By way of an example we use the same drilling procedure used in the previous section, replacing the simple drilling with a slanted drilling. For the “Frame Drilling_6” an angleA equal to 45° and an angleB equal to 30° are fixed, whilst for “Frame Drilling_7” an angleA equal to 90° and an angleB equal to 45° are fixed:

```
CreateSlantedDrill("Frame Drilling_6",1466.75 ,67 , 46.5, 45, 30, 26,12, "Slanted Frame Drilling");
CreateSlantedDrill("Frame Drilling_7",1466.75 ,25 , 46.5, 90, 45, 26,12, "Slanted Frame Drilling");
```



3.5.3 Routing (CreateRoughFinish)

To create a routing use the `CreateRoughFinish` instruction . The routing is always applied to the active geometry.

Name	Type	Description
name	string	Operation name
depth	double	Depth
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (not used, Specify TypeOfProcess.GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
correction	int (opt def 0)	Tool correction (0=central, 1=left, 2=right. The default value is 0)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overMaterial	double (opt def 0)	Overmaterial (The default value is 0: no overmaterial)

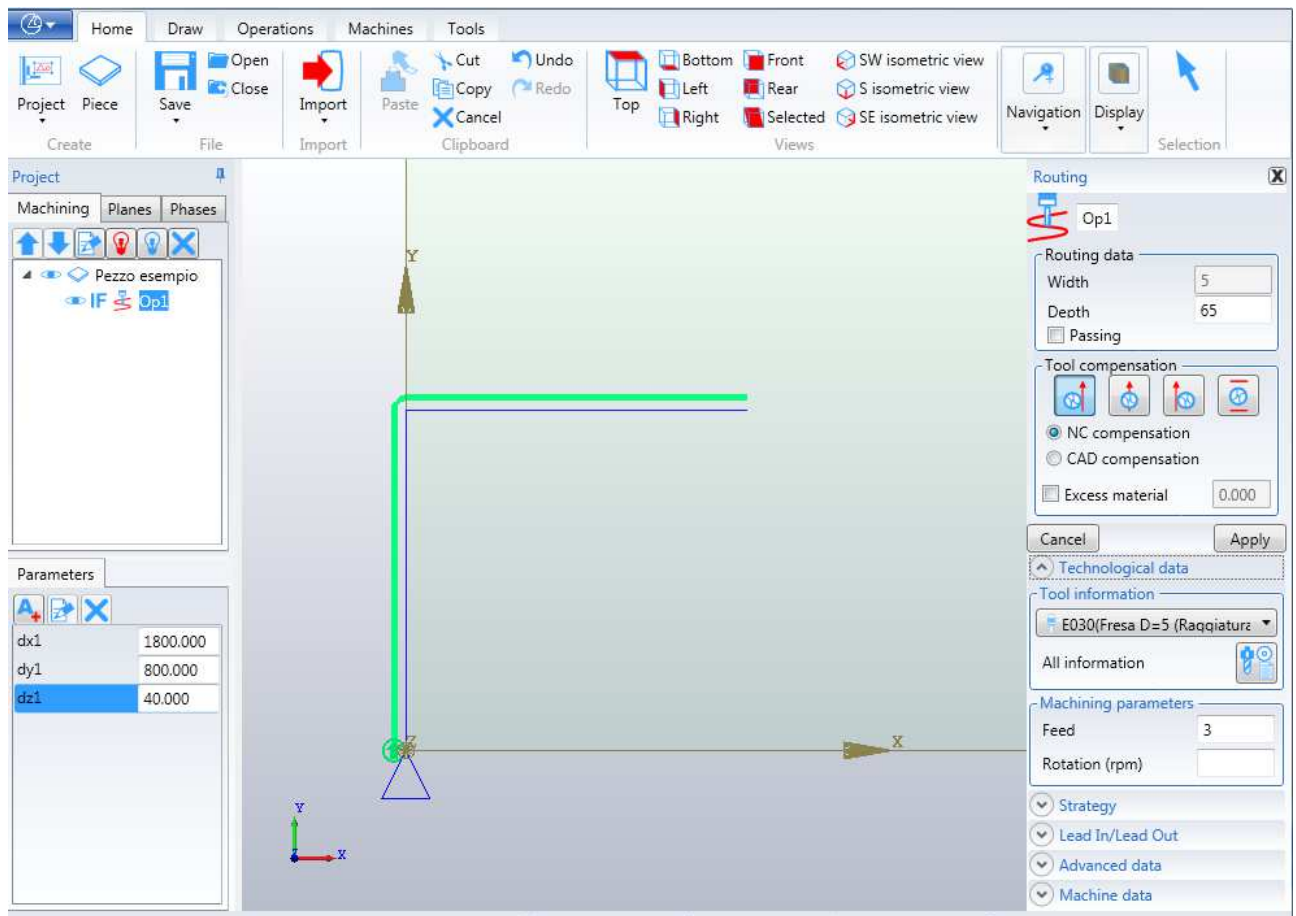
Table 30: CreateRoughFinish parameters

The example below shows a routing operation applied on a geometry defined on the top side of the workpiece.

```
// Plane selection
SelectWorkplane("Top");

// Geometry creation
CreatePolyline("",0,0);
AddSegmentToPolyline(0,70);
AddSegmentToPolyline(70,70);

// Operation creation
CreateRoughFinish("Op1",65,"", TypeOfProcess.GeneralRouting ,"E030","-1",1,-1,-1,3,0);
```



3.5.4 Slanted routing (CreateSlantedRoughFinish)

To create a slanted routing on a standard plane or a plane defined by the user use the `CreateSlantedRoughFinish` instruction. The routing is always applied to the active geometry.

Name	Type	Description
name	string	Operation name
angleA	double	Angle (in degrees) of routing rotation
angleB	double	Angle (in degrees) between tool axis and vertical plane
toolApproach	int	Tool approach to the geometry: 0 = tool parallel to axis X positive of the absolute reference system; 1 = tool perpendicular to the programmed trajectory (tool right side of path); 2 = tool perpendicular to the programmed trajectory (tool left side of path); 3 = tool parallel to the tangent to the programmed trajectory, in the path start point (angle remains fixed during the operation); 4 = tool parallel to the tangent to the programmed trajectory, in the current point (tool remains parallel to the path during the operation);
depth	double	Operation depth
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess. GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overMaterial	double (opt def 0)	Overmaterial (the default value is 0)

Table 31: CreateSlantedRoughFinish parameters

See below for a few examples of slanted routing. On the top side of the workpiece three segment geometries have been drawn and a different slanted routing has been applied to each one. In particular, for each routing a tool slant angle in relation to axis X equal to 20° and 45° in relation to the vertical plane have been selected, whilst the tool approaches to the geometry have been changed. For "Segment1" the tool approach is perpendicular to the geometry, with tool on the right side of the geometry, for "Segment2" the approach is perpendicular to the geometry, but with tool on the left side of the geometry and finally for "Segment3" the tool approach is parallel to the tangent to the trajectory at the start point.

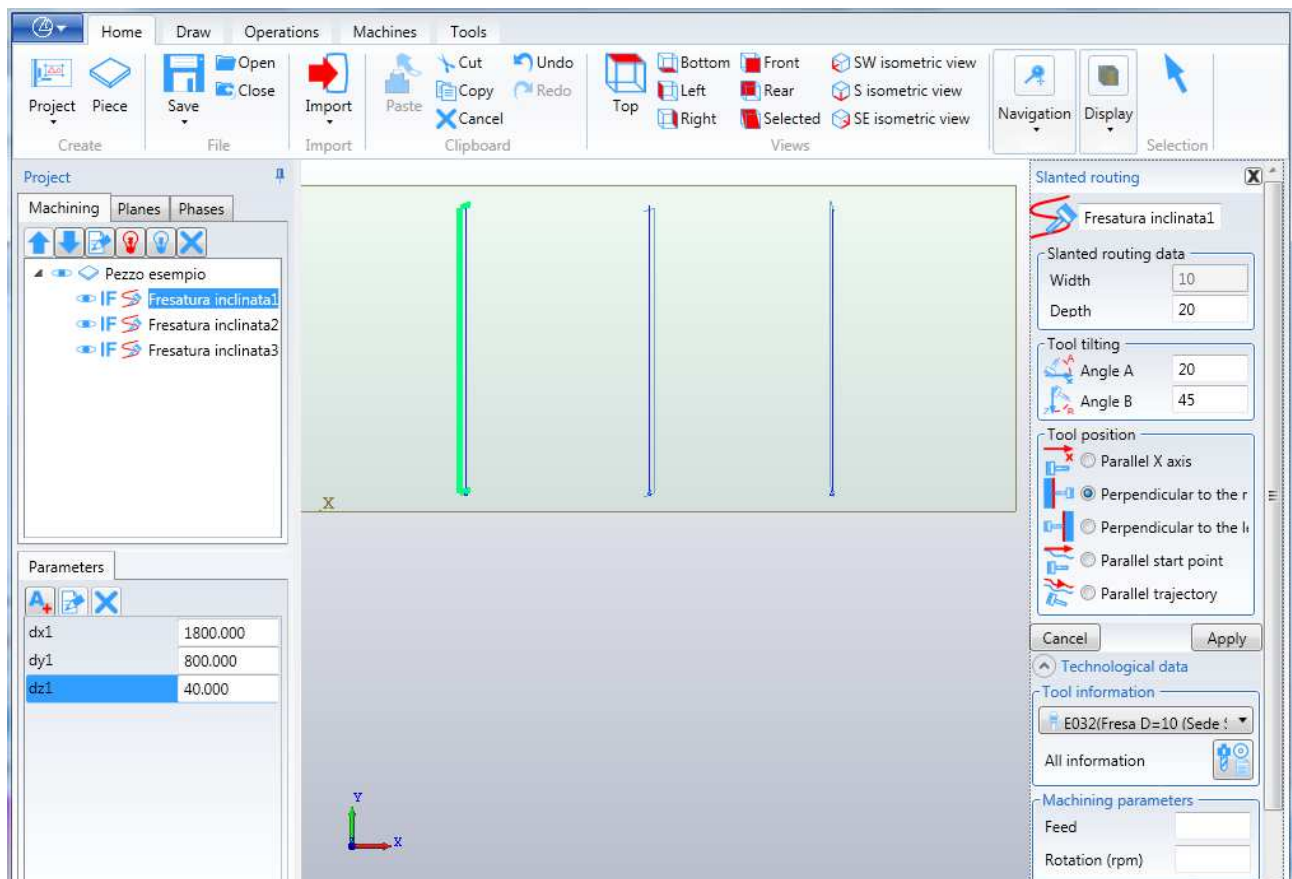
See below for the example code and subsequently the operations are displayed:

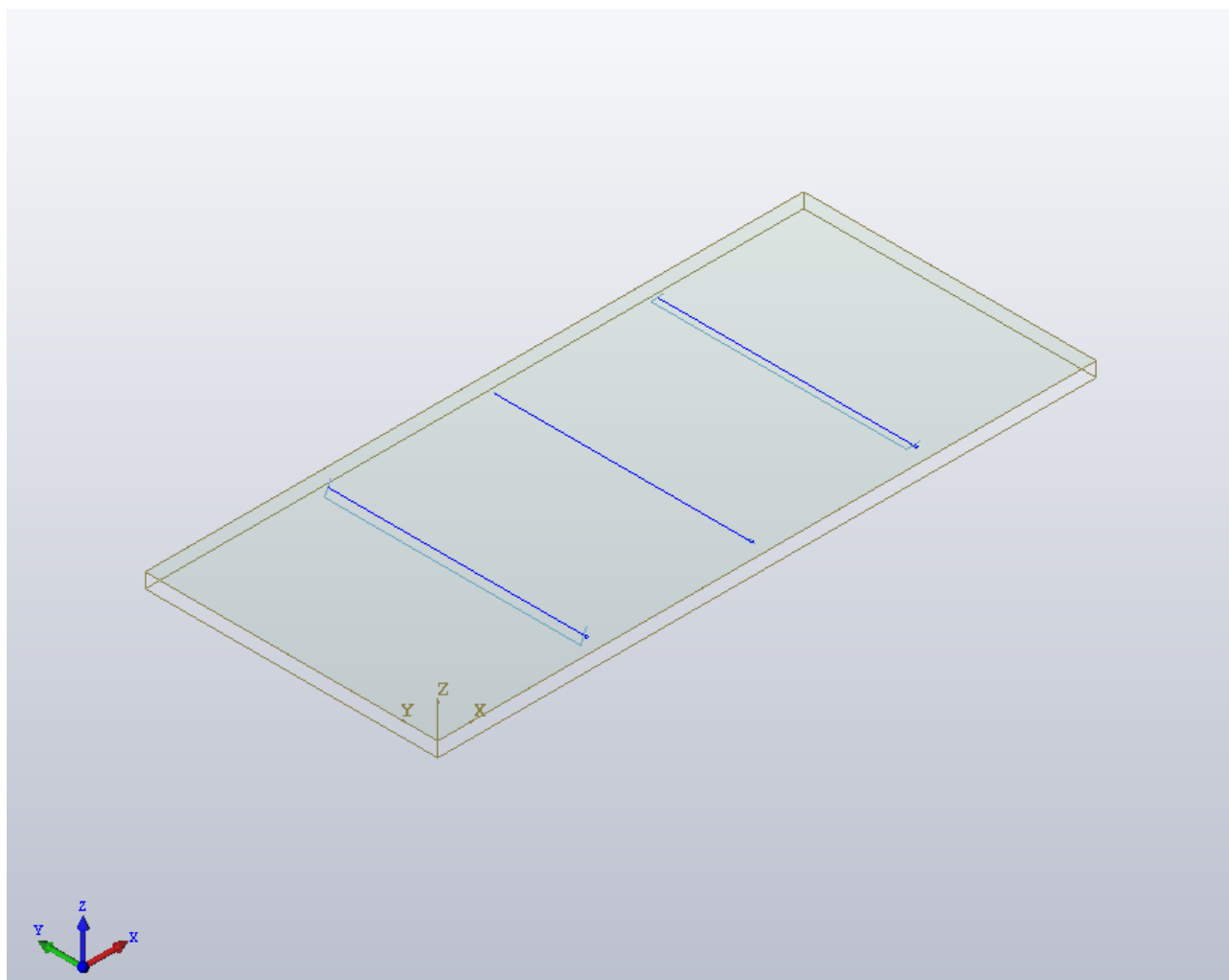
```
// Plane selection
CreateFinishedWorkpieceBox("Example workpiece",1800 800 40);

// Geometry creation
CreateSegment("Segment1", 450, 50, 450, 750);
// Operation creation
CreateSlantedRoughFinish("Slanted routing1", 20, 45, 1, 20, "Slanted routing",
TypeOfProcess.GeneralRouting, "E032", "-1", -1, -1, -1, 0);

// Geometry creation
CreateSegment("Segment2", 900, 50, 900, 750);
// Operation creation
CreateSlantedRoughFinish("Slanted routing2", 20, 45, 2, 20, "Slanted routing",
TypeOfProcess.GeneralRouting, "E032", "-1", -1, -1, -1, 0);

// Geometry creation
CreateSegment("Segment3", 1350, 50, 1350, 750);
// Operation creation
CreateSlantedRoughFinish("Slanted routing3", 20, 45, 3, 20, "Slanted routing",
TypeOfProcess.GeneralRouting, "E032", "-1", -1, -1, -1, 0);
```





3.5.5 Chamfer (CreateChamfer)

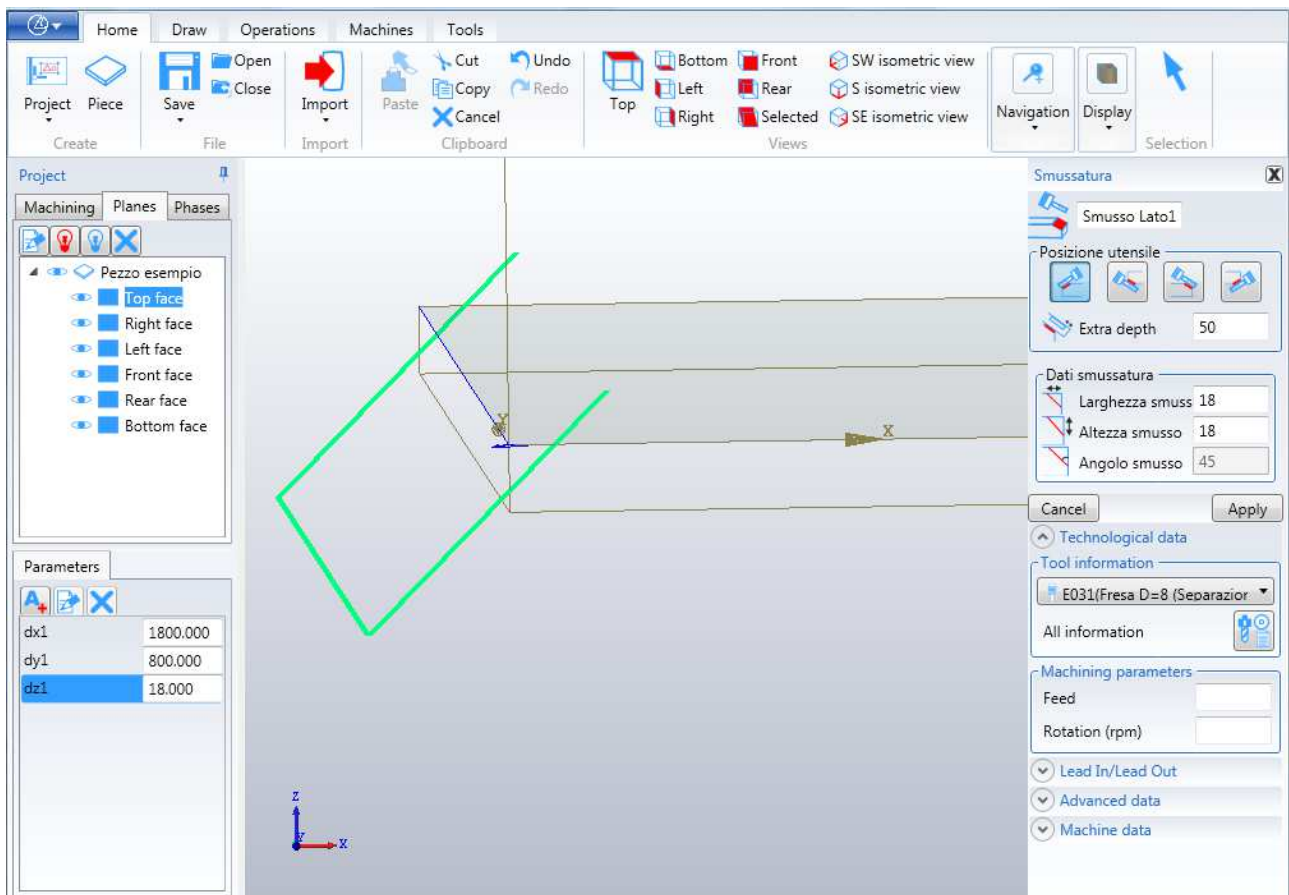
To create a chamfer use the CreateChamfer instruction. The chamfer is always applied to the active geometry.

Name	Type	Description
name	string	Operation name
chamferWidth	double	Chamfer width
chamferHeight	double	Chamfer height
overcutLength	double	Overcut length
toolPosition	int	Tool position in relation to the geometry: 0 = tool on left of geometry on the top plane ; 1 = tool on left of geometry on the bottom plane ; 2 = tool on right of geometry on the top plane ; 3 = tool on right of geometry on the bottom plane ;
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess.GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overMaterial	double (opt def 0)	Overmaterial (the default value is 0)

Table 32: CreateChamfer parameters

See below for a simple chamfer example on the left of the top side of a workpiece. To chamfer the side a segment geometry must be associated. To this geometry the chamfer operation is applied with the CreateChamfer instruction. In the example the corner is chamfered for a width and height equal to 18mm. Please note that for this operation the tool must be on the left of the corner (material being chamfered on the right of the tool).

```
createFinishedWorkpieceBox("Example workpiece", 1800, 800, 18);
CreateSegment("side1", 0, 0, 0, 800);
CreateChamfer("Side1 chamfer", 18, 18, 50, 0, "Chamfer1", TypeOfProcess.GeneralRouting,
"E031", "-1", -1, -1, -1, 0);
```



3.5.6 Routing 3D (Create3DRoughFinish)

To create a 3D routing use the `Create3DRoughFinish` instruction. As opposed to the other operations the 3D routing does not require the definition of an active geometry that it must be applied to, but it requires the definition of the 3D tool path. For the instructions on how to create a tool path see chapter 3.5.13, whilst the input parameters for the `Create3DRoughFinish` function are described below.

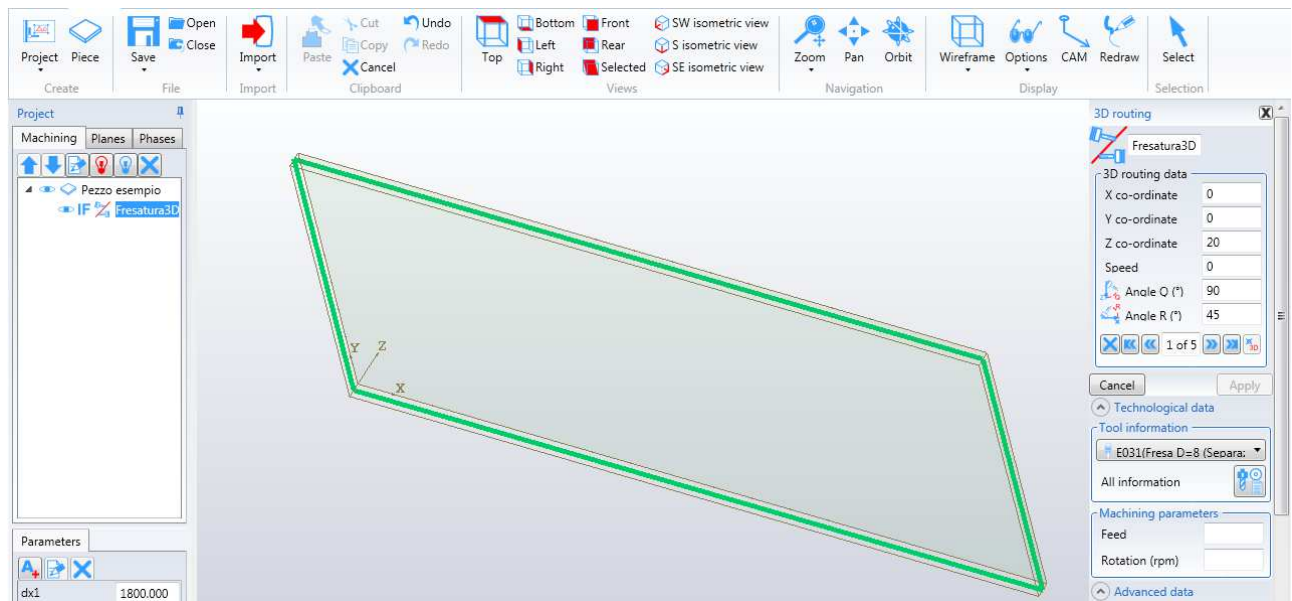
Name	Type	Description
name	string	Operation name
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (not used, Specify TypeOfProcess.GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
inputZRotation	Double (opt)	Rotation angle around axis Z
inputXRotation	double (opt)	Rotation angle around axis X

Table 33: Create3DRoughFinish parameters

The example below shows a 3D routing operation with rotation angle around axis Z equal to 90° and rotation angle around axis X equal to 45°. Note that the tool path that the operation needs to follow must be defined after the `Create3DRoughFinish` instruction.

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 40);
// Operation: 3D routing
Create3DRoughFinish("3D Routing", "3D Routing", TypeOfProcess.GeneralRouting, "E031", "-1", -1, -1, -1, 90, 45);
// 3D Tool path
CreateToolpath3D("3DToolPath", 0, 0, 20);
AddSegmentToToolpath(1800, 0, 20, 90, 45);
AddSegmentToToolpath(1800, 800, 20, 90, 45);
AddSegmentToToolpath(0, 800, 20, 90, 45);
AddSegmentToToolpath(0, 0, 20, 90, 45);
```


See below for a graphic representation of the tool path:



3.5.7 Slot (CreateSlot)

To create a slot operation use the CreateSlot instruction. The operation is always applied to the active geometry that must be a segment type.

Name	Type	Description
name	string	Operation name
depth	double	Slot depth
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess. GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
correction	int (opt def 0)	Tool correction (0=central, 1=left, 2=right. The default value is 0)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overMaterial	double (opt def 0)	Overmaterial (the default value is 0)
angle	double (opt def 90)	Operation angle in relation to the workplane.
endDepth	double (opt)	End depth

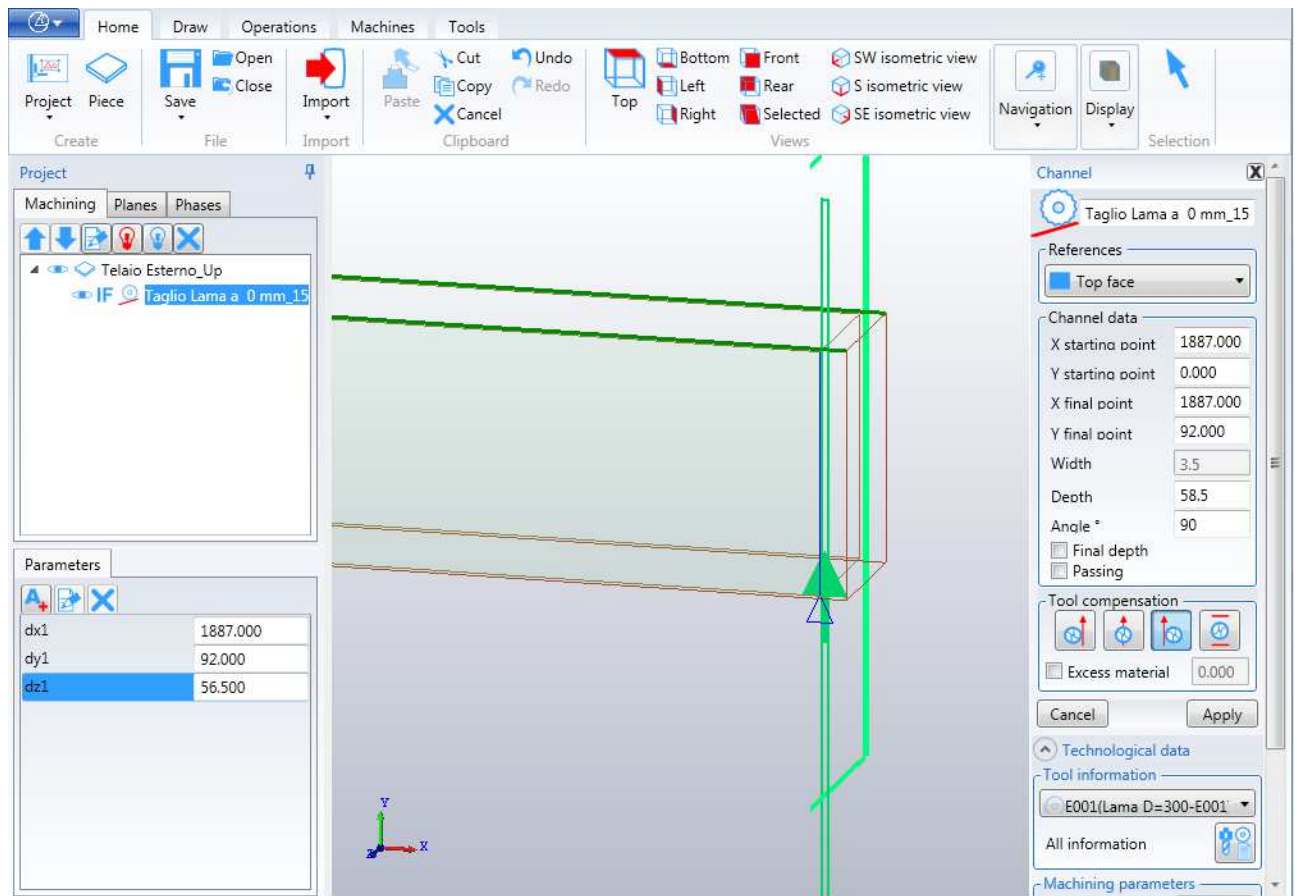
Table 34: CreateSlot parameters

See below for a slot application example. A segment geometry, active geometry for the next operation, is created on the top side of the workpiece. In particular, for the slot operation the following are selected: tool with circular blade, slot depth of 58.5 mm, a tool correction to the right of the geometry, a feed speed of 4 m/min and overmaterial equal to zero. The default values are used for the rotation speed, the input speed and the operation angle.

```
SetMachiningParameters("AD", 1, 21, 1114112, false);
CreateFinishedWorkpieceBox("External Frame_Up",1887,92,56.5);
CreateRawWorkpiece("External Frame_Up",10,10,1,1,0,0);
```

```
//external blocking profile
CreateBlockingProfile("External Frame_Up_2",1897,93,-15);
AddSegmentToBlockingProfile(-10,93, "");
SetWorkpieceSetupPosition(2740,709.5,0, 0);
```

```
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, .5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_14", 1887, 0, 1887, 92);
SetRetractStrategy(true, true, .5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_15",58.5,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting
,"E001","3",2,-1,-1,4,0);
```



3.5.8 Blade cut (CreateBladeCut)

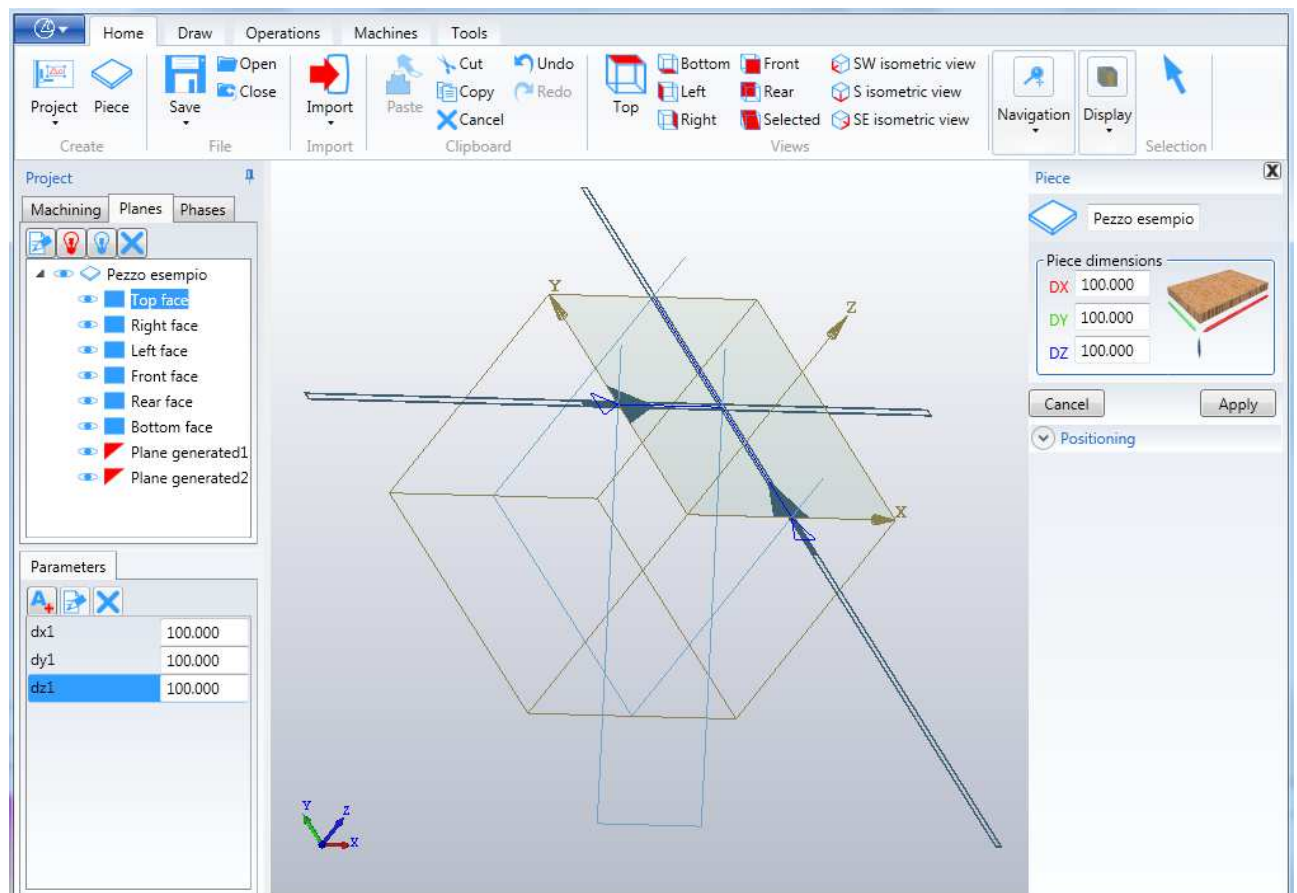
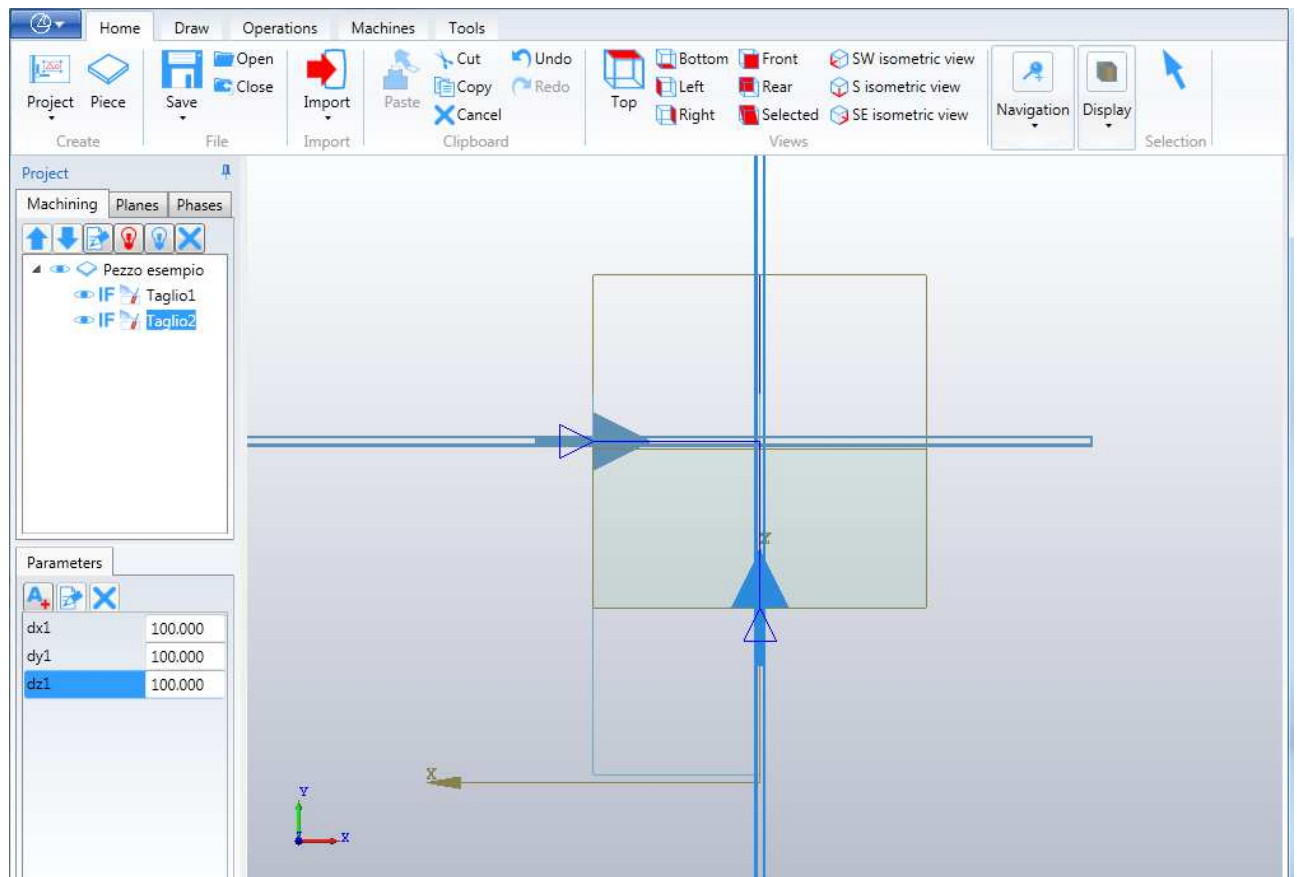
To create a blade cut use the `CreateBladeCut` instruction. The cut operation is always applied to the active geometry that must be a segment type. Please note that the blade cuts can only be applied to segment geometries and on the top side of the workpiece.

Name	Type	Description
name	string	Operation name
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess. GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
angle	double (opt def 90)	Rotation angle
correction	int (opt def 0)	Tool correction (0=central, 1=left, 2=right, 3=depth, 13=left/ depth, 23=right/depth. The default value is 0)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overMaterial	double (opt def 0)	Overmaterial (the default value is 0)
cutPositionUpper	bool (opt def true)	Blade cut reference position (true = upper position reference)
materialPositionLeft	bool (opt def true)	Workpiece position for subsequent operation (true = workpiece on the left of the cut)
referenceOffset	double (opt)	Distance from the blade cut reference position
extraDepth	double (opt)	Extra cut depth

Table 35: CreateBladeCut parameters

The example below shows a cube with dimensions 100mmx100mmx100mm and two blade cuts have been performed along the median directions of the top side of the cube. As the blade cut is always through, remember to use a circular blade tool with radius greater than the workpiece thickness (in this case the circular blade has a diameter of 300mm). Furthermore every blade cut operation is always associated to the creation of a new plane corresponding to the new side created on the workpiece. In the example below the first cut is perpendicular to the operation side (top side), whilst the second cut is slanted by an angle of 45° (slanted blade cut).

```
CreateFinishedWorkpieceBox("Example workpiece", 100, 100, 100);
SelectWorkplane("Top");
CreateSegment("Cut segment1", 50, 0, 50, 100);
CreateBladeCut("Cut1", "Blade Cut", TypeOfProcess.GeneralRouting, "E001", "3", 90, 0, -1, -1, -1, 0, true, true, 0, 0);
CreateSegment("Cut Segment2", 0, 50, 50, 50);
CreateBladeCut("Cut2", "Blade Cut", TypeOfProcess.GeneralRouting, "E001", "3", 45, 0, -1, -1, -1, 0, true, false, 0, 0);
```



3.5.9 Contour (CreateContour)

To create a contour use the `CreateContour` instruction. The contour can be performed on a workpiece or on an active geometry.

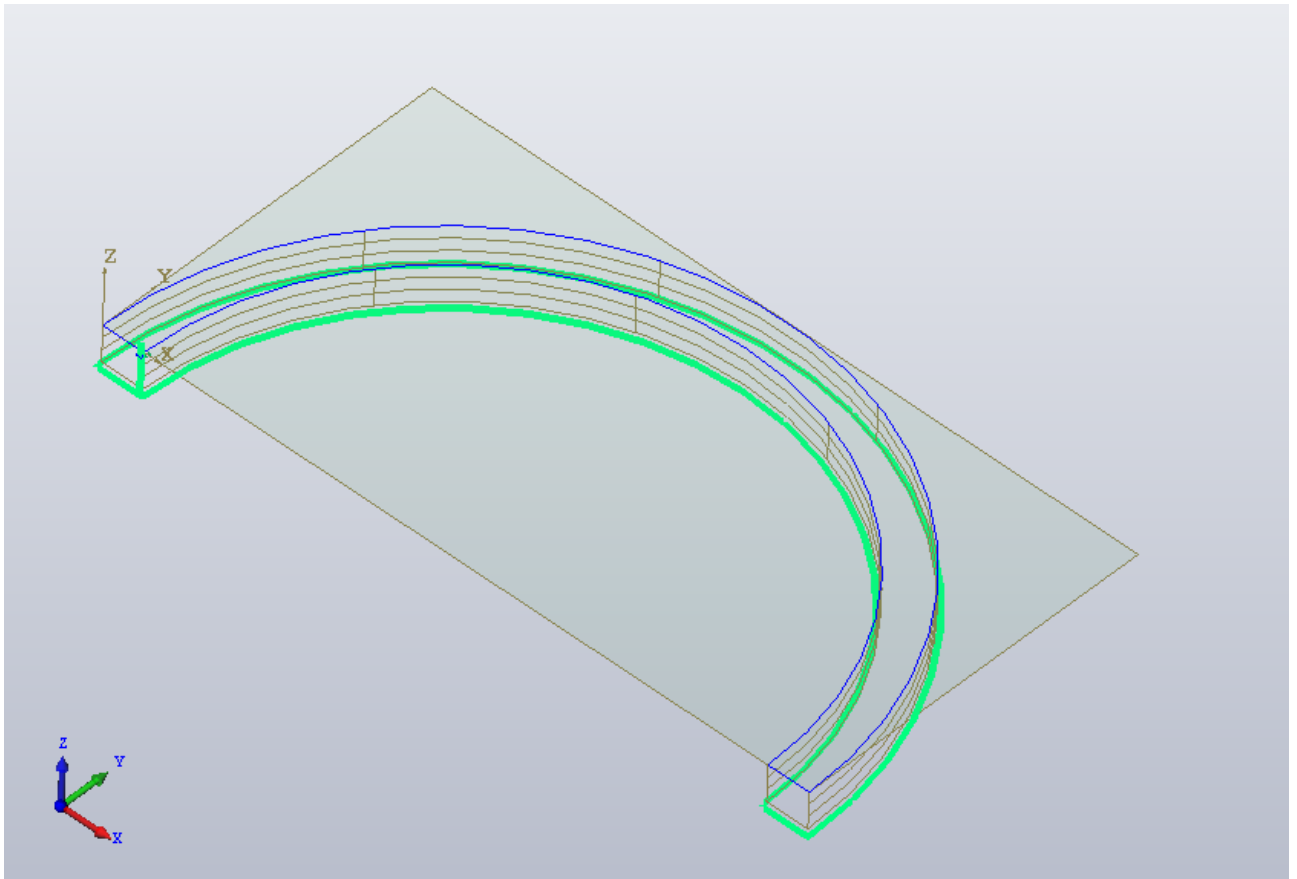
Name	Type	Description
name	string	Operation name
depth	double	Operation depth
typeOfContour	int	Type of contour (0 = workpiece, 1 = selected geometry)
sideOfContour	int	Contour side (0 = internal, 1 = external)
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess. GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
correction	int (opt def 0)	Tool correction (0=central, 1=left, 2=right. The default value is 0)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overMaterial	double (opt def 0)	Overmaterial (the default value is 0)

Table 36: CreateContour parameters

The example below shows the application of a contour operation on a workpiece from extrusion:

```
CreatePolyline("", 70, 600);
AddArc2PointRadiusToPolyline(1130, 600, 530, true, false);
AddSegmentToPolyline(1200, 600);
AddArc2PointRadiusToPolyline(0, 600, 600, false, false);
AddSegmentToPolyline(70, 600);
CreateFinishedWorkpieceFromExtrusion("Frame_Up", 63);

CreateContour("Contour", 70, 0, 1, "Geometry Contour", TypeOfProcess.GeneralRouting, "E032",
"-1", 0, -1, -1, -1, 0);
```



3.5.10 Pocket (CreateContourPocket)

To create a pocket use the `CreateContourPocket` instruction. The pocket is always applied to the active geometry.

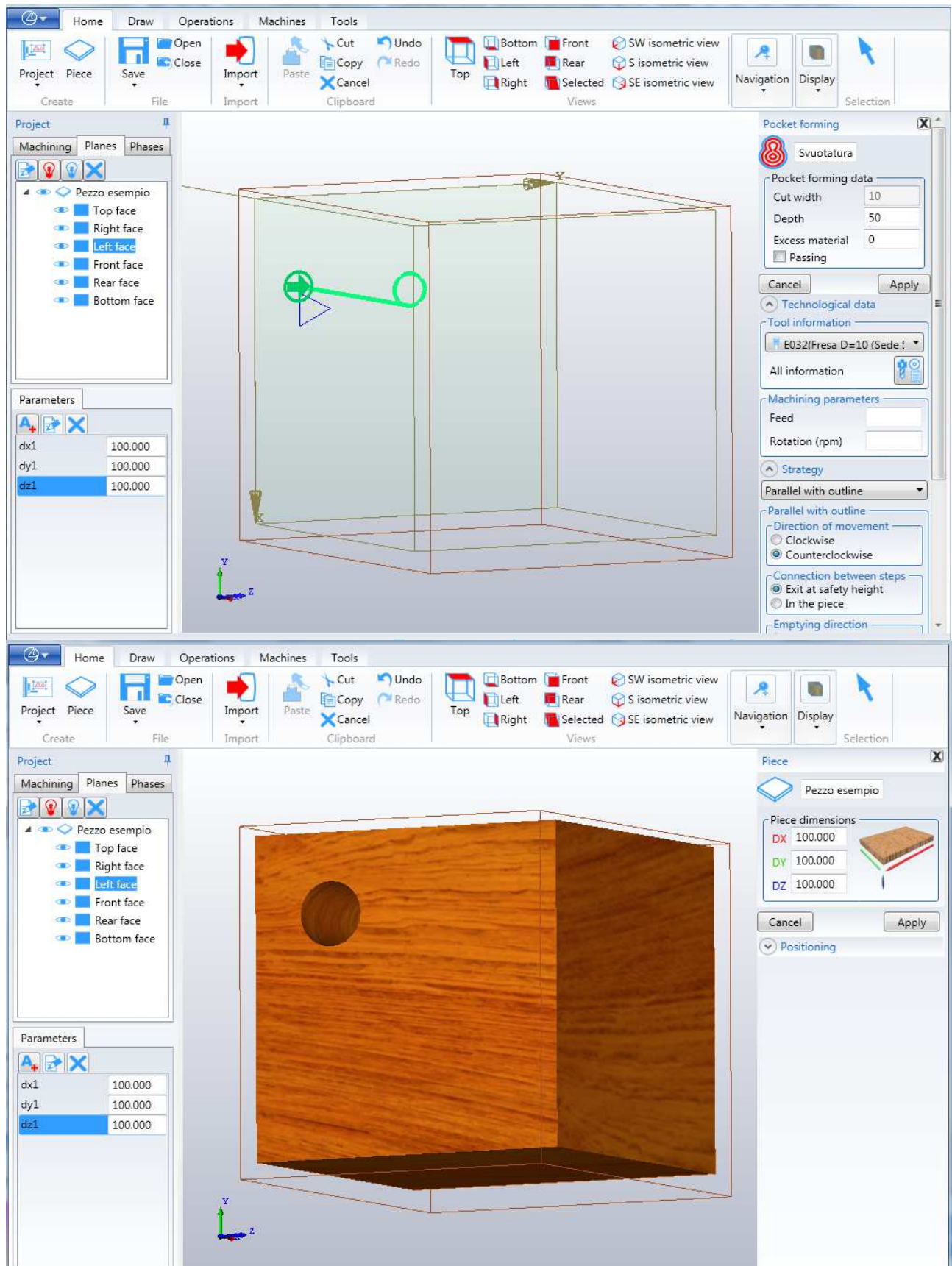
Name	Type	Description
name	string	Operation name
depth	double	Operation depth
description	string	Operation description
typeOfProcess	TypeOfProcess	Type of process (specify TypeOfProcess. GeneralRouting)
tool	String (opt)	Tool name ("-1" for automatic selection)
head	String (opt)	Work head number ("-1" for automatic selection)
inputSpeed	double (opt def -1)	Input speed
rotSpeed	double (opt def -1)	Tool rotation speed
speed	double (opt def -1)	Feed speed
overlap	double (opt def 50)	Overlap percentage
finalPass	bool (opt false)	Final pass application
bossNames	string[]	Name of the geometries that define the bosses

Table 37: CreateContourPocket parameters

The example below shows the application of pocket operation on a circular geometry defined on the left side of a cube with dimensions 100mmX100mmx100mm.

```
CreateFinishedWorkpieceBox("Example workpiece", 100, 100, 100);
CreateRawWorkpiece("Example workpiece",10 ,10 ,6 ,2 ,0 ,0);
SelectWorkpiece("Example workpiece");
SelectWorkplane("Left");
CreateCircleCenterRadius ("Circle", 25, 25, 10, false);
TypeOfProcess.GeneralRouting, "E032", "-1", 0, -1, -1, -1, 0);
CreateContourPocket("Pocket", 50, "Circle pocket", TypeOfProcess.ConcentricalPocket, "E032",
"-1", -1, -1, -1, 30);
```


In the example the pocket depth is of 50 mm, with an overlap percentage of 30%. The default values have been used for the remaining parameters. To perform the pocket correctly the tool diameter must be less than the geometry of the pocket.



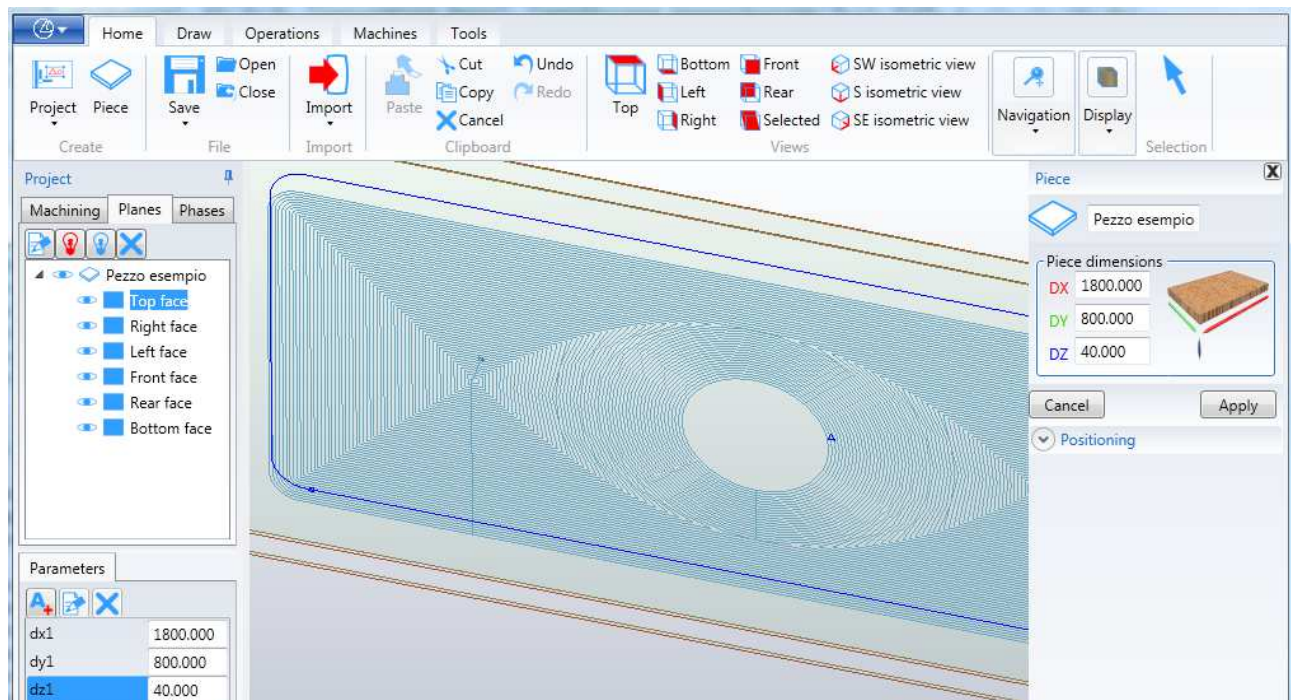
See below for another pocket example, with a boss inside the pocket geometry (a circle in this case):

```
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 40);  
CreateRawWorkpiece("Example workpiece",10 ,10 ,6 ,2 ,0 ,0);
```

```
CreateCircleCenterRadius ("Circle", 900, 400, 100, false);
```

```
CreatePolyline("Pocket Polyline", 250, 100);  
AddArc2PointRadiusToPolyline( 200, 150, 50, true, false);  
AddSegmentToPolyline( 200, 650);  
AddArc2PointRadiusToPolyline( 250, 700, 50, true, false);  
AddSegmentToPolyline( 1550, 700);  
AddArc2PointRadiusToPolyline( 1600, 650, 50, true, false);  
AddSegmentToPolyline( 1600, 150);  
AddArc2PointRadiusToPolyline( 1550, 100, 50, true, false);  
ClosePolyline("Pocket Polyline");
```

```
CreateContourPocket("Pocket", 30, "Circle pocket", TypeOfProcess.ConcentricPocket, "E032",  
"-1", -1, -1, -1, 50, false, "Circle");
```



3.5.11 Machining mode

The machining mode is set with the Strategy, the approach/retraction and any machining inversion.

To set a machining operation strategy, MSL provides various instructions. These instructions must be entered just before the operation instruction that they are applied to. The strategy functions are usually "non-modal", that is the first operation after the configuration uses and "consumes" the setting (for more information see the previous chapter).

The main instructions, divided by operation category that they can be applied to, are described below.

3.5.11.1 Drilling strategies

The following instructions are provided for the drilling operation strategy:

- `CreateSingleStepDrillingStrategy`: configures a single step drilling strategy;
- `CreateMultiStepDrillingStrategy`: configures a multiple step drilling strategy.

As the first instruction does not include input parameters. The parameters for the second function are listed in the table below.

Name	Type	Description
<code>isStepDepth</code>	<code>bool</code>	Enables the use of the next parameters: true = use of <code>stepDepth</code> ; false = use of <code>stepNumber</code> .
<code>stepNumber</code>	<code>int</code> (opt def 1)	Number of steps
<code>stepDepth</code>	<code>double</code> (opt def 0.0)	Depth of each step

Table 38: CreateMultiStepDrillingStrategy parameters

The example below shows the application of the drilling strategy in the production of a frame for doors:

```
SetMachiningParameters("AB", 1, 0, 196608, false);
CreateFinishedWorkpieceBox("Frame_Right",1200,70,63);
CreateRawWorkpiece("Frame_Right",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Right_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,2,0, 0);

... (continue)

SetPneumaticHoodPosition(0);
ResetAuxiliaryHood();
CreateWorkplane("Frame Drilling 63_13",17,0,0,0,90);
CreateMultiStepDrillingStrategy( true, 1, 44);
```

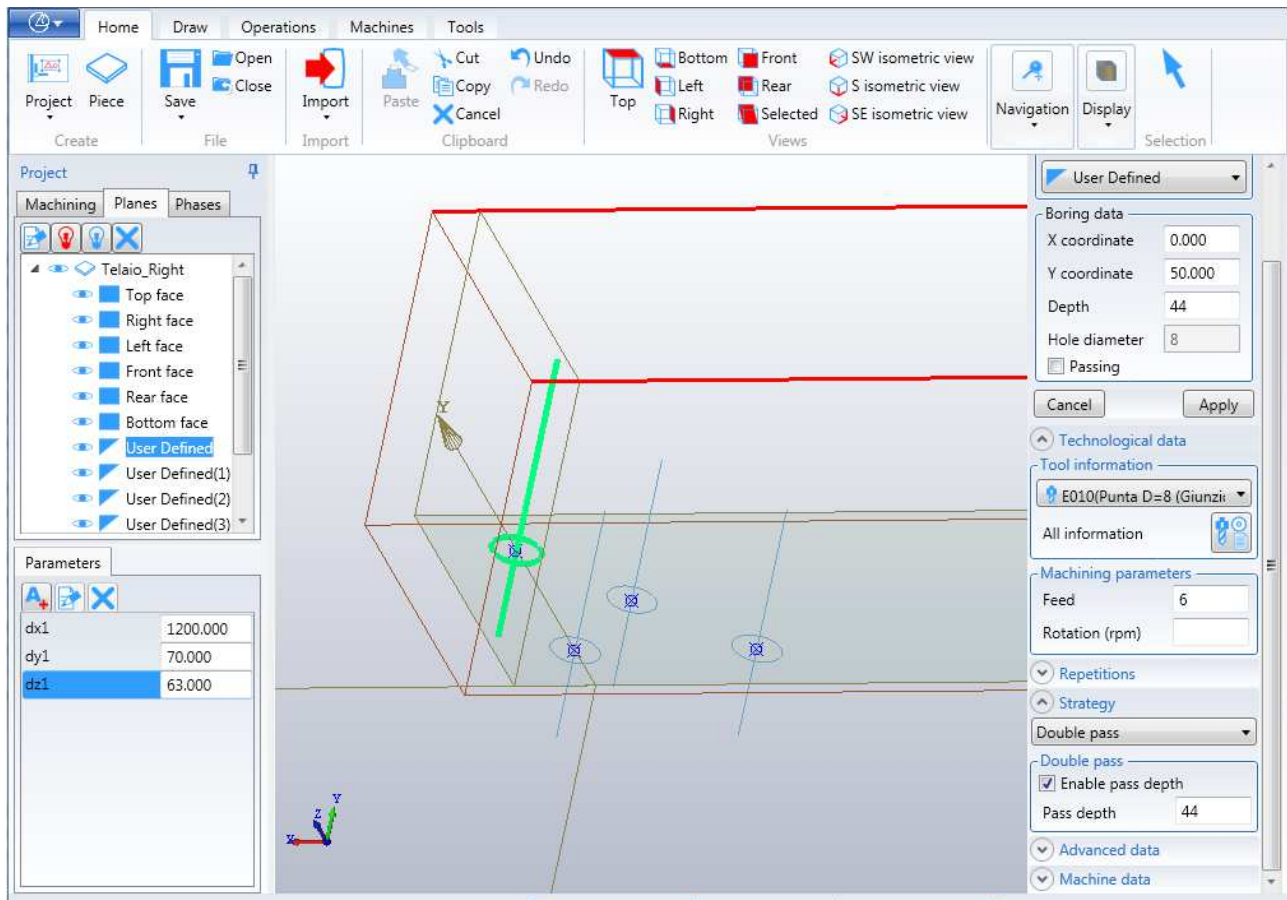
```

CreateDrill("Frame Drilling 63_14",0 ,50 ,44,10, "Frame Drilling 63",
TypeOfProcess.Drilling,"E011","3",0,0, 6);
CreateWorkplane("Frame Drilling 63_15",35,0,0,0,90);
CreateMultiStepDrillingStrategy( true, 1, 32);
CreateDrill("Frame Drilling 63_16",0 ,31.5 ,32,10, "Frame Drilling 63",
TypeOfProcess.Drilling,"E011","3",0,0, 6);
CreateWorkplane("Frame Drilling 63_17",17,0,0,0,90);
CreateMultiStepDrillingStrategy( true, 1, 32);
CreateDrill("Frame Drilling 63_18",0 ,13 ,32,10, "Frame Drilling 63",
TypeOfProcess.Drilling,"E011","3",0,0, 6);
CreateWorkplane("Frame Drilling 63_19",55,0,0,0,90);
CreateMultiStepDrillingStrategy( true, 1, 32);
CreateDrill("Frame Drilling 63_20",0 ,13 ,32,10, "Frame Drilling 63",
TypeOfProcess.Drilling,"E011","3",0,0, 6);
SelectWorkplane("Top");

```

... (continue)

Four drilling operations are performed in the example. All the operations use a multiple step drilling strategy, with step depth as active parameter. Note that even though three holes have the same strategy, the same configuration is repeated for each hole. This is because the strategy definition in the script is “non-modal” and therefore the instruction is valid only for the first operation after the setting. Once the configuration is used it is no longer available for the subsequent drilling. The instruction must be repeated to use the same strategy again.



3.5.11.2 Routing strategies

The following instructions can be used to apply a strategy to the routing operation:

- **CreateBidirectionalMillingStrategy**: configures the strategy for routing with bidirectional multiple passages;
- **CreateUnidirectionalMillingStrategy**: configures the strategy for routing with unidirectional multiple passes;
- **CreateHelicMillingStrategy**: configures the strategy for routing with helic tool path;

Please note that for all the instructions, the strategy created is valid only for the first operation performed after it is created.

See below for the descriptions of the input parameters for the three instructions.

Name	Type	Description
allowMultiplePasses	bool	Multiple pass roughing: true = standard roughing with multiple passages (different layers of material are removed in sequence, complying with the maximum cutting depth); false = special roughing with a single pass (for prefabricated)
cuttingDepth	double	Passage depth in the tool axial direction.
finishCuttingDepth	double	Final passage depth in the tool axial direction (if used corresponds to the final pass, used to reach the final programmed depth).

Table 39: CreateBidirectionalMillingStrategy parameters

Name	Type	Description
allowMultiplePasses	bool	Multiple pass roughing: true = standard roughing with multiple passages (different layers of material are removed in sequence, complying with the maximum cutting depth); false = special roughing with a single pass (for prefabricated)
cuttingDepth	double	Passage depth in the tool axial direction.
finishCuttingDepth	double	Final passage depth in the tool axial direction (if used corresponds to the final pass, used to reach the final programmed depth).
strokeConnectionType	int (opt def 1)	Tool movement between two passages: 0 = tool remains inside the workpiece and moves to the start point of the next pass; 1 = tool exits the workpiece and moves quickly to the start point of the next pass;

Table 40: CreateUnidirectionalMillingStrategy parameters

Name	Type	Description
cuttingDepth	double	Passage depth in the tool axial direction.
allowsFinishCutting	bool	Enables the execution of the final pass
finishCuttingDepth	double	Final passage depth in the tool axial direction (if used corresponds to the final pass, used to reach the final programmed depth).

Table 41: CreateHelicMillingStrategy parameters

The example below shows the application of the CreateUnidirectionalMillingStrategy, instruction used in a script to produce:

```
CreatePolyline("Squad_36",1323.5,878.5);
AddSegmentToPolyline(1323.5,105);
AddSegmentToPolyline(105,105);
AddSegmentToPolyline(105,878.5);
AddSegmentToPolyline(1323.5,878.5);
CreatePolyline("Squad_37",1428.5,983.5);
AddSegmentToPolyline(1428.50,0);
AddSegmentToPolyline(0,0);
AddSegmentToPolyline(983.5);
AddSegmentToPolyline(1428.50,983.5);
CreateFinishedWorkpieceFromExtrusion("Squad_40",78,"Squad_36");

... (continue)

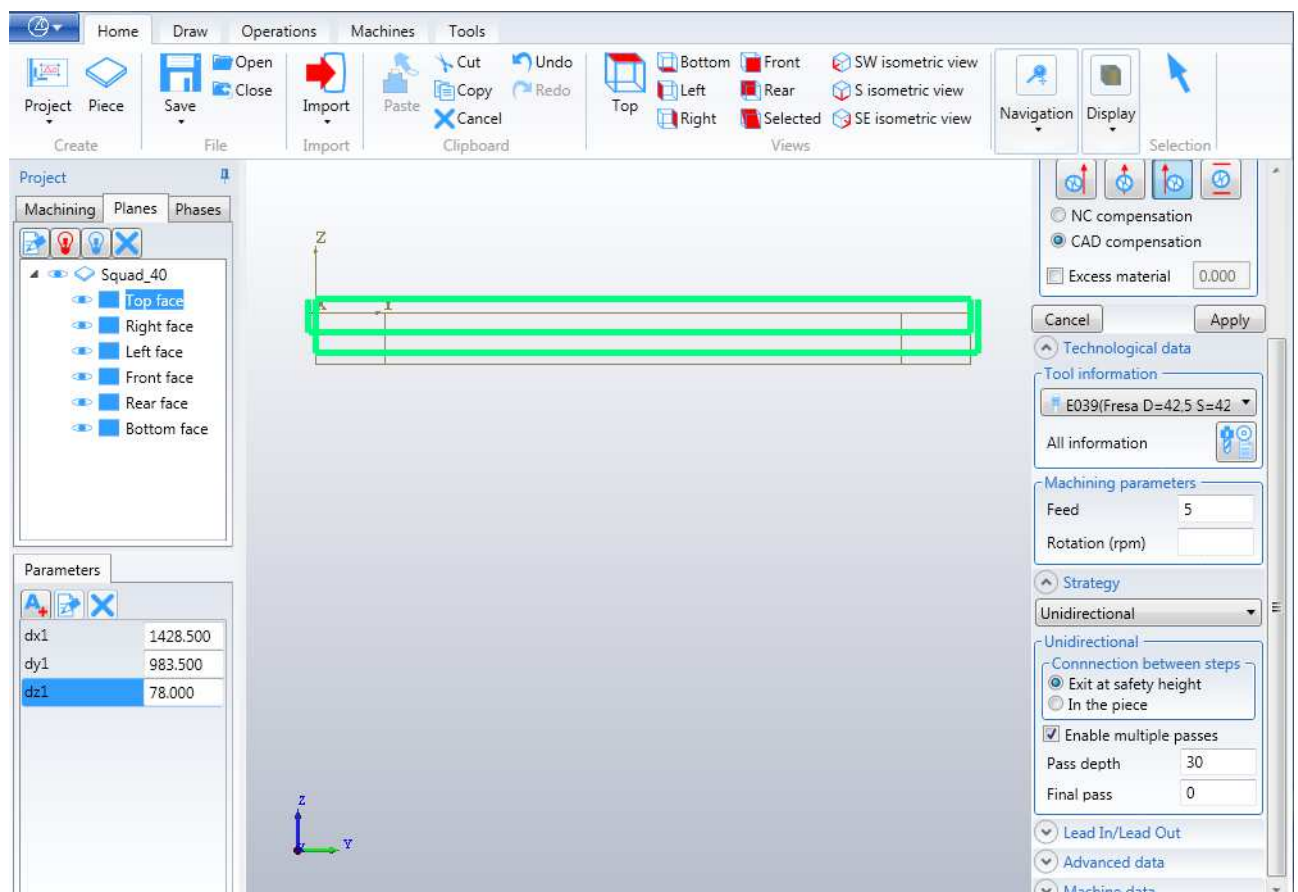
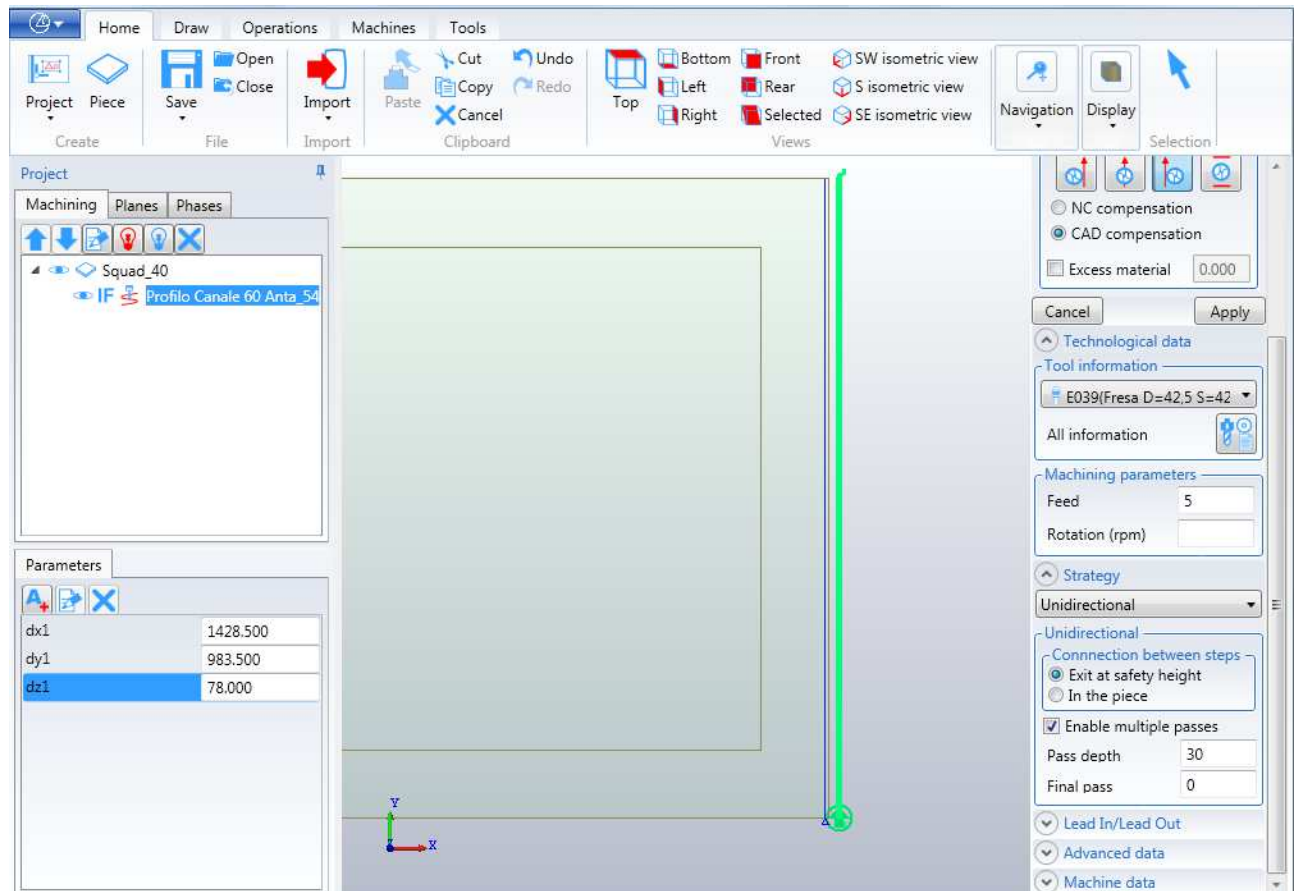
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
ResetDustpan();
CreatePolyline("Slot Profile 60 Door_53",1422.5,0);

AddSegmentToPolyline(1422.50,983.5);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateUnidirectionalMillingStrategy(true, 30, 0, 1);
CreateRoughFinish("Slot Profile 60 Door_54", 60,"Slot Profile 60 Door",
TypeOfProcess.GeneralRouting ,"E039","3",2,-1,-1,5,0);

... (continue)
```

In the routing example a unidirectional strategy has been selected with multiple passes, pass depth of 30mm (with final pass of 0mm) and with tool exit from workpiece for the movement between the final point of the previous pass and the start point of the next pass. As the routing depth is of 60mm, the strategy of 30mm per pass will achieve the desired depth with two passes.

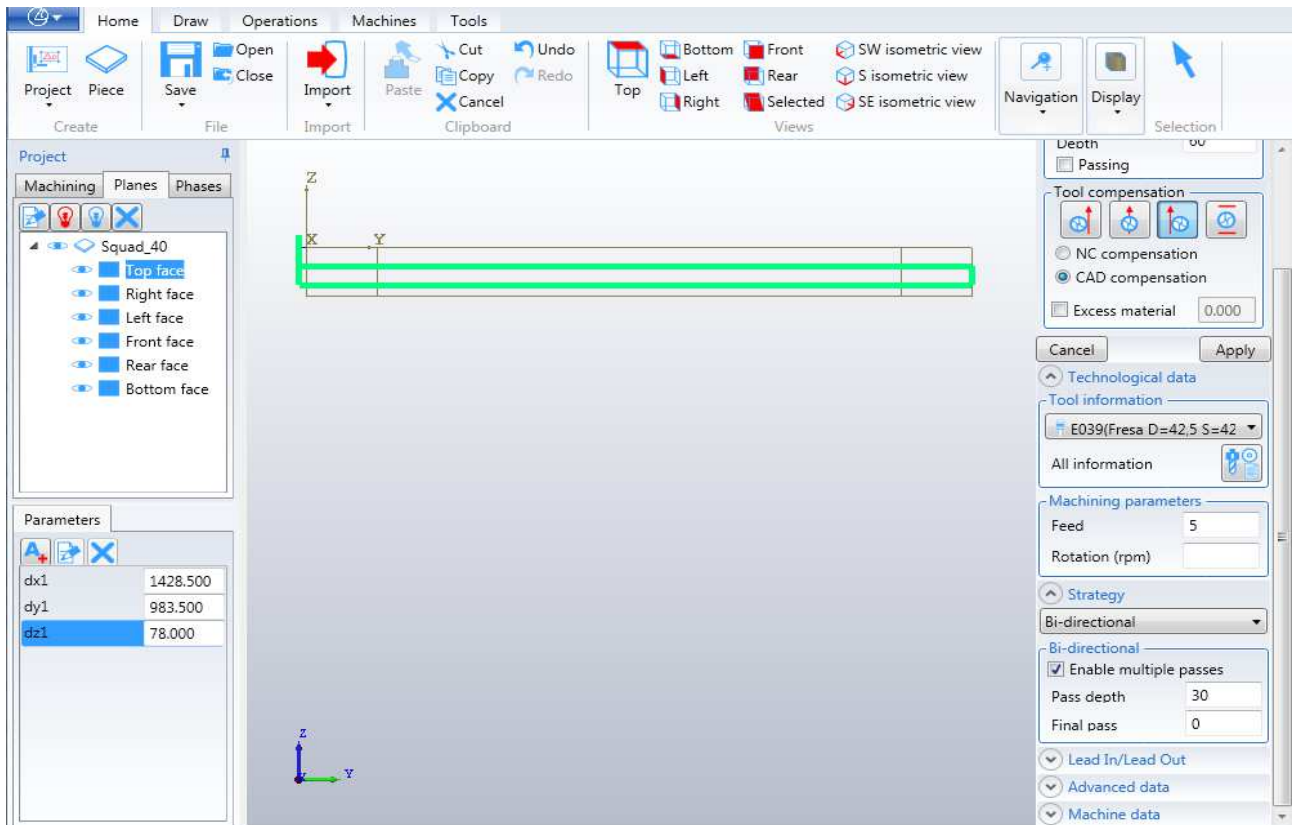
See below for the images of the operation with views of the top and right sides.



In order to repeat the previous operation with a different strategy, for example by using a bidirectional approach, replace the unidirectional instruction with the bidirectional one, `CreateBidirectionalMillingStrategy`, keeping the same input parameters, to achieve:

```
CreateBidirectionalMillingStrategy(true, 30, 0);
```

See below for the CAM view of the right side, for the new strategy.

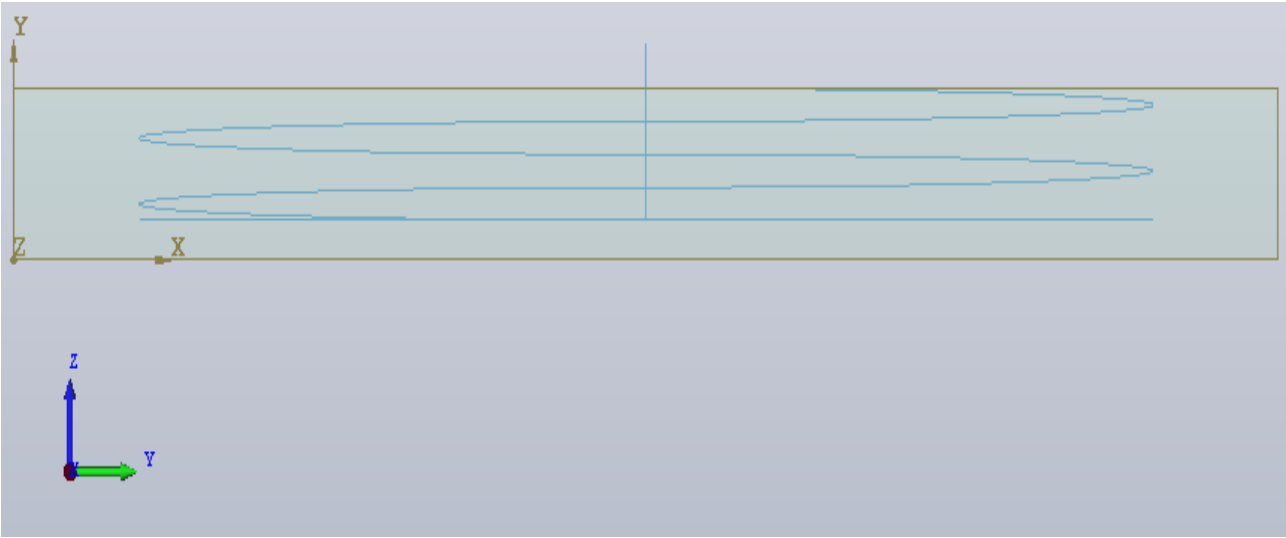


As can be seen in the image, in this case the tool does not exit the workpiece for the repositioning on the left side. With a bidirectional operation strategy, after the first pass the tool performs the second pass along the trajectory in reverse.

For an example of helic routing strategy apply the `CreateHelicMillingStrategy` instruction to an operation with circle geometry:

```
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 78);
CreateCircleCenterRadius ("Example circle", 1000, 400, 300, false);
CreateHelicMillingStrategy(30, true, 0);
CreateRoughFinish("Routing",60,"", TypeOfProcess.GeneralRouting , "E039", "3", 2, -1, -1, 5, 0);
```


See below for the CAM view of the right side of the workpiece, and the helic path used by the tool for the desired operation.



3.5.11.3 3D routing strategies

To set a 3D routing operation strategy use the `CreatePlaneCutterLocationStrategy` instruction. The input parameters for the instruction are described below.

Name	Type	Description
zRotation	double	Tool rotation direction around axis Z
xRotation	double	Tool rotation direction around axis X

Table 42: CreatePlaneCutterLocationStrategy parameters

By way of an example the strategy instruction is applied to the following 3D operation:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 40);
CreatePlaneCutterLocationStrategy(45, 30);
SetWorkpieceSetupPosition(10,435,0, 0);

Create3DRoughFinish("Routing 3D", "Routing 3D Example", TypeOfProcess.GeneralRouting, "E030", "-1");

CreateToolpath3D("3DToolPath", 0, 0, 20);
AddSegmentToToolpath(1800, 0, 20, 45, 30);
AddSegmentToToolpath(1800, 800, 20, 45, 30);
AddSegmentToToolpath(0, 800, 20, 45, 30);
AddSegmentToToolpath(0, 0, 20, 45, 30);
```

Comparing the post-process files generated without and with the strategy, we can see that the tool rotation information in relation to axes X and Z are present in the ISO only in the former case. When the strategy is applied, the operation is performed with the vertical tool axis (3-axes operation).

Post-process file without the use of the strategy:

```
H DX=1810.000 DY=1235.000 DZ=40.000 -AB C=0 T=1114112 R=1 *MM /"SerrSCM" BX=0.000 BY=0.000
BZ=0.000 V=0
;
; *****
; >> Post P. Release...
;Version : 1.00.006.1006;
; >> Program description...
; Creation date: 4/18/2013 3:28:34 PM
;
; >> Tools used...
;
; >> Program Start...
; *****
;Workpiece setup Example workpiece on Setup phase
REF DX=1800.000 DY=800.000 DZ=40.000 FLD=AB BX=10.000 BY=435.000 BZ=0.000
; *****
; *****
.OP1 ;Routing 3D
; *****
SET ZFAST=20.000
XG03D X=0.000 Y=0.000 H=20.000 T=130 F=1 C=0 Q=0.000 R=0.000 E=0 ;Example Routing 3D
XG13D X=1800.000 Y=0.000 H=20.000 Q=45.000 R=30.000 ;Example Routing 3D
XG13D X=1800.000 Y=800.000 H=20.000 Q=45.000 R=30.000 ;Example Routing 3D
XG13D X=0.000 Y=800.000 H=20.000 Q=45.000 R=30.000 ;Example Routing 3D
```

```

XG13D X=0.000 Y=0.000 H=20.000 Q=45.000 R=30.000 ;Example Routing 3D
PARK S=0
;*****
.END
;*****
F=1
O X=0 Y=0 Z=0 F=1 ;ChangePlane
;FINEPROG

```

Post-process file when using the CreatePlaneCutterLocationStrategy strategy:

```

H DX=1810.000 DY=1235.000 DZ=40.000 -AB C=0 T=1114112 R=1 *MM /"SerrSCM" BX=0.000 BY=0.000
BZ=0.000 V=0
;*****
; >> Post P. Release...
;Version : 1.00.006.1006;
; >> Program description...
; Creation date: 4/18/2013 3:26:28 PM
;
; >> Tools used...
;
; >> Program Start...
;*****
;Workpiece setup Example workpiece on Setup phase
REF DX=1800.000 DY=800.000 DZ=40.000 FLD=AB BX=10.000 BY=435.000 BZ=0.000
;*****
;*****
.OP1 ;Routing 3D
;*****
SET ZFAST=20.000
XG0 X=0.000 Y=0.000 Z=0.000 T=130 P=0 D=20 C=0 E=0 ;Example Routing 3D
XL2P X=1690.292 Y=-7.559 Z=-619.076 ;Example Routing 3D
XL2P X=1797.197 Y=733.097 Z=-336.233 ;Example Routing 3D
XL2P X=113.451 Y=733.097 Z=300.163 ;Example Routing 3D
XL2P X=6.547 Y=-7.559 Z=17.321 ;Example Routing 3D
PARK S=0
;*****
.END
;*****
F=1
O X=0 Y=0 Z=0 F=1 ;ChangePlane
;FINEPROG

```

3.5.11.4 Blade cut strategies

To apply a strategy to the blade cut operation use the `CreateSectioningMillingStrategy` instruction. The input parameters for the function are described below.

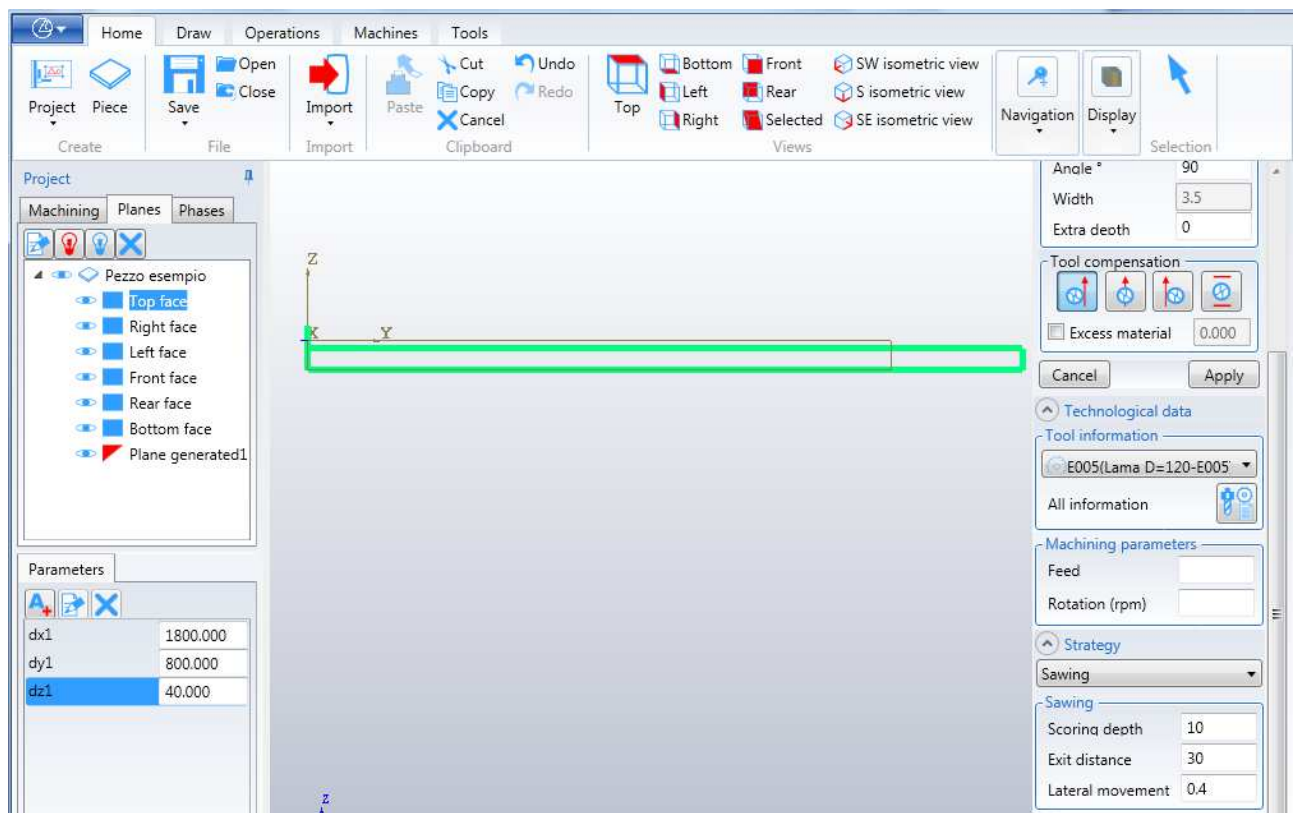
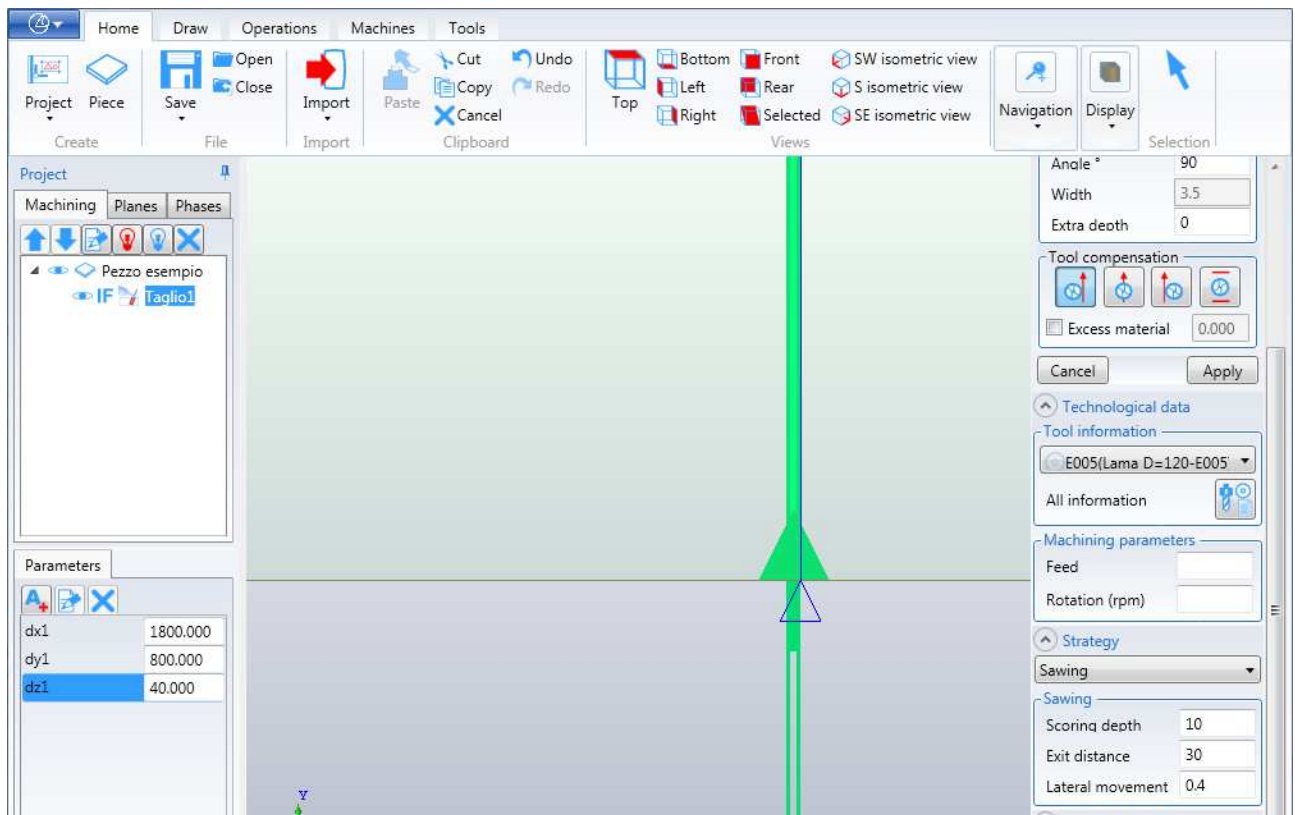
Name	Type	Description
firstCutDepth	double	Blade cut depth (first cut).
outDistance	double	Out distance from the edge of the workpiece, that the blade must consider as cut result (referred to the blade centre).
sideMovingDistance	double (opt def 0)	Movement at right angles to the programmed linear trajectory, based on the current tool correction (positive value for outside movement and negative for inside).

Table 43: CreateSectioningMillingStrategy parameters

The example below shows the strategy application to a blade cut operation. For the operation a first cut depth of 10mm is used, with workpiece edge out distance of 30mm and a blade movement for the tool correction equal to 0.4. Please note that the edge out distance refers to the blade centre and not the blade edge.

```
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 40);
SelectWorkplane("Top");
CreateSegment("Cut segment1", 900, 0, 900, 800);
CreateSectioningMillingStrategy(10, 30, 0.4);
CreateBladeCut("Cut1", "Blade Cut", TypeOfProcess.GeneralRouting, "E005", "-1", 90, 1, -1, -1, -1, 0, true, true, 0, 0);
```

See below for the relative CAM views of the top and right sides of the workpiece. In the first we can see the tool correction, whilst in the second we can see the blade path. In the latter we can see the first cut and the blade exit from the workpiece edge, as specified in the configured strategy.



3.5.11.5 Pocket strategies

To set a pocket strategy use the `CreateContourParallelStrategy` instruction. The input parameters for the instruction are described below.

Name	Type	Description
<code>insideToOutSide</code>	<code>int (opt def true)</code>	Operation direction: true = operation performed from inside to outside; false = operation performed from outside to inside;
<code>rotationDirection</code>	<code>int (opt def 1)</code>	Operation rotation direction: 0 = clockwise; 1 = counterclockwise.
<code>allowMultiplePasses</code>	<code>bool (opt def false)</code>	Multiple pass roughing: true = standard roughing with multiple passages (different layers of material are removed in sequence, complying with the maximum cutting depth); false = special roughing with a single pass (for prefabricated)
<code>cuttingDepth</code>	<code>double (opt def 0)</code>	Passage depth in the tool axial direction.
<code>finishCuttingDepth</code>	<code>double (opt def 0)</code>	Final passage depth in the tool axial direction (if used corresponds to the final pass, used to reach the final programmed depth).
<code>strokeConnectionType</code>	<code>int (opt def 1)</code>	Tool movement between two passages: 0 = tool remains inside the workpiece and moves to the start point of the next pass; 1 = tool exits the workpiece and moves quickly to the start point of the next pass;
<code>isHelicStrategy</code>	<code>bool (opt def false)</code>	Enables the helic strategy

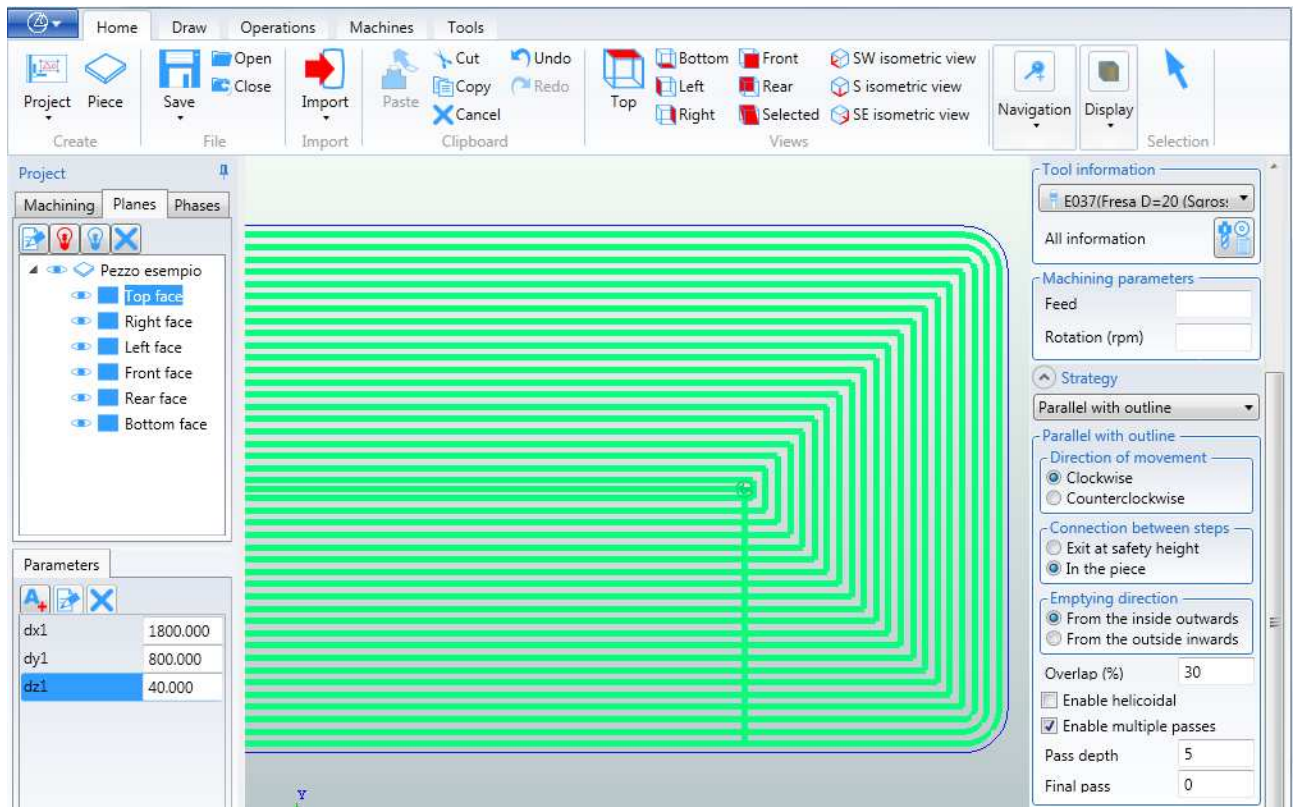
Table 44: CreateContourParallelStrategy parameters

The example below shows the application of a pocket strategy for a slot:

```
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 40);
//Slot geometry creation
CreatePolyline("Slot Geometry", 250, 100);
AddArc2PointRadiusToPolyline( 200, 150, 50, true, false);
AddSegmentToPolyline( 200, 650);
AddArc2PointRadiusToPolyline( 250, 700, 50, true, false);
AddSegmentToPolyline( 1550, 700);
AddArc2PointRadiusToPolyline( 1600, 650, 50, true, false);
AddSegmentToPolyline( 1600, 150);
AddArc2PointRadiusToPolyline( 1550, 100, 50, true, false);
ClosePolyline("Slot Geometry");

//Strategy application
CreateContourParallelStrategy(true, 0, true, 5, 0, 0);
//Creation of the pocket strategy
CreateContourPocket("Slot", 30, "slot creation operation", TypeOfProcess.ConcentricPocket,
"E037", "-1", -1, -1, -1, 30);
```

In the example the geometry pocket is performed from inside to outside, with clockwise rotation direction. The pocket mode is also performed in multiple passes, with pass depth of 5 mm. The tool is not lifted for the connection between the passes. The default values are used for the other parameters. See below for the tool operation path, in CAM view.



3.5.11.6 Approach/Retraction

To apply a tool approach and retraction to the workpiece for routing operations, MSL provides the following instructions:

- **SetApproachStrategy**: used to configure the tool approach specifications to the operation surface (approach);
- **SetRetractStrategy**: used to configure the tool retraction specifications from the operation surface (retraction);

To disable the approach and retraction modes, use the following instructions:

- **ResetApproachStrategy**: disables the active approach mode;
- **ResetRetractStrategy**: disables the active retraction mode;

As opposed to the strategies described previously, these instructions are not linked to specific operations, but are more general as they describe the approach and retraction modes to the workpiece for generic routing operations.

See below for the descriptions of the input parameters for the two instructions. The disable functions do not require any input parameter.

Name	Type	Description
isLinear	bool	Type of approach: true = linear approach; false = circle arc approach.
isQuote	bool	Type of approach start: true = in quota, from the start point; false = in downstroke, from security height along Z.
Distance	double	Radius multiplier for segment/approach arc segment (distance = tool radius multiplier).
Speed	double (opt)	Segment/ approach arc speed

Table 45: SetApproachStrategy parameters

Name	Type	Description
isLinear	bool	Type of retraction: true = linear retraction; false = circle arc retraction.
isQuote	bool	Type of retraction start: true = in quota, from the start point; false = in upstroke, from security height along Z.
distance	double	Radius multiplier for segment/retraction arc segment (distance = tool radius multiplier).
overlapLength	double	Profile overlap length
speed	double (opt)	Segment/ retraction arc speed

Table 46: SetRetractStrategy parameters

The example below shows the application of the approach and retraction to the workpiece in the production of a frame for doors: In particular, the approach and retraction strategies are configured for four operations: two slot and two generic routing operations. To create the slots a linear type of approach has been set, in quota on the operation start point and with radius multiplier value of 0.5 (approach segment distance = radius * multiplier). The same specifications have been set for the retraction, with a null overlap on the profile. For the routings an arc approach has been set, in quota on the operation start point, with radius multiplier of 3. The same specifications have been used for the retraction with null overlap on the profile.

The instructions concerned are highlighted in bold:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0, 0);

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, .5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_30", 1200, 0, 1200, 70);
SetRetractStrategy(true, true, .5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_31",65,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting
,"E001","3",2,-1,-1.6,0);

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, .5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_34", 0, 70, 0, 0);
SetRetractStrategy(true, true, .5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_35",65,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting
,"E001","3",2,-1,-1.6,0);

... (continue)

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetDustpanOffset(10, 1, false);
CreatePolyline("Door Housing Routing_4",0,0);
AddSegmentToPolyline(1200.0);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateRoughFinish("Door Housing Routing_5",63,"Door Housing Routing",
TypeOfProcess.GeneralRouting ,"E152","3",2,-1,-1.6,0);

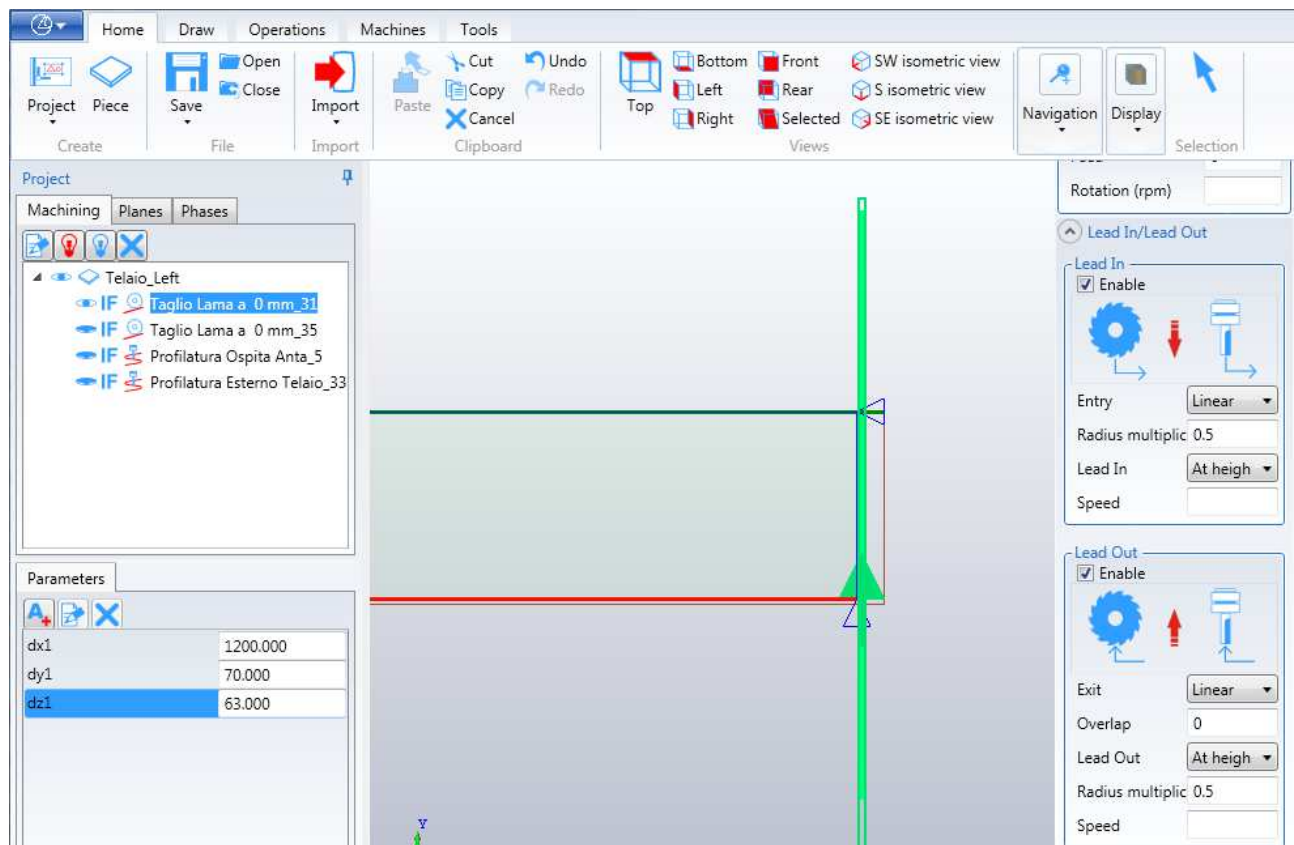
CreateWorkplan("Swap");
SetWorkpieceSetupPosition(10,435,0, 0);
//internal blocking profile
CreateBlockingProfile("Frame_Left_3",0,0, -17);
AddSegmentToBlockingProfile(1200.0, "");
```

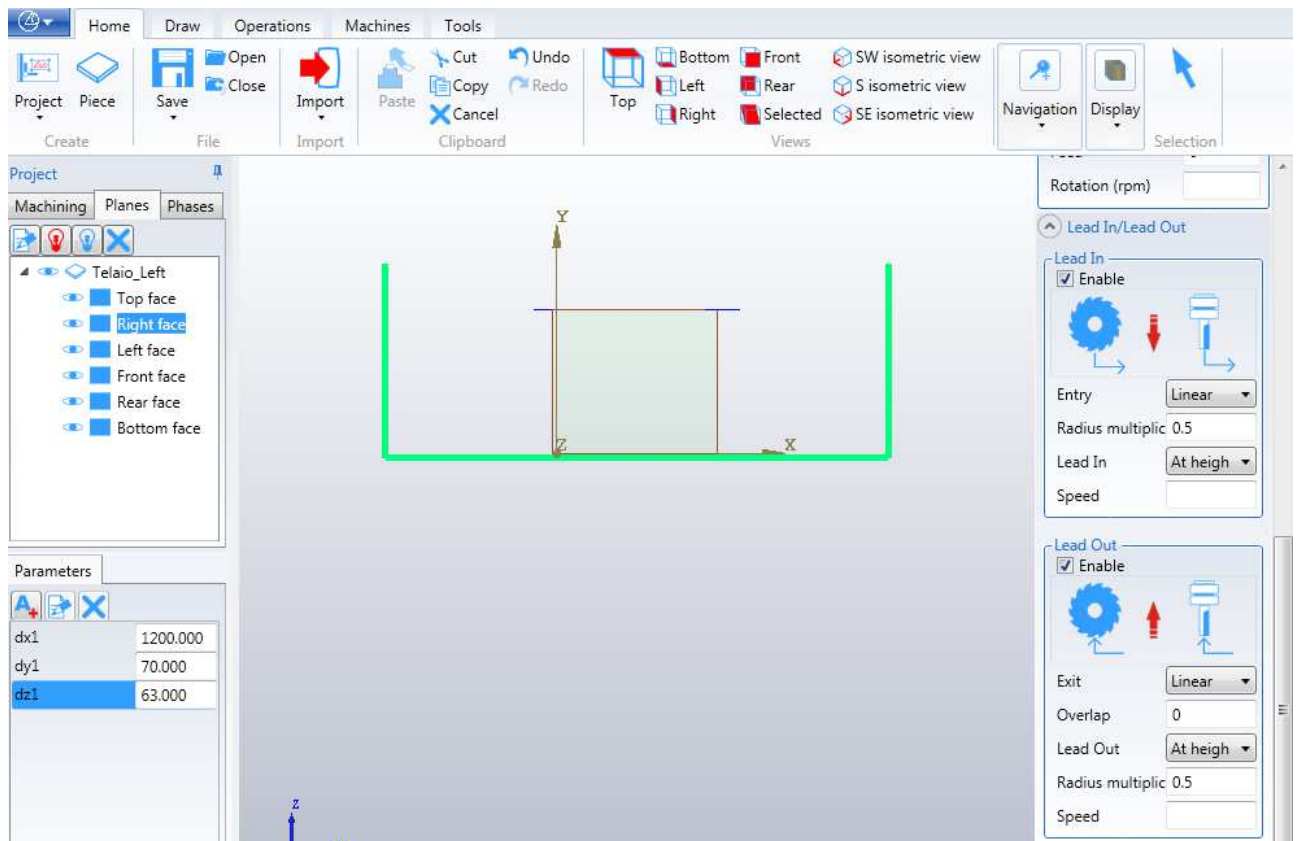
```

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 3);
SetDustpanOffset(10, 1, false);
CreatePolyline("Frame External Routing_32",1200,70);
AddSegmentToPolyline(0.70);
SetRetractStrategy(false, true, 3, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing_33",65,"Frame External Routing",
TypeOfProcess.GeneralRouting ,"E200","3",2,-1,-1,6,0);

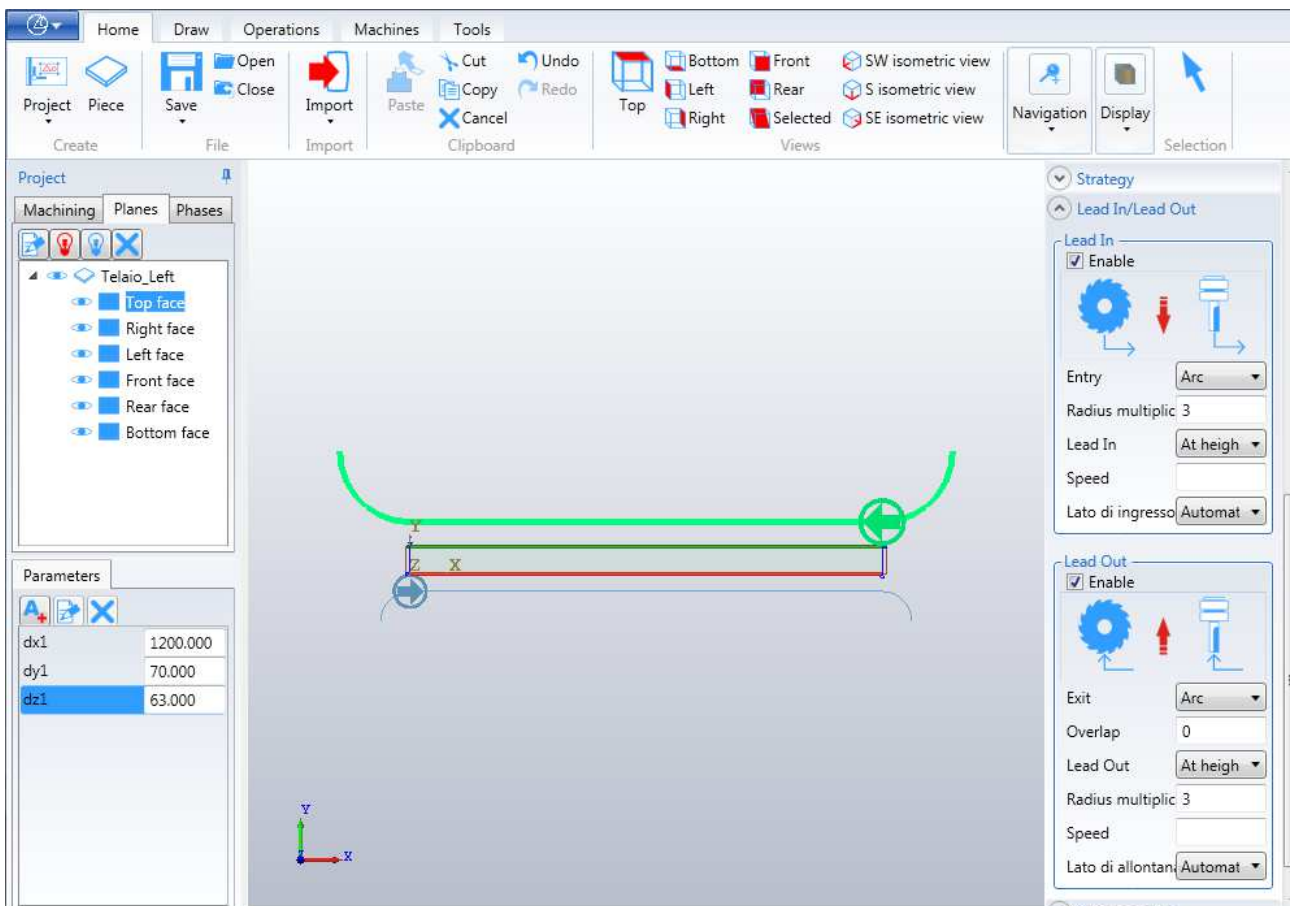
```

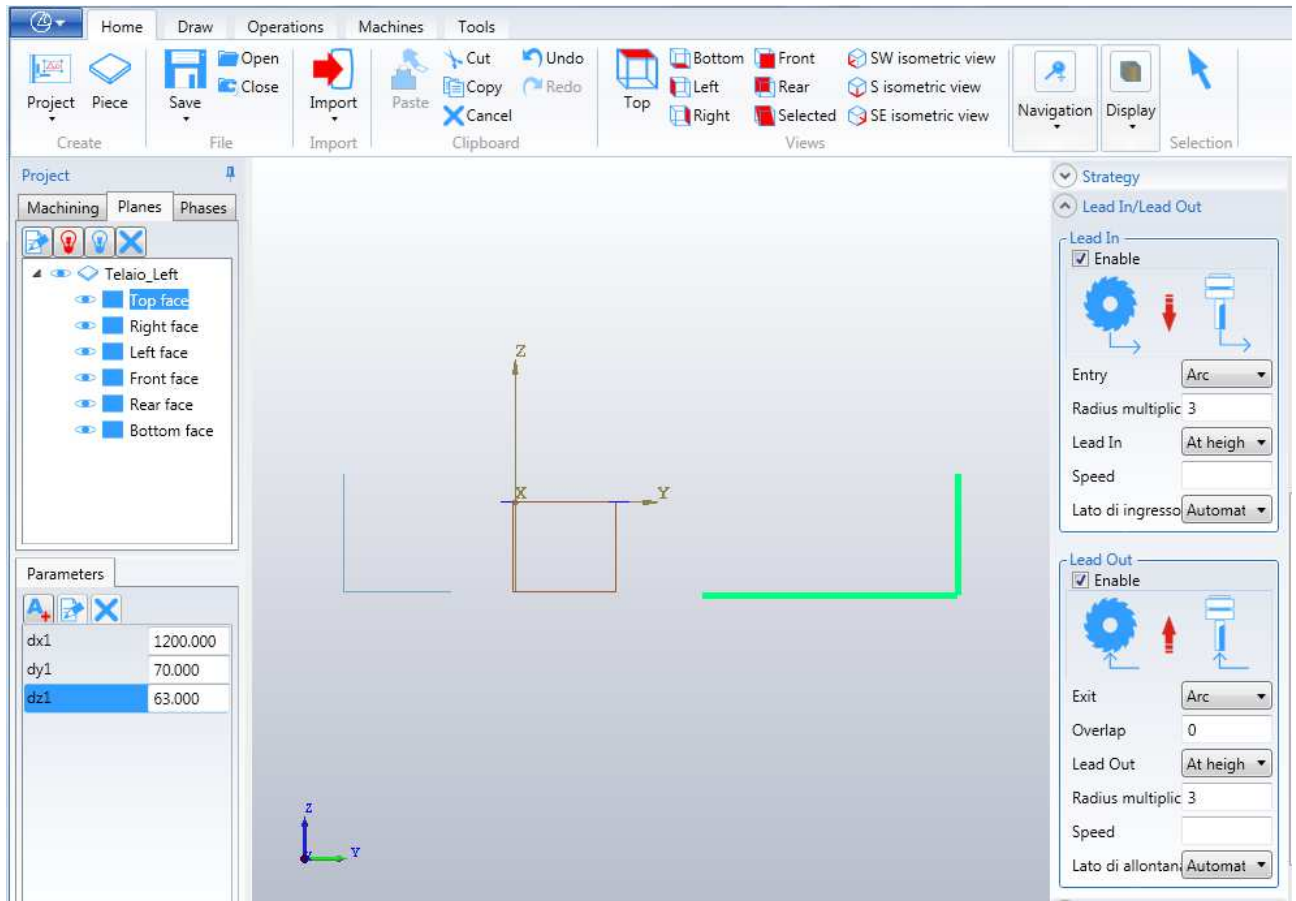
See below for the CAM views for the workpiece approach and retraction modes. In particular, for the slot creation the images show the top and right sides of just one operation. The view of the second operation is similar to the first. For the routing operation the views show both operations to highlight how the tool radius affects the calculation of the approach/retraction distance to the workpiece, with the same radius multiplier.





See below for the routing operation views:





3.5.11.7 Machining inversion

The direction of geometry can be inverted, so that it starts from the end point and ends at the start point. The following instructions are available in MSL:

- **SetMachiningDirection**: used to maintain or invert the direction of a geometry.

Clearly this option can be used for all machining operations that have a segment or a broken line as geometric primitive, but not a point like drilling.

The function is not a mode and will therefore only affect the subsequent machining operation.

See below for a description of the only parameter necessary for the function.

Name	Type	Description
sameDirection	bool	Type of direction: true = Machining with the geometry; false = machining in the opposite direction of the geometry;

Table 47: SetMachiningDirection Parameters

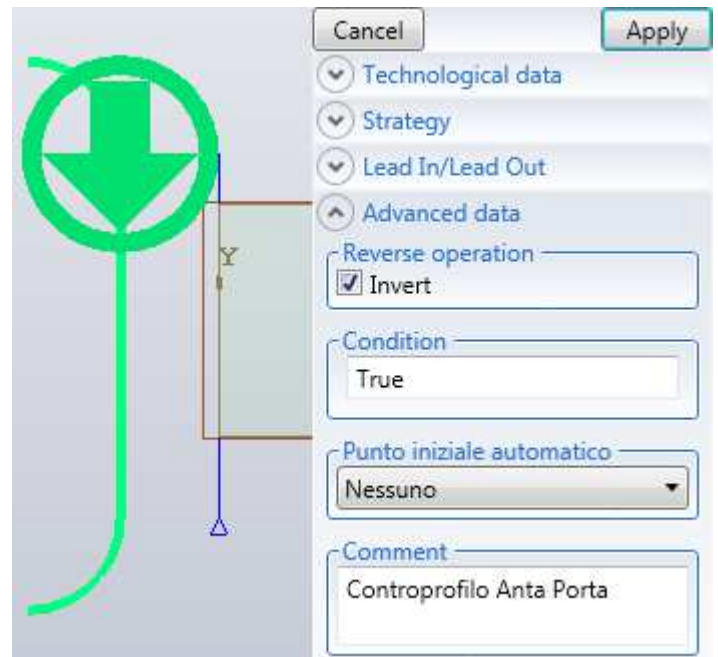
See below for an example in which the SetMachiningDirection instruction has been inserted.

The instruction is highlighted in bold:

```
SetMachiningParameters("AB", 1, 11, 1114112, false);
CreateFinishedWorkpieceBox("Anta_Up",1238,141,44);
CreateRawWorkpiece("Anta_Up",10,10,2,1,0,0);
SetWorkpieceSetupPosition(10,659,0, 0);

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetMachiningDirection(false);
ResetDustpan();
CreatePolyline("Controprofilo Anta Porta_14",0,-50);
AddSegmentToPolyline(0,-29);
SetAttribute("FEED", 1);
AddSegmentToPolyline(0,171);
SetAttribute("FEED", 3);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateRoughFinish("Controprofilo Anta Porta_15",44,"Controprofilo Anta Porta",
TypeOfProcess.GeneralRouting ,"E110","3",1,-1,-1,3,0);
```

The effect of the instruction is shown in the image at the side, where the geometry (Blue) rises, whilst the machining (Green) drops.



3.5.12 Machine functions

To program the machine and perform specific procedures during the operations the user can use certain instructions called **machine functions**.

This category includes, for example, functions used to enable, disable and configure the positioning of pneumatic hoods (PneumaticHood), the dustpan (Dustpan), etc....

Please note that this type of instructions are usually treated as “modal” functions inside the scripting. This means that the configurations set with these functions remain active for the subsequent operations, unless the settings are changed (reset or new setting). The operations that use these configurations do not “consume” the setting: the user must remember to reset or change the configuration set if he no longer wants to use it in the subsequent operations, otherwise it remains active.

There are also functions called “non-modal”. These functions set configurations whose validity is linked only to the first use: the first operation (after setting) uses and “consumes” the configuration. In this case, after the operation, the configuration is no longer available for the subsequent operations and it must be reset. By way of an example, many instructions defined “operation strategies”, described later, are part of this category.

See below for the main machine functions available to the user and the main input parameters that they accept.

3.5.12.1 Jerk control functions

To set the machine Jerk function, MSL provides two instructions:

- SetJerk: sets the standard Jerk function;
- SetJerk3D: sets the Jerk function in 3D mode.

To disable the Jerk settings, use the following instructions:

- ResetJerk: cancels the standard Jerk function;
- ResetJerk3D: cancels the Jerk function in 3D mode.

The input parameters of the first two functions are described below. The last two do not accept parameters.

Name	Type	Description
gain	double	Jerk value (0 : reset default values; >= 60 : recommended drilling value; >= 90 : recommended roughing value; >=160 : recommended finish value;

Table 48: SetJerk parameters

Name	Type	Description
gain	double	Jerk3D value: 0 = disabled; 1 = roughing; 2 = intermediate; 3 = finish.

Table 49: SetJerk3D parameters

The example below shows an application of the Jerk instructions for a routing operation:

```
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 40);
```

```
//setJerk
SetJerk(160);
```

```
CreatePolyline("Contour Polyline", 250, 100);
AddArc2PointRadiusToPolyline( 200, 150, 50, true, false);
AddSegmentToPolyline( 200, 650);
AddArc2PointRadiusToPolyline( 250, 700, 50, true, false);
AddSegmentToPolyline( 1550, 700);
AddArc2PointRadiusToPolyline( 1600, 650, 50, true, false);
AddSegmentToPolyline( 1600, 150);
AddArc2PointRadiusToPolyline( 1550, 100, 50, true, false);
ClosePolyline("Pocket Polyline");
```

```
CreateContour("Contour", 50, 1, 1, "Geometry Contour", TypeOfProcess.GeneralRouting, "E032",
"-1", 0, -1, -1, -1, 0);
ResetJerk();
```

The Jerk configured for the operations can be seen in Maestro in the *Machine data* menu associated to the operation.



Please note that the setting used remains active for the subsequent operations, as this is a “modal” type of instruction. It is therefore good practice to use the ResetJerk instruction after the operation, if you don't want the instruction to affect subsequent operations. For the 3D routing operations use the SetJerk3D and ResetJerk3D instructions similarly to the example in 2.5D.

3.5.12.2 Hood control functions

For the hood control, MSL provides the following instructions:

- **SetPneumaticHoodPosition**: enables the pneumatic hood and sets its position;
- **ResetPneumaticHood**: resets the last pneumatic hood position and enables the automatic positioning;
- **SetAuxiliaryHoodPosition**: enables the auxiliary hood and sets its position
- **ResetAuxiliaryHood**: resets the auxiliary hood position.

See below for the description of the input parameters for the functions that allow them. The reset functions do not allow the input parameters.

Name	Type	Description
position	int	Pneumatic hood position: 0 = high hood 1 = low hood in position 1 2 = low hood in position 2 3 = low hood in position 3 4 = low hood in position 4 5 = low hood in position 5 6 = low hood in position 6 7 = low hood in position 7 8 = low hood in position 8

Table 50: SetPneumaticHoodPosition parameters

Name	Type	Description
position	int	Auxiliary hood position: 0 = high hood 1 = low hood in position 1 2 = low hood in position 2 3 = low hood in position 3 4 = low hood in position 4 5 = low hood in position 5 6 = low hood in position 6 7 = low hood in position 7 8 = low hood in position 8

Table 51: SetAuxiliaryHoodPosition parameters

The example below shows the application of machine functions for the pneumatic and auxiliary hoods control in the operation of a frame:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0, 0);

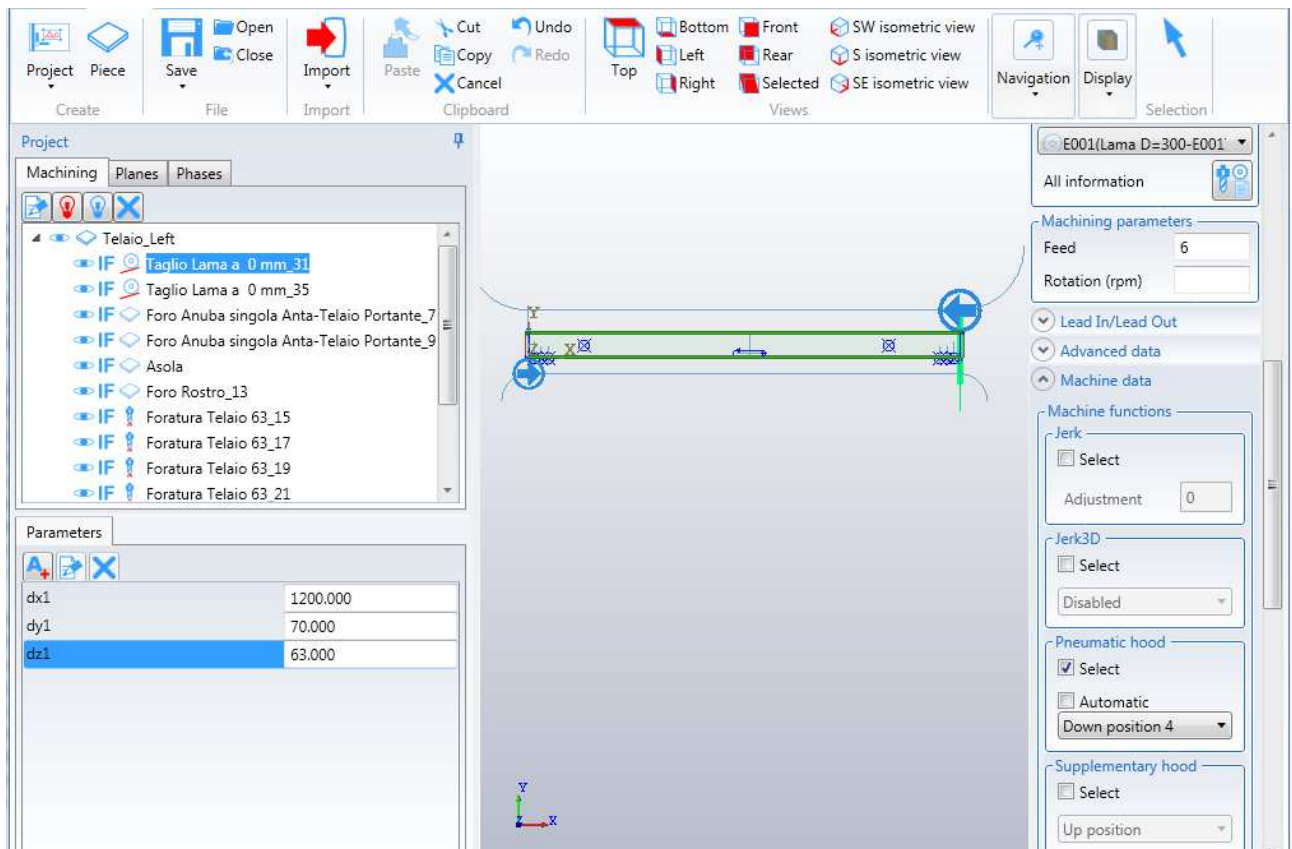
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, .5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_30", 1200, 0, 1200, 70);
SetRetractStrategy(true, true, .5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
SetAuxiliaryHood(2);
CreateSlot("Blade Cut at 0 mm_31",65,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting
,"E001", "3",2,-1,-1.6,0);

    ... (continue)

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 3);
SetDustpanOffset(10, 1, false);
CreatePolyline("Frame External Routing_32",1200,70);
AddSegmentToPolyline(0.70);
SetRetractStrategy(false, true, 3, 0);
ResetPneumaticHoodPosition();
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing_33",65,"Frame External Routing",
TypeOfProcess.GeneralRouting ,"E200", "3",2,-1,-1.6,0);
```

As can be seen, the script contains various operations; for example there is a slot operation “Blade Cut at 0 mm_31” with previous activation of the pneumatic hood at position 4 and the auxiliary hood in position 2. Then a routing is performed, “Frame External Routing_33”, that requires a type 1 automatic pneumatic hood and disabling the auxiliary hood. As can be seen in the example, it is good practice enable or disable the settings that are not required for the current operation. In fact they may have been activated in previous operations and as they are modal, they would still be active. Similarly the settings required must be activated and changed before the operation.

The settings configured with the hood control functions can be seen in Maestro, by displaying the operation properties, in the “Machine Functions” category: “Pneumatic hood”, “Auxiliary hood”.



3.5.12.3 Dustpan control functions

For the dustpan control, MSL provides the following instructions:

- **SetDustpanOffset**: enables the NC functions for the dustpan control (based on the input parameters, two different implementations can be selected);
- **SetDustpanPosition**: sets the dustpan position;
- **ResetDustpan()**: disables the NC function to control the dustpan .

See below for the description of the input parameters for the functions that allow them.

Name	Type	Description
offset	double	Dustpan distance (between dustpan and measured net radius/net radius measured at right angles to the tool path)

Table 52: SetDustpanOffset parameters (signature1)

Name	Type	Description
offset	double	Dustpan distance (between dustpan and measured net radius/net radius measured at right angles to the tool path)
position	int	Dustpan position: 0 = off; 1 = on in position 1; 2 = on in position 2;
isInOutEnable	bool	Enables approach and retraction

Table 53: SetDustpanOffset parameters (signature2)

Name	Type	Description
position	int	Dustpan position: 0 = off; 1 = on in position 1; 2 = on in position 2;
isInOutEnable	bool	Enables approach and retraction

Table 54: SetDustpanPosition parameters

The example below shows the application of the machine functions to control the dustpan. As always the example uses the operation of a frame:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0, 0);

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, .5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_30", 1200, 0, 1200, 70);
```

```

SetRetractStrategy(true, true, .5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_31",65,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting
,"E001","3",2,-1,-1.6,0);

```

... (continue)

```

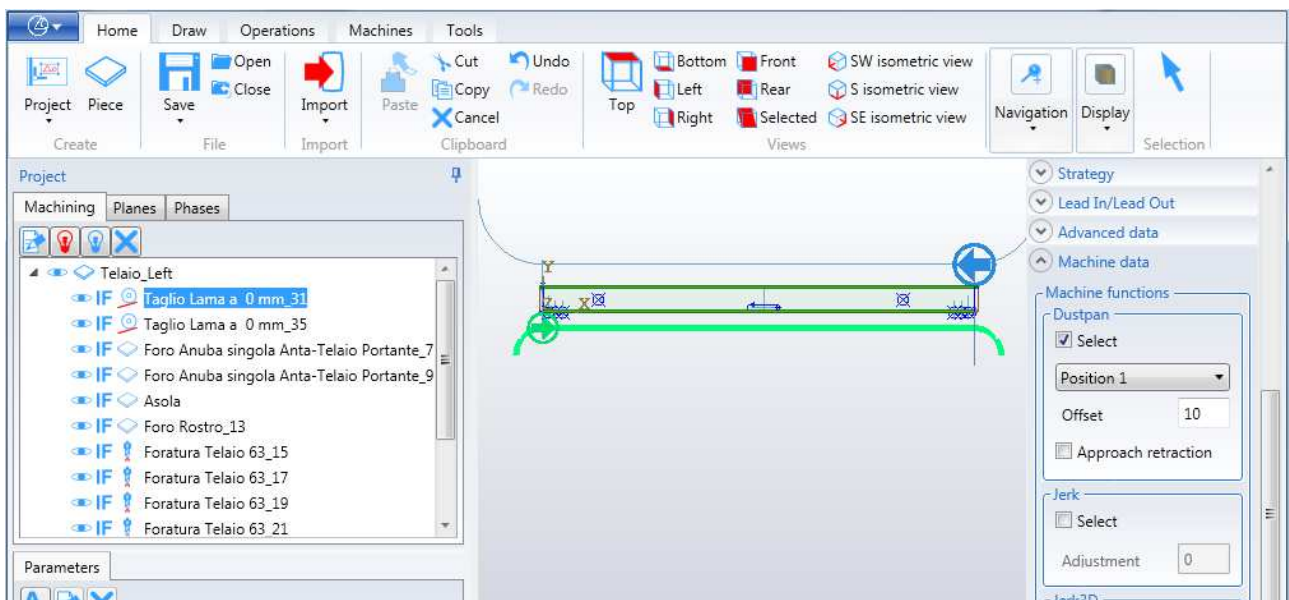
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetDustpanOffset(10, 1, false);
CreatePolyline("Door Housing Routing_4",0,0);
AddSegmentToPolyline(1200.0);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateRoughFinish("Door Housing Routing_5",63,"Door Housing Routing",
TypeOfProcess.GeneralRouting ,"E152","3",2,-1,-1,6,0);

```

... (continue)

As can be seen the script contains various operations. Before the slot creation “Blade Cut at 0 mm_31” operation the dustpan is disabled: this operation is performed without dustpan. In the second operation, the routing “Door Housing Routing_5”, the dustpan is enabled and applied to the operation with the **SetDustpanOffset** instruction. The operation activates the dustpan and configures it at a distance of 10, position 1, without the workpiece approach and retraction function.

The configured settings can be seen in Maestro, by selecting the operation properties, in the “Machine functions”:” Dustpan” category.



3.5.12.4 Rotating axes control functions

The following instructions are available for the control of the machine rotating axes:

- **SetUnrollHeadMode**: configures the 5-axes “unroll head” mode;
- **SetUnrollHeadRadiusMultiplier**: configures the 5-axes “unroll head” mode radius multiplier;
- **SetBrakes**: activates the rotating axes brakes (only for the subsequent operation).

See below for the descriptions of the input parameters for the instructions.

Name	Type	Description
mode	int (opt def 0)	Mode. 0 = disabled; 1 = perpendicular input and output; 2 = tangential input and perpendicular output; 3 = perpendicular input and tangential output; 4 = tangential input and output;
radiusMultiplier	double (opt def 1.0)	Radius multiplier

Table 55: SetUnrollHeadMode parameters

Name	Type	Description
radiusMultiplier	double (opt def 1.0)	Radius multiplier

Table 56: SetUnrollHeadRadiusMultiplier parameters

Name	Type	Description
activeBrakes	string	Active brakes (e.g. “B”, “B,C”).

Table 57: SetBrakes parameters

The application example shows a 3D routing operation with the configuration of the axes “unrolling” and the control of the axis brakes:

```
CreateFinishedWorkpieceBox("Example workpiece", 1800, 800, 40);

// Axes control
SetUnrollHeadMode(1,1.0);
SetUnrollHeadRadiusMultiplier(1.0);
SetBrakes("B,C");

// Operation: 3D routing
Create3DRoughFinish("3D Routing", "3D Routing", TypeOfProcess.GeneralRouting, "E031", "-1",
-1, -1, -1, 45, 45);
// 3D Tool path
CreateToolpath3D("3DToolPath", 0, 0, 20);
AddSegmentToToolpath(1800, 0, 20, 45, 45);
AddSegmentToToolpath(1800, 800, 20, 45, 45);
AddSegmentToToolpath(0, 800, 20, 45, 45);
AddSegmentToToolpath(0, 0, 20, 45, 45);
```

The parameters set with the rotating axes control instructions can be seen in Maestro in the operation properties, in the “Machine functions”.:” Rotating axes brakes”, “5-axes head unroll” category.

The image shows a screenshot of the Maestro software interface, specifically the 'Machine functions' section. It contains two expandable panels. The first panel, titled 'Rotation axes brakes', has a 'Select' checkbox which is checked, and a dropdown menu currently showing 'Axis BC'. The second panel, titled '5 axes head unroll', also has a checked 'Select' checkbox, a dropdown menu showing 'Perpendicular infeed', and a 'Radius multiplier' input field with the value '1'.

Category	Option	Value
Rotation axes brakes	Select	☒
	Axis	BC
5 axes head unroll	Select	☒
	Perpendicular infeed	
	Radius multiplier	1

3.5.13 Tool path

To set a tool path use the `CreateToolpath` instruction. The start point is used as the start point for the subsequent arc and segment geometries. The input parameters for the instruction are described in the table.

Name	Type	Description
name	string	Tool path name
startX	double	Coordinate X of the first tool path point.
startY	double	Coordinate Y of the first tool path point.
startZ	double	Coordinate Z of the first tool path point.

Table 58: CreateToolpath parameters

To set a tool path in 3D use the `CreateToolpath3D` instruction. The start point is used as the start point for the subsequent arc and segment geometries. The rotation angles `ZRotation` and `XRotation`, that define the rotation of the tool axes at the start point are specified in the operation creation instruction `Create3DRoughFinish`. The input parameters for the instruction are described in the table.

Name	Type	Description
name	String	Tool path name
startX	Double	Coordinate X of the first tool path point.
startY	Double	Coordinate Y of the first tool path point.
startZ	Double	Coordinate Z of the first tool path point.

Table 59: CreateToolpath3D parameters

To add a segment to the tool path use the `AddSegmentToToolpath` instruction. The start point of the segment is the point specified in the previous instruction, similarly to the definition of the polyline geometry. MSL defines two different implementation for the function, based on the type of parameters, either 2D or 3D. The input parameters for the two implementations are described in the table.

Name	Type	Description
endX	Double	Coordinate X of the last tool path point.
endY	Double	Coordinate Y of the last tool path point.
endZ	Double	Coordinate Z of the last tool path point.

Table 60: AddSegmentToToolpath parameters (2D version)

Name	Type	Description
endX	Double	Coordinate X of the last tool path point.
endY	Double	Coordinate Y of the last tool path point.
endZ	Double	Coordinate Z of the last tool path point.
ZRotation	Double	Rotation around axis Z applied to the current segment.
XRotation	Double	Rotation around axis X applied to the current segment.

Table 61: AddSegmentToToolpath parameters (3D version)

The following instructions are available to add an arc to the tool path:

- **AddArc3PointsToToolpath:** adds an arc with three points of the arc and the centre point
- **AddArc2PointCenterToToolpath:** adds an arc with two points of the arc and the centre point
- **AddArc2PointRadiusToToolpath:** adds an arc with two points of the arc and the radius
- **AddArcCenterAngleToToolpath:** adds an arc with the arc centre point and the arc angle

The arc start point is the final point of the tool path (point specified in the previous instruction).

See below for the descriptions of the input parameters for the four functions.

Name	Type	Description
p2X	Double	Coordinate X of the second point.
p2Y	Double	Coordinate Y of the second point.
p3X	Double	Coordinate X of the third point.
p3Y	Double	Coordinate Y of the third point.
p3Z	Double	Coordinate Z of the third point.

Table 62: AddArc3PointsToToolpath parameters

Name	Type	Description
endX	Double	Coordinate X of the last tool path point.
endY	Double	Coordinate Y of the last tool path point.
endZ	Double	Coordinate Z of the last tool path point.
centerX	Double	Coordinate X of the arc centre point
centerY	double	Coordinate Y of the arc centre point
isClockwise	bool	Arc constructed clockwise

Table 63: AddArc2PointCenterToToolpath parameters

Name	Type	Description
endX	double	Coordinate X of the last tool path point.
endY	double	Coordinate Y of the last tool path point.
endZ	double	Coordinate Z of the last tool path point.
radius	double	Arc radius
isClockwise	bool	Arc constructed clockwise
isOver180	bool	Arc angle greater than 180°

Table 64: AddArc2PointRadiusToToolpath parameters

Name	Type	Description
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
endZ	double	Coordinate Z of the last tool path point.
angle	double	Arc angle

Table 65: AddArcCenterAngleToToolpath parameters

Note that the `CreateToolpath`, `AddSegmentToToolpath`, etc... instructions used to specify the tool path can be used with all the routing operations 2.5D (`CreateRoughFinish`, `CreateContourPocket`, etc...). If the tool path is specified, the machining will be performed solely as specified by the user and no strategies will be considered by the system.

For an example of the application of the instructions described above see chapter **3.5.6**, where a tool path was created for the 3D routing.

Note that the instructions that describe the tool path must always be positioned in the script after the instruction that indicates the type of machining.

Another example is provided with an extract of code used for the “Clamex” macro. We can see how the instructions for the tool path are defined after the definition of the machining geometry and the actual machining.

```

... (continue)

Tool tool = GetTool(Tool); //Load tool to read the radius information
double Rag = 50.2;

CreateWorkplane("", -(Prof-Spes/2));

if ((Face == 2 & Side == 2) // Front Right
|| (Face == 2 & Side == 4) // Front Left
|| (Face == 3 & Side == 2) // Rear Left
|| (Face == 3 & Side == 4) // Front Left
|| (Face == 4 & Side == 2) // Rear Left
|| (Face == 4 & Side == 4) // Front Left
|| (Face == 5 & Side == 2) // Rear Left
|| (Face == 5 & Side == 4) // Rear Right
{
    //Clamex geometry creation
    CreatePolyline("SlotGeometry", PosY, (PosX+35.8));
    AddArc2PointRadiusToPolyline( PosY, (PosX-35.8) , Rag, true, false);
    ClosePolyline();
} else {
    //Clamex geometry creation
    CreatePolyline("SlotGeometry", PosX-35.8, PosY);
    AddArc2PointRadiusToPolyline( PosX+35.8, PosY , Rag, true, false);
    ClosePolyline();
}

//Clamex mark creation
CreateContourPocket("Clamex", Spes, "slot creation operation",
TypeOfProcess.ConcentricalPocket, Tool, Testa);

double x1,y1,z1,y2,z3,z4;

x1 = PosX;
y1 = -F-tool.Radius;
z1 = -Spes/2;
y2 = PosY+15-tool.Radius;
z3 = z1 + (Spes-SpFr)/2;
z4 = z1 - (Spes-SpFr)/2;

if ((Face == 2 & Side == 2) // Front Right
|| (Face == 2 & Side == 4) // Front Left
|| (Face == 3 & Side == 2) // Rear Left
|| (Face == 3 & Side == 4) // Front Left

```

```

|| (Face == 4 & Side == 2) // Rear Left
|| (Face == 4 & Side == 4) // Front Left
|| (Face == 5 & Side == 2) // Rear Left
|| (Face == 5 & Side == 4) // Rear Right
{
    CreateToolpath("Clamex path", y1, x1, z1);
    AddSegmentToToolpath(y2, x1, z1);
    AddSegmentToToolpath(y2, x1, z3);
    AddSegmentToToolpath(y2, x1, z4);
    AddSegmentToToolpath(y2, x1, z1);
    AddSegmentToToolpath(y1, x1, z1);
} else {
    CreateToolpath("Clamex Path", x1, y1, z1);
    AddSegmentToToolpath(x1, y2, z1);
    AddSegmentToToolpath(x1, y2, z3);
    AddSegmentToToolpath(x1, y2, z4);
    AddSegmentToToolpath(x1, y2, z1);
    AddSegmentToToolpath(x1, y1, z1);
}

```

3.5.14 Machining technologies

For the specific documentation of the specifications and the creation method of the machining technologies refer to the Xilog-Maestro manual.

To apply a technology to an active geometry use the ApplyTechnology instruction. The geometry must be compatible with the technology.

The input parameters for the instruction are described in the table.

Name	Type	Description
name	string	Technology identification name

Table 66: ApplyTechnology parameters

All the technologies available to the user are in Maestro in the *Technologies* folder, in the folder that Maestro is installed in (e.g.: *C:\Program Files\Scm Group\Maestro\Technologies*).

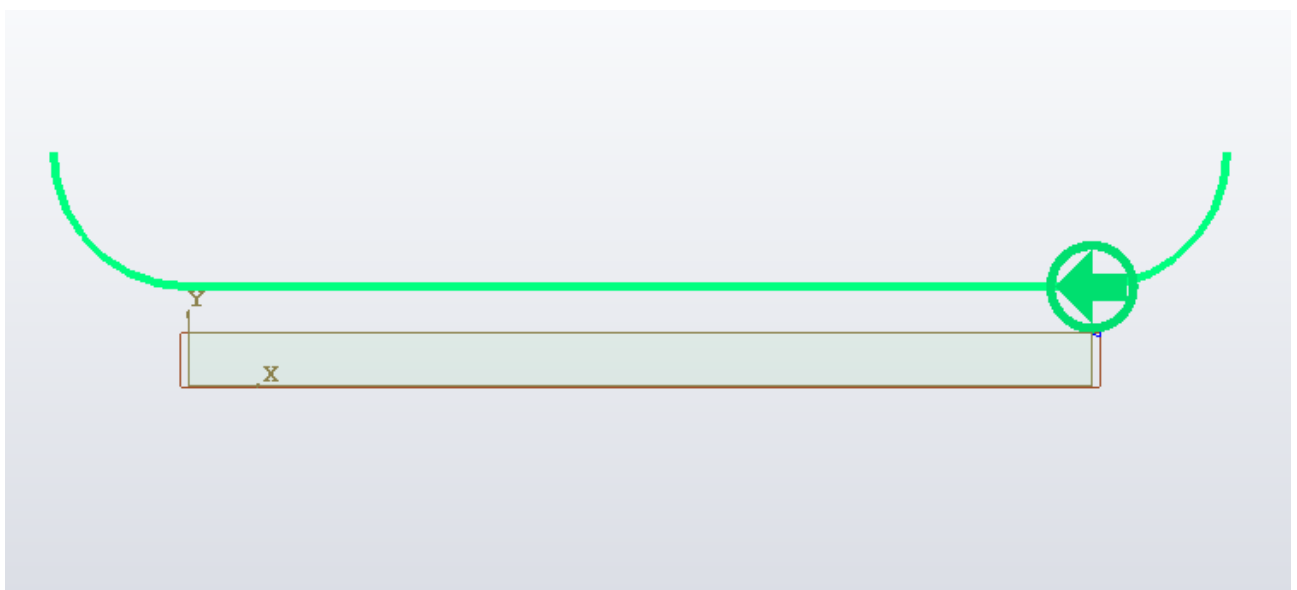
The user can create and save new technologies starting from single machining, with the procedure described in the Maestro manual.

See below for the application of the “Profiling” technology to the segment geometry, along the edge of the workpiece .

```
SetMachiningParameters("AB", 1, 0, 1114112, false);  
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);  
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
```

```
CreatePolyline("External Profiling",1200,70);  
AddSegmentToPolyline(0.70);  
ApplyTechnology("Profiling");
```

The image below shows the application of the technology with the CAM view of the workpiece machining.



3.5.14.1 EdgeBanding work technology

The edge banding process can be done by Scripting from a technology and the geometry defining the profile to edge.

3.5.14.1.1 Technology

It is necessary to define an edge banding technology where a color defines a machining, specially same color to machining having same meaning like EdgeBanding and EndTrimming that are depending on edge.

3.5.14.1.2 Geometry Definition

Upon technology definition on Xcs file the first step is the geometry definition that identifies the profile to edge.

Two cases are possible :

1) Closed Geometry where the same Edge is applied on each geometry elements: it is necessary to create a closed geometry, es polyline or circle, and the whole process will be applied over it with 1 Edge following standard edge banding process rules.

Example :

```
CreatePolyline("Geometria", 0, 800);  
AddSegmentToPolyline(800, 1200, "");  
AddSegmentToPolyline(800, 0, "");  
AddSegmentToPolyline(0, 0, "");  
AddSegmentToPolyline(0, 800, "");
```

2) Closed Geometry where a different Edge is applied on each geometry elements : it is necessary to create a list of open geometry (ex segment) that must be define a closed geometry.

Example : the geometry to edge is a rectangle, it is necessary to define 4 segments that must compose the rectangle.

```
CreateSegment("Sopra", 0, 800, 620, 800);  
CreateSegment("Destro", 620, 800, 620, 0);  
CreateSegment("Sinistro", 0, 0, 0, 800);  
CreateSegment("Sotto", 620, 0, 0, 0);
```

3.5.14.1.3 Technology Selection

Upon geometry definition it is necessary to select the edgebanding technology to use and is necessary to indicate the list of all geometry that define the profile to edge (case 1 and case 2)

Example :

```
SelectTechnology("Bordatura", "Sopra", "Sotto", "Destro", "Sinistro");
```

In this case "Bordatura.tchx" will be selected and are specified the segments that describe the profile to edge.

3.5.14.1.4 Geometry Technology Selection

Upon initialization of Technology and Profile is necessary to define how the whole geometry will be machined.

Command :

```
SelectGeometryTechnology("Nome", R, G, B);
```

needs geometry Name and Color Code (RGB) to use.

The command call sequence defines the edge application sequence on profile.

In case 1 of chapter 3.5.14.1.2 we have, by example

```
SelectGeometryTechnology("Geometria", 255, 0, 0);//Rosso
```

in this way to the polyline previously created will be applied the edgebanding process relative to Bordatura and depending machinings with Red color on technology. In this case one edge will be used and applied to all geometry elements respecting edgebanding process rules.

All other machinings with same color will be created, excepted for Milling and EndTrimming that will be automatic performed.

In case 2 of chapter 3.5.14.1.2, we have more possibilities, by example

```
SelectGeometryTechnology("Sopra", 255, 0, 0);//Red  
SelectGeometryTechnology("Destro", 0, 255, 0);//Green  
SelectGeometryTechnology("Sinistro", 255, 255, 0);//Yellow  
SelectGeometryTechnology("Sotto", 0, 255, 255);//Azure
```

where segments correspond to

"Sopra" -> upper segment of rectangle

"Destro" -> right segment of rectangle

"Sinistro" -> left segment of rectangle

"Sotto" -> lower segment of rectangle

in this way the calls sequence defines the order of edge application:

- Edge on segment "Sopra" and relative machinings like EndTrimming if existing on technology.
- Edge on segment "Destro" and relative machinings like EndTrimming if existing on technology.
- Edge on segment "Sinistro" and relative machinings like EndTrimming if existing on technology.
- Edge on segment "Sotto" and relative machinings like EndTrimming if existing on technology.

3.5.14.1.5 Technology Application

Upon completed the process definition, with the command

```
ApplyTechnology();
```

the process will be created.

The command analyzes defined geometries, colors and machinings, and with this informations optimize some machinings like milling, trimming, sraping.

complete example:

```
SetMachiningParameters("EH", 1, 21, 1114112, false);
CreateFinishedWorkpieceBox("Pezzo",620,800,18);
SetWorkpieceSetupPosition(0,0,0, 0);

CreateSegment("Sopra", 0, 800, 620, 800);
CreateSegment("Destro", 620, 800, 620, 0);
CreateSegment("Sinistro", 0, 0, 0, 800);
CreateSegment("Sotto", 620, 0,0, 0);

// Initialization of technology and relevant geometries
SelectTechnology("Bordatura", "Sopra", "Sotto", "Destro" , "Sinistro");

// Selection of the technology to be applied to the single geometry
SelectGeometryTechnology("Sopra", 255, 0, 0);//Red
SelectGeometryTechnology("Destro", 0, 255, 0);//Green
SelectGeometryTechnology("Sinistro", 255, 255, 0);//Yellow
SelectGeometryTechnology("Sotto", 0, 255, 255);//Azure

// Application of Technology
ApplyTechnology();
```

3.5.15 Machining attributes

The attributes are properties that can be associated to sections of geometry or tool path. MSL defines three types of attributes: "FEED", "ROT", "DEPTH". The first attribute ("FEED") is used to specify the tool feed speed, the second ("ROT") the rotation of the edge bander head and the third ("DEPTH") the application depth of machining. As the user can specify these attributes for each section of the tool path (or geometry) he can change the behaviour of the generic machining it is associated to, in specific section of the path, based on his requirements.

To apply the attributes to a geometry or a tool path, the following instructions are defined:

- **SetAttribute**: sets an attribute for the current active geometry (this instruction is usually used during the creation of the geometry, before an operation);
- **SetParametricAttribute**: identical to the **SetAttribute** with the possibility to determine its position in percentage
- **SetToolpathAttribute**: sets an attribute for the current tool path.
- **SetAttribute2**: inserts a bridge at the end of the segment
- **SetParametricAttribute2**: identical to the **SetAttribute2** with the possibility to determine its position in percentage.

3.5.15.1 Attributes FEED,ROT,DEPTH

Name	Type	Description
name	string	Attribute name Only the following names are allowed: "FEED", "ROT", "DEPTH".
value	double	Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE).

Table 67: SetAttribute parameters

Name	Type	Description
name	string	Name of the attribute. Only the following names are admitted: "FEED", "ROT", "DEPTH".
value	double	Attribute value (for the boolean use 0 for FALSE, different than zero for TRUE).
uPar	Double	Position attribute (allowable values 0.0-1.0)

Table 68: Parameters SetParametricAttribute

Name	Type	Description
name	string	Attribute name Only the following names are allowed: "FEED", "ROT", "DEPTH".
value	double	Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE).

Table 69: SetToolpathAttribute parameters

The example below shows the application of attributes for the geometries. Similar observations can be performed for the attributes for a tool path.

The first example shows the application of attributes to change the feed speed along a geometry. In this case the segment is divided in three parts and an attribute is associated to each one to slow down the feed speed in the initial and final section of the geometry.

```
double V1 = 0.5;
double V2 = 4;

CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 3);
SetDustpanOffset(10, 1, false);

CreatePolyline("Frame External Geometry",1200,70);
//AddSegmentToPolyline(0,70);

AddSegmentToPolyline(1190.70);
SetAttribute("FEED", V1);
AddSegmentToPolyline(10.70);
SetAttribute("FEED", V2);
AddSegmentToPolyline(0.70);
SetAttribute("FEED", V1);

SetRetractStrategy(false, true, 3, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();

CreateRoughFinish("Frame External Routing",65,"Frame External Routing",
TypeOfProcess.GeneralRouting ,"E200","3",2,-1,-1,V2,0);
```

In the post-process file, containing the ISO commands, we can see that the first and last section have a feed speed of $1000 \times 0.5 = 500$ mm/min, whilst the central section has a speed of $1000 \times 4 = 4000$ mm/min.

```
H DX=1200.000 DY=70.000 DZ=63.000 -AB C=0 T=1114112 R=1 *MM /"SerrSCM" BX=0.000 BY=0.000
BZ=0.000 V=0
;*****
; >> Post P. Release...
;Version : 1.00.006.1006;
; >> Program description...
; Creation date: 4/18/2013 1:44:01 PM
;
; >> Tools used...
;
; >> Program Start...
;*****
;Workpiece setup Frame_Left on Setup phase
REF DX=1200.000 DY=70.000 DZ=63.000 FLD=AB BX=0.000 BY=0.000 BZ=0.000
;*****
;*****
.OP1 ;Frame External Routing
;*****
SET DUSTPAN=1
```

```

SET DUSTPANINOUTOFF=1
SET DUSTPANOFFSET=10
F=1
O X=0.000 Y=0.000 Z=0.000 F=1 ;ChangePlane
SET ZFAST=20.000
XGIN G=2 R=3 Q=0
XG0 X=1200.000 Y=70.000 Z=-65.000 T=300 P=0 D=20 C=1 E=1 s=0 ;Frame External Routing
XL2P X=1190.000 Y=70.000 Z=-65.000 V=500
XL2P X=10.000 Y=70.000 Z=-65.000 V=4000
XL2P X=0.000 Y=70.000 Z=-65.000 V=500
XGOUT G=2 R=3 Q=0 L=0
PARK S=0
;*****
.END
;*****
;FINEPROG

```

Similarly the depth applied to the geometry can be changed with the SetAttribute instruction. For example by changing the machining depth of the previous example the relevant script section becomes:

```

double V1 = 0.5;
double V2 = 4;
double deep1 = 65;
double deep2 = 30;

...(continue)...

CreatePolyline("Frame External Geometry",1200,70);
AddSegmentToPolyline(1190.70);
SetAttribute("FEED", V1);
SetAttribute("DEPTH", deep1);
AddSegmentToPolyline(10.70);
SetAttribute("FEED", V2);
SetAttribute("DEPTH", deep2);
AddSegmentToPolyline(0.70);
SetAttribute("FEED", V1);
SetAttribute("DEPTH", deep1);

...(continue)...

CreateRoughFinish("Frame External Routing",deep1,"Frame External Routing",
TypeOfProcess.GeneralRouting ,"E200","3",2,-1,-1,V2,0);

```

With the subsequent changes to the ISO file:

```

XG0 X=1200.000 Y=70.000 Z=-65.000 T=300 P=0 D=20 C=1 E=1 s=0 ;Frame External Routing
XL2P X=1190.000 Y=70.000 Z=-65.000 V=500
XL2P X=10.000 Y=70.000 Z=-30.000 V=4000
XL2P X=0.000 Y=70.000 Z=-65.000 V=500

```

3.5.15.2 TAB Attributes (Bridges)

These attributes are used to keep the scrap in the panels, application used in nesting or in machines where the scrap is a problem (Cyflex)

Name	Type	Description
name	string	Name of the attribute. Only the name: "TAB" is permitted
Value1	double	Bridge thickness
Value2	double	Bridge length

Table 70: Parameters SetAttributes2

Name	Type	Description
name	string	Name of the attribute. Only the name: "TAB" is permitted
Value1	double	Bridge thickness
Value2	double	Bridge length
uPar	Double	Position attribute (allowable values 0.0-1.0)

Table 71: Parameters SetParametricAttribute2

Example 1 :

Creation of a bridge thick 3 and long 10

```
CreatePolyline("Polilinea",-10,556);
AddSegmentToPolyline(94,556);
```

```
SetAttribute2("TAB", 3, 10);
```

```
AddSegmentToPolyline(94.610);
```

```
CreateRoughFinish("Toekick",15,"Toekick", TypeOfProcess.GeneralRouting ,"Fresa _D12","",0,-1,-1,5,0); (*fresa = mill)
```

Iso Product

```
;*****
.OP1 ;Toekick
;*****
F=1
O X=0.000 Y=0.000 Z=0.000 F=1 ;ChangePlane
SET ZFAST=20.000
XG0 X=-10.000 Y=556.000 Z=20.000 T=101 P=0 D=20 C=0 V=5000 E=0 s=0 ;Toekick
XL2P X=-10.000 Y=556.000 Z=-15.000 V=5000
XL2P X=71.900 Y=556.000 Z=-15.000 V=5000
XL2P X=71.900 Y=556.000 Z=-9.000 V=5000
XL2P X=93.900 Y=556.000 Z=-9.000 V=5000
XL2P X=93.900 Y=556.000 Z=-15.000 V=5000
XL2P X=94.000 Y=556.000 Z=-15.000 V=5000
XL2P X=94.000 Y=610.000 Z=-15.000 V=5000
XL2P X=94.000 Y=610.000 Z=20.000 V=5000
```

Example 2:

Creation of three bridges thickness 3 and long 10 segments on 2 broken segments

```
CreatePolyline("Polilinea",-10.556);
AddSegmentToPolyline(94.556);

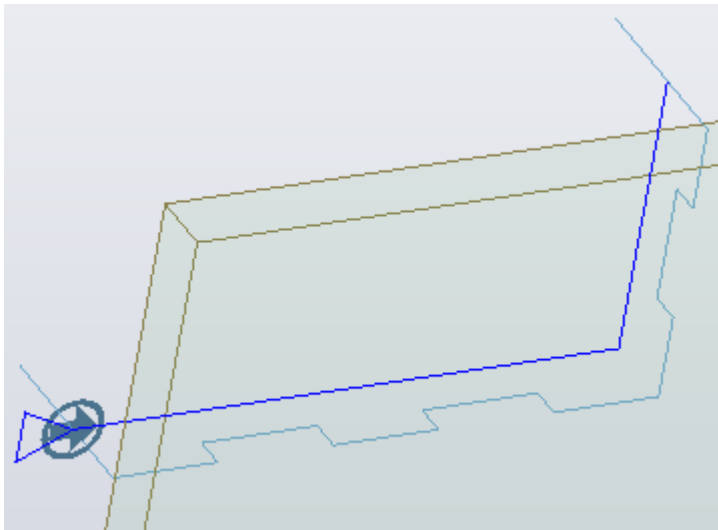
                //"TAB", SpesPont, LungPont, Percentuale); (*Bridge thickness, Bridge
Length. Percentage);
SetParametricAttribute2("TAB",          3,          10,          0.2981); // Bridge1
SetParametricAttribute2("TAB",          3,          10,          0.7019); // Bridge2

AddSegmentToPolyline(94.610);

                //"TAB", SpesPont, LungPont, Percentuale); (*Bridge thickness, Bridge
Length. Percentage);
SetParametricAttribute2("TAB",          3,          10,          0.5000); // Bridge 3

CreateRoughFinish("Toekick",15,"Toekick", TypeOfProcess.GeneralRouting ,"Fresa _D12","",0,-
1,-1,5,0); (*fresa = mill)
```

Machining obtained:



Iso Product

```
;*****
.OP1 ;Toekick
;*****
F=1
O X=0.000 Y=0.000 Z=0.000 F=1 ;ChangePlane
SET ZFAST=20.000
XG0 X=-10.000 Y=556.000 Z=20.000 T=101 P=0 D=20 C=0 V=5000 E=0 s=0 ;Toekick
XL2P X=-10.000 Y=556.000 Z=-15.000 V=5000
XL2P X=10.002 Y=556.000 Z=-15.000 V=5000
XL2P X=10.002 Y=556.000 Z=-9.000 V=5000
XL2P X=32.002 Y=556.000 Z=-9.000 V=5000
XL2P X=32.002 Y=556.000 Z=-15.000 V=5000
XL2P X=51.998 Y=556.000 Z=-15.000 V=5000
XL2P X=51.998 Y=556.000 Z=-9.000 V=5000
XL2P X=73.998 Y=556.000 Z=-9.000 V=5000
XL2P X=73.998 Y=556.000 Z=-15.000 V=5000
XL2P X=94.000 Y=556.000 Z=-15.000 V=5000
XL2P X=94.000 Y=572.000 Z=-15.000 V=5000
XL2P X=94.000 Y=572.000 Z=-9.000 V=5000
XL2P X=94.000 Y=594.000 Z=-9.000 V=5000
XL2P X=94.000 Y=594.000 Z=-15.000 V=5000
XL2P X=94.000 Y=610.000 Z=-15.000 V=5000
XL2P X=94.000 Y=610.000 Z=20.000 V=5000
```

3.5.16 Maintain Technological Sequence (SetPriority)

This method allows you to group project work maintaining their technological sequence being optimized.

The optimization is done in these cases:

1. When you intend to work 2 or more pieces on the floor
2. When operating a piece of machinery CYFLEX

When machining 2 or more workpieces on the plane by minimising the number of tool changes with the Maestro optimiser, the programmed technological sequence may be changed.

In fact in such cases the Maestro paths optimiser probably changes the technological sequence, as it can change the order of the machining operations carried out with the same tool.

To maintain the desired technological sequence even when using the optimiser, use the SetPriority instruction.

The SetPriority must be entered in the script before each machining operation that you want to maintain the technological sequence defined in the script.

A value that represents the priority is the only parameter entered in the SetPriority.

The priority values must be positive but they do not necessarily have to be progressive, the machining operations that can be done with the same tool will maintain the sequence of the priority from the lowest to the highest value, whilst the ones without the SetPriority can be changed by the optimiser.

Example:

```
SetPriority(1);
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.3);
CreatePolyline("Controprofilo Telaio 20_18",-20,-20);
AddSegmentToPolyline(-20,1);
SetAttribute("FEED", 1);
AddSegmentToPolyline(-20,70);
SetAttribute("FEED", 3);
SetRetractStrategy(false, true, 1.3, 0);
SetPneumaticHoodPosition(4);
CreateRoughFinish("Controprofilo Telaio 20_19",63,"Controprofilo Telaio 20",
TypeOfProcess.GeneralRouting ,"E140","3",1,-1,-1,3,0);
```

```
SetPriority(2);
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.3);
CreatePolyline("Controprofilo Telaio 10_20",-10,-20);
AddSegmentToPolyline(-10,1);
SetAttribute("FEED", 1);
AddSegmentToPolyline(-10,70);
```

```

SetAttribute("FEED", 3);
SetRetractStrategy(false, true, 1.3, 0);
SetPneumaticHoodPosition(4);
CreateRoughFinish("Controprofilo Telaio 10_21",63,"Controprofilo Telaio 10",
TypeOfProcess.GeneralRouting ,"E140","3",1,-1,-1,3,0);

```

By adding the SetPriority(1); and SetPriority(2); instructions, you are certain that the 1-2 sequence will be maintained with these two operations, even when the programs will be optimised.

Whilst any other operations that do not have the SetPriority can be moved by the optimiser.

When working on CYFLEX machine, the related processes with increasing values of SetPriority will be built at the same stage.

The necessity of this method is kind to precision problems in the CYFLEX machines where the workpiece is moved between one phase and another.

The machining tools consist of multiple paths, such as a counterbore hole made with two different tools if it is realized in two different phases might be inaccurate.

How you use the SetPriority plans to place a growing value in all the processes that must be performed at the same stage.

When the value is set to zero or to a value lower than the last value entered will stop the series of processes to be performed in the same phase.

Example pgmx

```

CreateDrill("Foratura"      , 20, 20, 13, 8, "Foratura", TypeOfProcess.Drilling);
CreateDrill("Foratura"aa"   , 20, 60, 13, 8, "Foratura", TypeOfProcess.Drilling);

CreateDrill("Foratura-1"    , 20, 250,13, 8, "Foratura", TypeOfProcess.Drilling);
CreateDrill("Foratura-1a"   , 20, 270,13, 8, "Foratura", TypeOfProcess.Drilling);

SetPriority(1);
CreateDrill("Foratura(1)"    ,880, 20, 13, 8, "Foratura", TypeOfProcess.Drilling);
SetPriority(2);
CreateDrill("Foratura(1a)"   ,880, 60, 13, 8, "Foratura", TypeOfProcess.Drilling);

SetPriority(3);
CreateDrill("Foratura(1-1)"  ,880, 250,13, 8, "Foratura", TypeOfProcess.Drilling);
SetPriority(4);
CreateDrill("Foratura(1-1a)" ,880, 290,13, 8, "Foratura", TypeOfProcess.Drilling);

SetPriority(5);
CreateDrill("Foratura(2-1)"  ,880, 30, 13, 5, "Foratura", TypeOfProcess.Drilling);
SetPriority(6);
CreateDrill("Foratura(2-1a)" ,740, 30, 13, 5, "Foratura", TypeOfProcess.Drilling);

CreateDrill("Foratura(2)"    , 20, 30, 13, 5, "Foratura", TypeOfProcess.Drilling);
CreateDrill("Foratura(2a)"   ,260, 30, 13, 5, "Foratura", TypeOfProcess.Drilling);

```

When this program will be optimized with the holes SetPriority 1 to 6 will be performed in the same phase, while those without SetPriority may be performed at any stage

3.6 Macros and subprograms

The macros (XSP) realised with Maestro are used to group together a sequence of various instructions/machinings. When the macro is inserted in a program the instructions it contains are executed in sequence.

3.6.1 Creating machining-macros (.xsp)

The “.XSP” Macro are a sequence of instructions used to add geometries or machining operations to a project.

The macros can be launched more than once in the same project and the names associated to them will be transcribed in the workpiece machining tree, as if they were elementary machining operations.

The Maestro installation package contains various “.XSP” macros that can be used as outline to create new macros.

The macro creation procedure can be described in two phases:

1. Creation of the macro basic structure (wizard);
2. Implementation of the functions in the macro.

3.6.1.1 Creation of the macro basic structure (wizard);

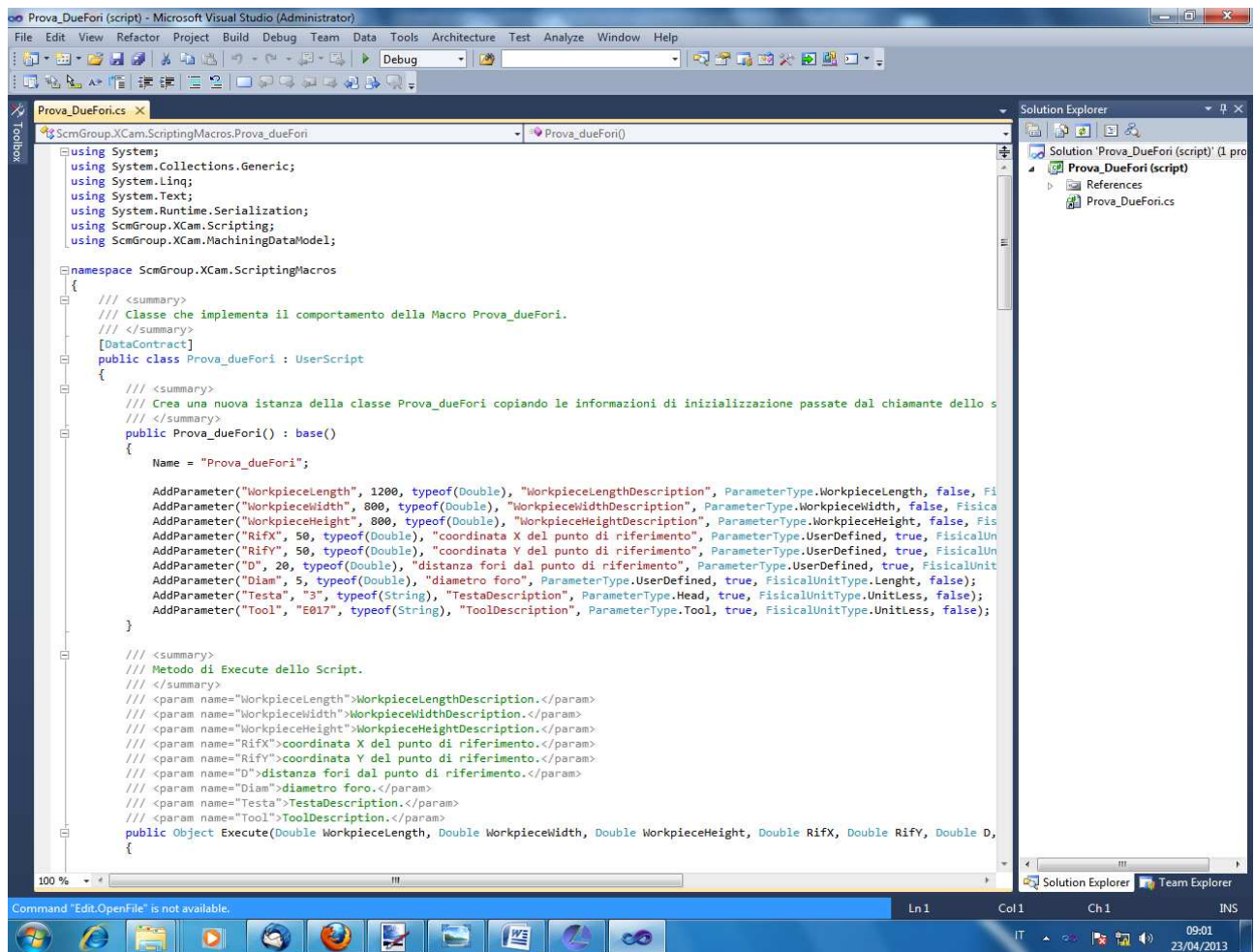
The first phase can be created by executing the wizard supplied in Maestro. See the Maestro manual for the details.

3.6.1.2 Implementation of the functions in the macro.

The second, on the other hand, requires a basic knowledge of the C# programming language and of the scripting primitives in Maestro and described in this manual.

The interactive mode described in the Maestro manual can be used as the macro editor or any other editor can be used to change the .cs file.

The representation in the Microsoft Visual Studio development environment is as follows:



The source code of the macro in question has the following general structure:

1. import of the libraries necessary to the development (System libraries and Maestro Scripting libraries);
2. declaration of the namespace;
3. definition of the class that extends the UserScript basic class and implements the following methods:
 - constructor: defines the parameters and the values supplied with the wizard procedure. This method must not be edited by the user;
 - Execute: this is the method that executes the operations associated to the macro, when it is applied. The body of this method is not implemented. The wizard provides only the method signature: the implementation of the method is left up to the user. The method presents in input the input parameters set for the macro in the wizard procedure. To implement the desired functions the user must use the primitives supplied by the Maestro scripting language. The main functions available are the ones described in this manual, in the chapters on the geometry, machining, etc.;

- Validate: this method, similarly to the Execute method, must be implemented by the user. As opposed to the previous one, its implementation is optional. It is used to create validation tests of the values of the parameters in input to the macro.

See below for a source example, as it appears to the user, before being implemented:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Runtime.Serialization;
using ScmGroup.XCam.Scripting;
using ScmGroup.XCam.MachiningDataModel;

namespace ScmGroup.XCam.ScriptingMacros
{
    /// <summary>
    /// Class that implements the behaviour of the Macro Test_TwoHoles.
    /// </summary>
    [DataContract]
    public class Test_TwoHoles : UserScript
    {
        /// <summary>
        /// Creates a new instance of the class Test_TwoHoles copying the initialization
        information passed from the script caller.
        /// </summary>
        public Test_TwoHoles() : base()
        {
            Name = "Test_TwoHoles";

            AddParameter("WorkpieceLength", 1200, typeof(Double),
"WorkpieceLengthDescription", ParameterType.WorkpieceLength, false, FisicalUnitType.Length, false);
            AddParameter("WorkpieceWidth", 800, typeof(Double),
"WorkpieceWidthDescription", ParameterType.WorkpieceWidth, false, FisicalUnitType.Length, false);
            AddParameter("WorkpieceHeight", 800, typeof(Double),
"WorkpieceHeightDescription", ParameterType.WorkpieceHeight, false, FisicalUnitType.Length, false);
            AddParameter("axisX", 10, typeof(Double), "axisXDescription",
ParameterType.UserDefined, true, FisicalUnitType.Length, false);
            AddParameter("axisY", 10, typeof(Double), "axisYDescription",
ParameterType.UserDefined, true, FisicalUnitType.Length, false);
            AddParameter("Distance", 5, typeof(Double), "DistanceDescription",
ParameterType.UserDefined, true, FisicalUnitType.Length, false);
        }

        /// <summary>
        /// Script Execute Method.
        /// </summary>
        /// <param name="WorkpieceLength">WorkpieceLengthDescription.</param>
        /// <param name="WorkpieceWidth">WorkpieceWidthDescription.</param>
        /// <param name="WorkpieceHeight">WorkpieceHeightDescription.</param>
        /// <param name="axisX">axisXDescription.</param>
        /// <param name="axisY">axisYDescription.</param>
        /// <param name="Distance">DistanceDescription.</param>
        public Object Execute(Double WorkpieceLength, Double WorkpieceWidth, Double
WorkpieceHeight, Double axisX, Double axisY, Double Distance)
        {
            return null;
        }

        /// <summary>
        /// Script Validate Method.
        /// </summary>
        /// <param name="WorkpieceLength">WorkpieceLengthDescription.</param>
        /// <param name="WorkpieceWidth">WorkpieceWidthDescription.</param>
        /// <param name="WorkpieceHeight">WorkpieceHeightDescription.</param>
        /// <param name="axisX">axisXDescription.</param>
        /// <param name="axisY">axisYDescription.</param>
        /// <param name="Distance">DistanceDescription.</param>
        public Object Validate(Double WorkpieceLength, Double WorkpieceWidth, Double
WorkpieceHeight, Double axisX, Double axisY, Double Distance)
        {
        }
    }
}
```

See below for the implementation of the Execute method:

```
public Object Execute(Double WorkpieceLength, Double WorkpieceWidth, Double WorkpieceHeight, Double RifX,
Double RifY, Double D, Double Diam, String Testa, String Tool)
{
    if (D > 0)
    {
        CreateDrill("Hole 1", RifX - D / 2, RifY, 10, Diam, "", TypeOfProcess.Drilling, Tool, Head, 0, -1,
-1, "-1", 0);
        CreateDrill("Hole 2", RifX + D / 2, RifY, 10, Diam, "", TypeOfProcess.Drilling, Tool, Head, 0, -1,
-1, "-1", 0);
    }
    else
    {
        CreateDrill("Hole ", RifX, RifY, 10, Diam, "", TypeOfProcess.Drilling, Tool, Head, 0, -1, -1, "-1",
0);
    }
    return null;
}
```

See below for the implementation of the Validate method:

```
public void Validate(Double WorkpieceLength, Double WorkpieceWidth, Double WorkpieceHeight, Double RifX,
Double RifY, Double D, Double Diam, String Head, String Tool)
{
    if (D < 0) Print("@DError");
    if (Diam < 0) Print("@DiamError");
}
```

For more examples we recommend editing one of the macros supplied in Maestro.

3.6.1.3 Debug of a macro

After having implemented the desired behaviour for the macro, its correct operation can be checked with the code debug operation.

The debug the macro created proceed as follows:

1. Open Maestro (UI00.exe);
2. Open the macro source file .cs with Visual Studio.NET, in the runtime folder (this folder is in the Maestro installation path);
3. Place a breakpoint in the line that the macro source debug must start;
4. From the debug menu of the VS.NET, select "attach to process" and from the active processes select the Maestro UI00.exe
5. Apply the macro in the project in Maestro; when "Apply" is pressed the macro is executed and the debug can be performed starting from the point at the breakpoint inserted.

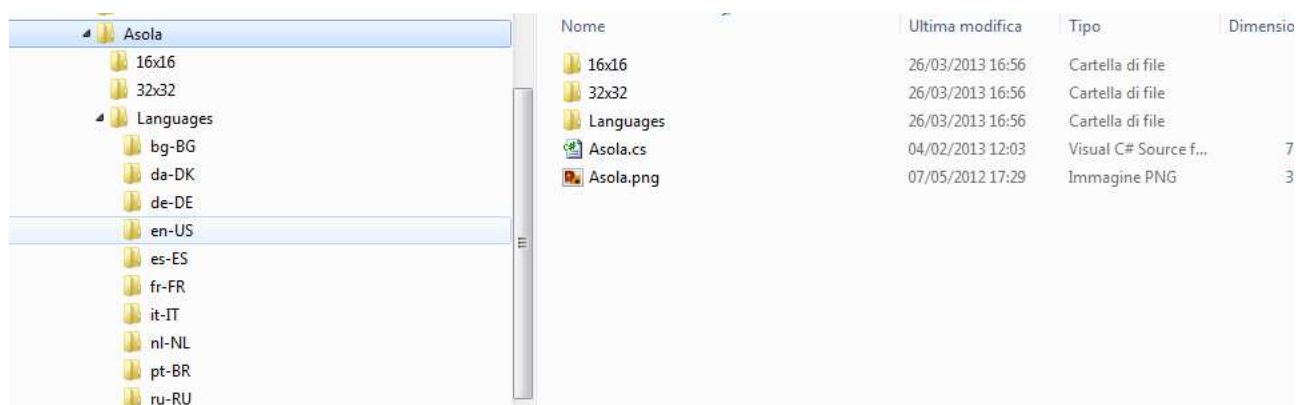
NOTE: the "attach to process" function is not available in the VE.NET Express version, and therefore it cannot be used to debug the macro.

3.6.1.4 Structure of a file .xsp

The wizard procedure generates a file with extension .xsp. It is a file compressed in zip format containing the source codes for the macro created, along with other accessory files. This structure of compressed files must contain the following information:

1. Images
2. Location of the macro in the menus
3. Definition of the parameters
4. Body of the macro function
5. Texts of the translations in the languages managed

See below for an image of the compressed structure in the file .xsp.



The detailed description of this structure lie outside the aim of this manual. The files contain all the information entered with wizard procedure.

For example the images:

- 16x16 pixel image: is in the 16x16 folder;
- 32x32 pixel image: is in the 16x16 folder;
- 200x154 pixel image: is in the same level of the folders in the .xsp file.

3.6.2 Use of .XSP macros (CreateMacro)

To recall a machining-macro inside the script use the CreateMacro instruction. The macro will be applied to the active workplane. Please note that even though the instruction has the “Create” prefix, the command does not generate any new macro, but simply applies an existing macro already defined in the system. Remember that the macros that can be used from script are usually in the Macros folder, in the Maestro installation folder (e.g.: C:\Program Files\Scm Group\Maestro\Macros)

Name	Type	Description
name	string	Macro name
macroName	string	Parametric instruction name
macroParameters	Object[]	Array of the macro input parameters

Table 72: CreateMacro parameters

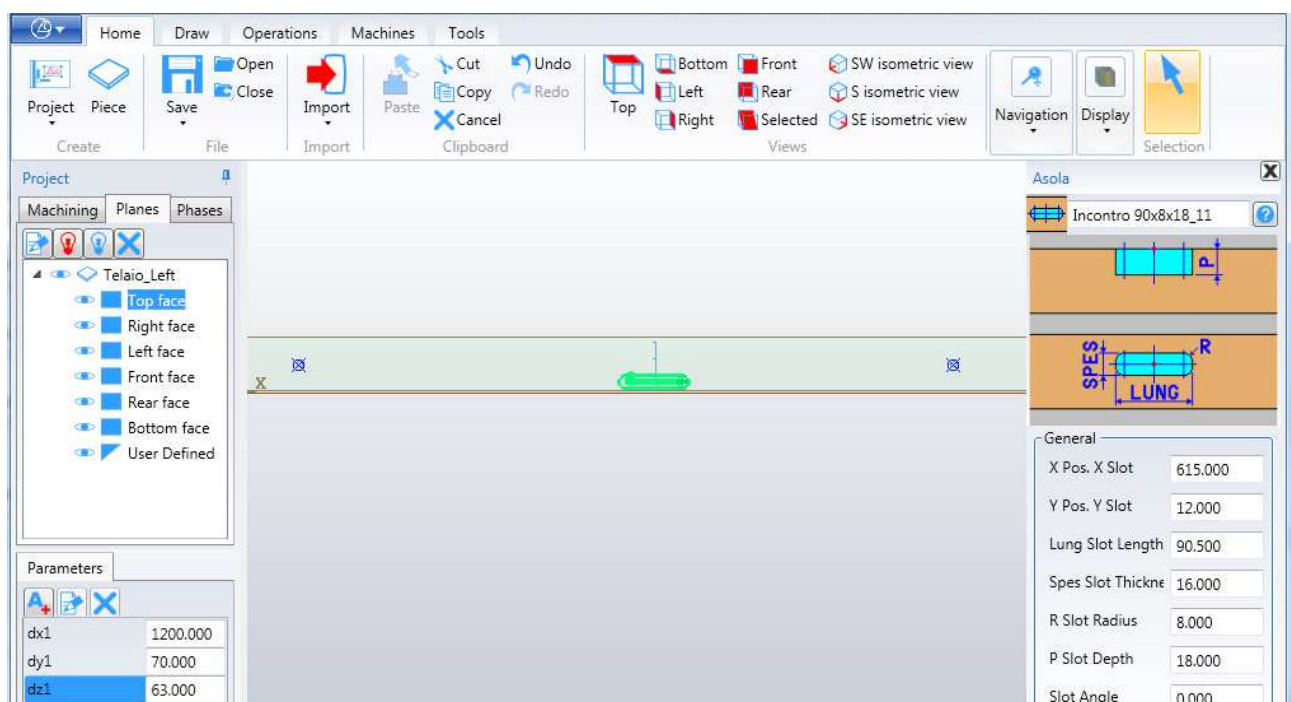
The example below shows the application of machining-macros applied in a script to produce a door-frame. Certain machining-macros are applied on the top side of the workpiece for the anuba hinge, slot and hole. See below for an operation script extract:

```
SelectWorkplane("Top");
CreateMacro("Single Anuba Hinge hole Door-Supporting Frame_7","Anuba",157,35,0,0, true
,7,51,"3","E015",0,0);

SelectWorkplane("Top");
CreateMacro("Single Anuba Hinge hole Door-Supporting Frame_9","Anuba",1003,35,0,0, true
,7,51,"3","E015",0,0);

SelectWorkplane("Top");
CreateMacro("Fastening 90x8x18_11","Slot",615,12,90.5,16,8,18,"3","E035",9,0);

CreateWorkplane("Rostrum Hole_12", 615, 20, 49.5, 0, 90);
CreateMacro("Rostrum Hole_13","Hole",0, 0,20,44,"3","E035",22);
```



3.6.2.1 Use of macros. XSP using SetMacroParam

To recall a macro without having to set up all parameters of the macro you can exclusively set different parameters other than default with the instruction SetMacroParam and then use the CreateMacro without parameters.

The result will be to apply the macro with all default settings other than those which can be more dynamic.

The typical case would be to maintain the geometric data as default and change those relating to position (X, Y).

Name	Type	Description
parName	string	Name of the parameter
value	Object	Parameter value (Double, Int32, String, Bool)

Table 73: Parameters SetMacroParam

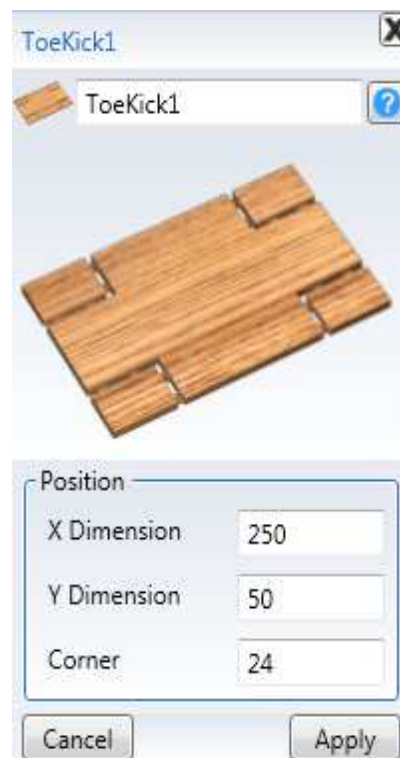
Name	Type	Description
Name	string	Macro name
macroName	string	Name of the parametric instruction

Table 74: CreateMacro parameters (without parameters)

Marco call example ToeKick1

```
SetMacroParam("X", 250);           //Double
SetMacroParam("E", 2);              //Int32
SetMacroParam("Tool","Fresa _D12"); (*Fresa = mill) //String
SetMacroParam("Clockwise",false);  //Boolean
SetMacroParam("Sharp corners", 24); //Int32
```

```
CreateMacro("ToeKick1","ToeKick1");
```



3.6.3 Import Script

To use a workpiece program in .XCS format in the Maestro interactive environment, use the “Import script” command in the submenu “Import” of the “Home” menu.

3.7 NC functions

Used to define and make the machine perform operations different from the machining operations.

The following NC functions can be programmed in the scripting file:

1. Enter an ISO instruction: `CreateIso`
2. Machine parking: `CreatePark`
3. Null operation: `CreateNullOperation`
4. Workpiece probing: `CreateWorkPieceProbing`
5. Display an operator message `CreateMessage`

3.7.1Enter an ISO instruction

The Createlso function is used to enter an ISO instruction. It has the following parameters:

Name	Type	Description
name	string	ISO function name visible in the tree
Instruction	String	ISO instruction to execute
optionalParameters	String	Optional parameters to the ISO instruction
isXiso	bool	Xiso instruction false = it's not a Xiso instruction true = it is a Xiso instruction

Table 75: Createlso Parameters

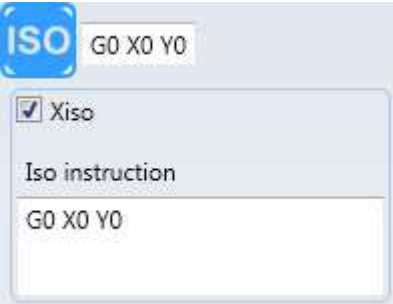
Examples:

Createlso("G0 X0 Y0","G0 X0 Y0","", true);

Machining Tree



Dialogue window



Post generated

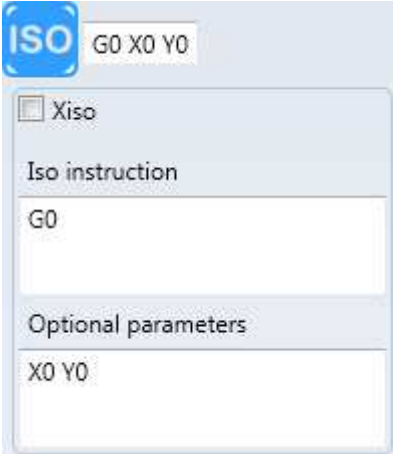
```
*****  
,  
.OP1 ;G0 X0 Y0  
*****  
,  
G0 X0 Y0
```

Createlso("G0", "G0", "X0 Y0", false);

Machining Tree



Dialogue window



Post generated

```
*****  
,  
.OP1 ;G0 X0 Y0  
*****  
,  
ISO "G0" X0 Y0
```


3.7.2 Machine parking

The CreatePark function is used to enter a machine parking. It has the following parameters:

Name	Type	Description
name	string	Parking name visible in the tree
stopType	String	Type of stop to perform: Nothing = No stop NoUnlock = Stop with start wait Unlock = Stop with release and start wait
toMinQuote	bool	Type of parking false = no park at minimum quota true = park at minimum quota

Table 76: CreatePark parameters

The toMinQuote parameter is currently not used, we recommend leaving it always as “false”.

Example:

```
CreatePark("Parking", "NoUnlock", false);
```



3.7.3 Null operation

The CreateNullOperation function is used to perform a null machine operation. It has the following parameters:

Name	Type	Description
name	string	Operation name visible in the tree
X	double	Parking X quota (optional)
Y	double	Parking Y quota (optional)
Q	double	Quotas reference: (optional) 0 = Quotas referred to machine zero 1 = Quotas referred to workpiece zero
speed	double	Movement speed (optional)
spindleEnable	bool	Spindle state (optional) false = Spindle off true = Spindle on
Tool	string	Tool to fit (optional)

Table 77: CreateNullOperation parameters

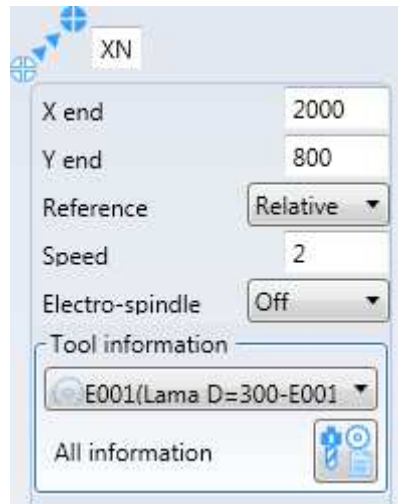
Example:

```
CreateNullOperation("XN", 2000, 800, 1, 2, false, "E001");
```

Machining Tree



Dialogue window



Post Generated:

```
*****
,
.OP1 ;XN
*****
,
XN X=2000 Y=800 Q=1 S=0 V=2000 T=1001
*****
,
```

3.7.4 Workpiece probing

There are three workpiece probing modes:

- 1) Probing with side position correction
- 2) Probing with tool length correction
- 3) Probing with workpiece roto-translation

3.7.4.1 Probing with side position correction

The CreateWorkPieceProbing function is used to perform the probing with side position correction. It has the following parameters:

Name	Type	Description
name	string	Probing name visible in tree
X	double	Probing X quota
Y	double	Probing Y quota
tool	string	Tool for probing
head	string	Probing head
type	int	Type of probing
headReturnAfterProbing	Int	Return mode after probing: 0 = Parking quota 1 = Probing quota 2 = Skimming quota

Table 78: CreateWorkPieceProbing parameters

The type parameter is currently not used, we recommend leaving it always as 0.

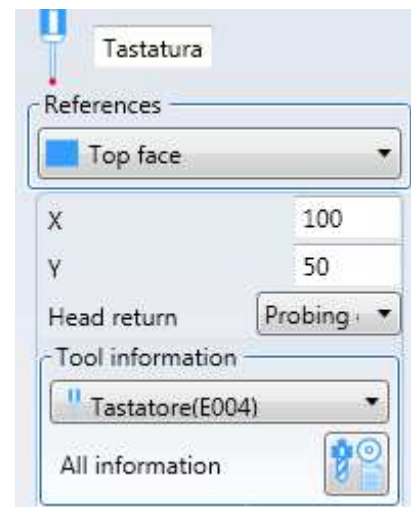
Example:

```
CreateWorkPieceProbing("Probing", 100, 50, "Probe", "3", 0, 1);
```

Machining Tree



Dialogue window



Post Generated:

```
.*****  
,  
.OP1 ;Probing  
.*****  
,  
F=1  
O X=0.000 Y=0.000 Z=0.000 F=1 ;ChangePlane  
XTA X=100 Y=50 Q=1 T=011  
.*****  
,
```

N.B.: use the SelectWorkplane(); to probe the desired side

3.7.4.2 Probing with tool length correction

The CreateWorkPrismaProbing function is used to perform the probing with tool length correction. It has the following parameters:

Name	Type	Description
name	string	Probing name visible in tree
X	object	Probing X quota
Y	object	Probing Y quota
tool	string	Tool to correct
speed	object	Probing speed (optional)

Table 79: CreatePrismaProbing parameters

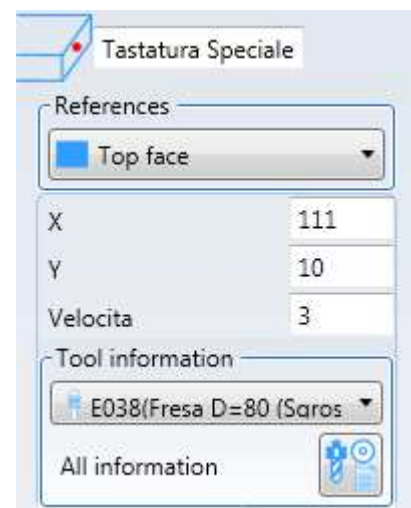
Example:

```
CreatePrismaProbing("Special Probing", 111, 10, "E038", 3);
```

Machining Tree



Dialogue window



Post Generated:

```
*****
;
.OP1 ;Special Probing
*****
;
F=1
O X=0.000 Y=0.000 Z=0.000 F=1 ;ChangePlane
XTASPEC X=111 Y=10 T=1038 F=1 V=3000
*****
;
.OP2 ;Roughing 2 mm of Head_22
*****
;
XG0 X=-107.907 Y=-38.471 Z=20.000 T=1038 P=0 D=20 C=0 V=3000 s=0
XL2P X=-107.907 Y=-38.471 Z=-33.000 V=3000
XTASPECRESET
*****
;
```

N.B.: - The XTASPECRESET instruction is written only if there are other machining operations

- Use the SelectWorkplane(); to probe the desired side

3.7.4.3 Probing with workpiece roto-translation

The CreateWorkPieceTranslationRotationProbing function is used to perform the probing with side with workpiece roto-translation. It has the following parameters:

Name	Type	Description
name	string	Probing name visible in tree
probingXFirstPointQuote	object	First probing point X
probingXSecondPointQuote	object	Second probing point X
probingYPointQuote	object	Probing Y position
probingZQuote	object	Probing Z position
tool	string	Tool for probing
Head	String	Head to use

Table 80: CreateWorkPieceTranslationRotationProbing parameters

Example:

CreateWorkPieceTranslationRotationProbing("Roto Probing", 10, 100, 5, 3, "Probe", "3");

Machining Tree



Dialogue window



Post Generated:

```

*****
;
.OP1 ;Roto Probing
*****
XORGACQ N="ORGX ORGY ANGA" QX1=10.000 QX2=100.000 QY3=5.000 QZ=3.000 T=1004
ROT A=ANGA
*****
;

```

3.7.5 Display an operator message

The CreateMessage function is used to display an operator message. It has the following parameters:

Name	Type	Description
name	string	Operation name visible in the tree
text	string	Text displayed for the operator
waitForUserInput	bool	= true Wait for operator
releaseWorkpiece	bool	= true workpiece release request

Table 81: CreateMessage parameters

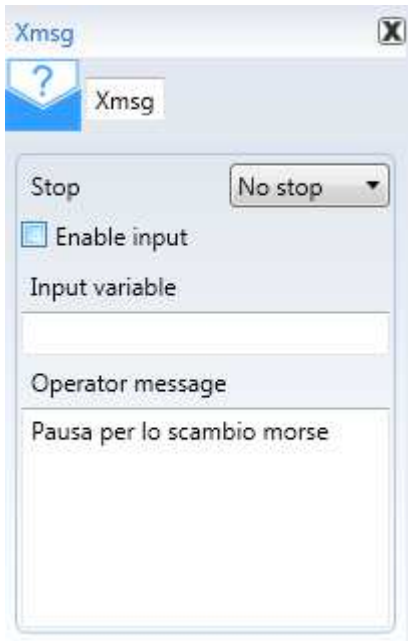
Example:

CreateMessage("Message", "Pause for clamps exchange", true, true);

Machining Tree



Dialogue window



Post Generated:

```
.*****  
,  
.OP1 ;Message  
.*****  
,  
XMSG N="Pause for clamps exchange" Q=2 I=0  
O X=0 Y=0 Z=0  
.*****  
,
```

3.8 Workplans

When a new project is generated a default workplan, called “Setup”, is created automatically. This is also the case when executing a scripting project. In order to generate further workplans, as well as the default workplan, use the CreateWorkplan instruction. All the operations that follow this instruction will be associated to the new workplan. The previous instructions will be linked to the previously defined workplan. This can be the setup, if no workplan has been defined or a workplan defined with a previous CreateWorkplan instruction.

The table below lists the input parameters for the instruction.

Name	Type	Description
Name	String	Name of the workplan created

Table 82: CreateWorkplan parameters

The application example below shows creation of a workplan called “Swap”, after the default one:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0, 0);

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, .5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_30", 1200, 0, 1200, 70);
SetRetractStrategy(true, true, .5, 0);
SetPneumaticHoodPosition(4);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_31",65,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting
,"E001","3",2,-1,-1.6,0);

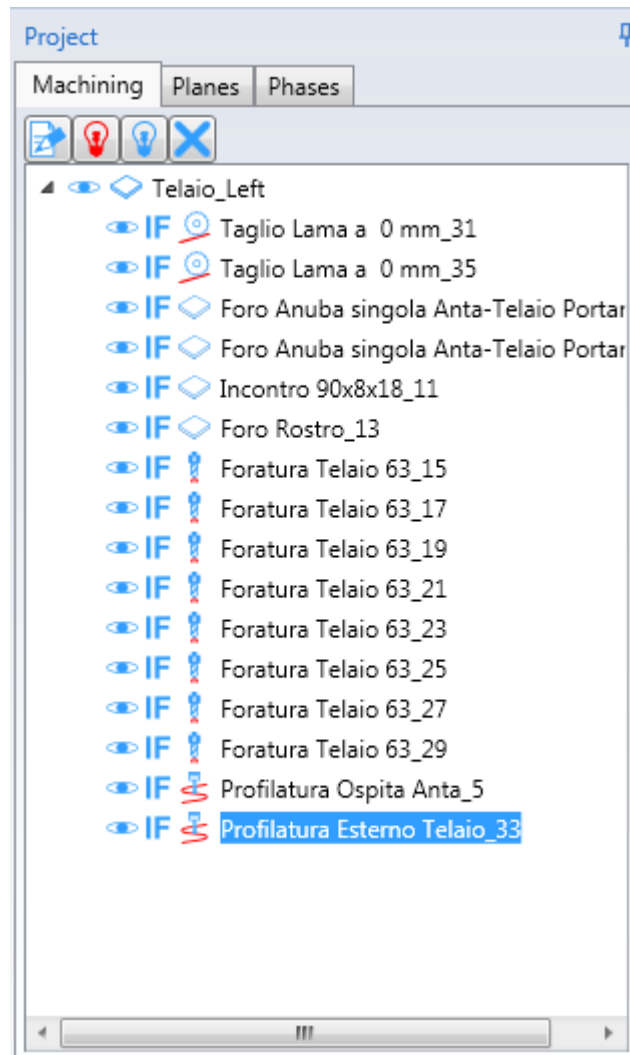
... (continue)

CreateWorkplan("Swap");
SetWorkpieceSetupPosition(10,435,0, 0);
//internal blocking profile
CreateBlockingProfile("Frame_Left_3",0,0, -17);
AddSegmentToBlockingProfile(1200.0, "");

ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 3);
SetDustpanOffset(10, 1, false);
CreatePolyline("Frame External Routing_32",1200,70);
AddSegmentToPolyline(0.70);
SetRetractStrategy(false, true, 3, 0);
SetPneumaticHoodPosition(1);
```

```
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing_33",65,"Frame External Routing",
TypeOfProcess.GeneralRouting ,"E200","3",2,-1,-1,6,0);
```

Nell'albero della fasi di lavoro è possibile osservare la fase creata ("Swap") e la lavorazione ad essa associata ("Profilatura Esterno Telaio_33").



When a new workplan is created it becomes an active workplan for the subsequent operations. In order to activate a different workplan from the current one in the script, it must be recalled with the `SelectWorkplan` instruction. Please note that the workplan must have been created previously to be activated.

The table below lists the input parameters for the `SelectWorkplan` function.

Name	Type	Description
Name	string	Name of the workplan to activate

Table 83: SelectWorkplan parameters

3.9 Workpiece setup in machine

The following must be set, based on the machine that the operation is performed on:

1. workpiece position on the machine plane;
2. workpiece orientation.

3.9.1 The SetMachiningParameters instruction

This instruction is used to configure the machine parameters used to execute the program. Generally this function must be the first instruction in the script, because it configures the main parameters for the correct execution of the program.

The table below shows the input parameters for the instruction.

Name	Type	Description
executionFields	String	Work area
repetitions	Int32	Number of repetitions
tableOptions	Int64	Type of blocking configuration
mechanicalOptions	Int64	Mechanical options configuration
continuousCycle	Bool	Enables the continuous cycle execution

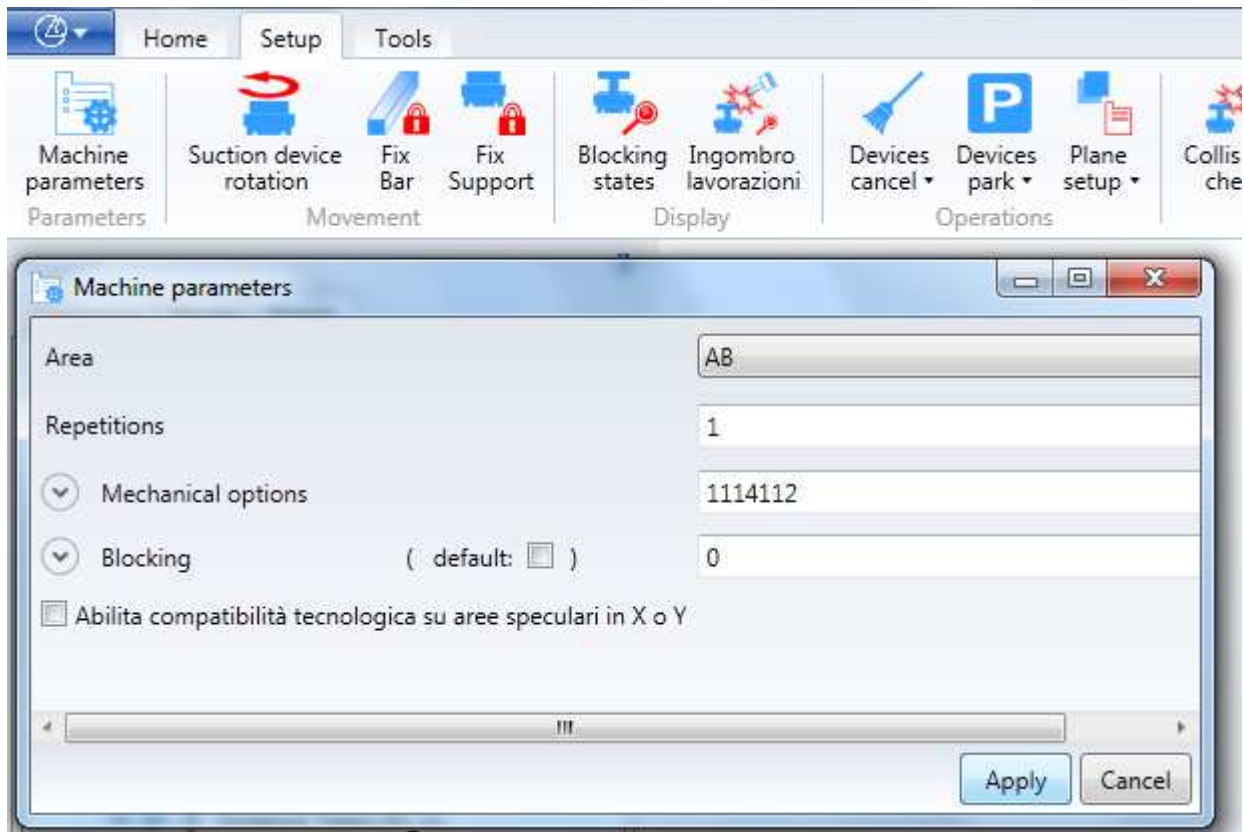
Table 84: SetMachiningParameters parameters

The example below shows an extract code where the SetMachiningParameters instruction is applied on an operation in AB area machine that requires only one execution repetition:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0, 0);
```

... (continue)

The machine parameters set can be seen in Maestro, by clicking *Machine parameters*.



3.9.1.1 Reading machine parameters with GetMachiningParameters

The unit parameters can be read by using GetMachiningParameters.

The function has no parameters and can extract the following data:

- Continuous cycle
- Work area
- Mechanical options
- Locking Options
- Technology on mirrored areas

3.9.1.1.1 Continuous cycle

You can check if the program must be run in a continuous loop

Example:

```
MachineParameters mp = GetMachiningParameters();  
bool CicloContinuo;  
CicloContinuo = mp.IsContinuousCycle;
```

3.9.1.1.2 Work area

You can check the work area on which to run the program.

Example:

```
MachineParameters mp = GetMachiningParameters();  
string Field;  
Field = mp.Field;
```

3.9.1.1.3 Mechanical options

You can check the mechanical options of the machine.

Example:

```
MachineParameters mp = GetMachiningParameters();  
Int64 OpzioniMeccaniche;  
OpzioniMeccaniche = mp.MechanicalOptions;
```

3.9.1.1.4 Locking options

You can check the blocking options of the plan.

Example:

```
MachineParameters mp = GetMachiningParameters();  
Int64 BloccaggioPiano;  
BloccaggioPiano = mp.TableOptions;
```

3.9.1.1.5 Technology on mirrored areas

You can check whether the machining should be carried out keeping the technology on the mirrored X or Y areas.

Example:

```
MachineParameters mp = GetMachiningParameters();  
bool TecnologiaSuAreeSpeculari;  
TecnologiaSuAreeSpeculari = mp.IsTechnologicalMirror;
```

3.9.1.2 Reading of Work area parameter with GetWorkingField

The work area parameters can be read using the instruction GetWorkingField.

The function can extract the following data:

- Work area symmetry
- Work area origin
- Work Area Dimensions
- First bar of the Area number
- Work area bar number
- Area quadrant
- Reference bar code

Nome	Tipo	Descrizione
Fields	String	Area di lavoro

Tabella 85: Parametri GetWorkingField

3.9.1.2.1 Work Area symmetry

You can check if the work area is mirrored in X or Y

Example:

```
WorkingField wf = GetWorkingField(Field);  
bool SpecX, SpecY;  
SpecX = wf.SpecularityX; // Indicates whether the work area is symmetric in X.  
SpecY = wf.SpecularityY; // Indica se l'area di lavoro è simmetrica in Y.
```

3.9.1.2.2 Work area origin

You can check the origin of the work area

Example:

```
WorkingField wf = GetWorkingField(Field);  
double OX, OY, OZ;  
OX = wf.OrigineX; // Indicates the origin in X of the work area.  
OY = wf.OrigineY; // Indica l'origine in Y dell'area di lavoro.  
OZ = wf.OrigineZ; // Indica l'origine in Z dell'area di lavoro.
```

3.9.1.2.3 Work Area Dimensions

You can control the size of the work area

Example:

```
WorkingField wf = GetWorkingField(Field);  
double Lung, Larg;  
Lung = wf.Length; // Indicates the length of the work area.  
Larg = wf.Width; // Indicates the width of the work area.
```

3.9.1.2.4 Number of the first bar of the Area

You can check the number of the first work area bar

Example:

```
WorkingField wf = GetWorkingField(Field);  
int PrimaBarra;  
PrimaBarra = wf.FirstLockingBarNumber;
```

3.9.1.2.5 Work area bar number

You can check the number of the bars in the work area

Example:

```
WorkingField wf = GetWorkingField(Field);  
int QtBarre;  
QtBarre = wf.LockingBarQuantity;
```

3.9.1.2.6 Area quadrant

You can check which quadrant is in the work area

Example:

```
WorkingField wf = GetWorkingField(Field);  
int Quadrante;  
Quadrante = wf.PiecesDispositionQuadrant;
```

3.9.1.2.7 Reference bar code

You can check the reference bar code associated with the area.

Example:

```
WorkingField wf = GetWorkingField(Field);  
string CodiceBarra;  
CodiceBarra = wf.ReferenceBarUniqueCode;
```

3.9.2 Workpieces positioning on machine plane

To set the workpiece position on the machine plane (position referred to the active workplan) use the `SetWorkpieceSetupPosition` instruction.

The table describes the parameters of the function.

The workpiece point that is positioned is the edge of the finished workpiece, that has the following minimum quotas: $X=0, Y=0, Z=0$

Name	Type	Description
x	double	Position X of workpiece.
y	double	Position Y of workpiece.
z	double	Position Z of workpiece.
zRot	double	Workpiece rotation in relation to axis Z

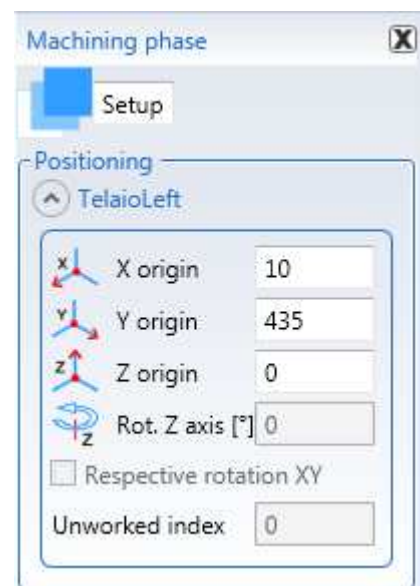
Table 86: SetWorkpieceSetupPosition parameters

The example below shows an extract of code where the `SetWorkpieceSetupPosition` instruction is applied for the “Setup” workplan:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0,0);
... (continue)
```

The workpiece is positioned at point P(10,435,0) of the machine plane, as can be seen in Maestro in the “Setup” workplan properties and with null rotation in relation to axis Z.

For expert users there is also the `GetWorkPieceSetupPosition()` instruction that returns the “Class” `WorkPieceSetupPosition` containing all the information on the workpiece position in the current phase shown in the image at the side.



See below for an example where the position x and y assumed by the workpiece can be printed:

```
WorkPieceSetupPosition ClasseDelPezzo;  
ClasseDelPezzo = GetWorkPieceSetupPosition();  
  
double posx = 0, posy = 0;  
posx = ClasseDelPezzo.LocationX;  
posy = ClasseDelPezzo.LocationY;  
  
String s;  
  
s = String.Format("Posizione X = {0}  posizione Y = {1} ", posx, posy);  
Print(s);
```

Executing the program in the example will print the string indicating the position of the workpiece.

3.10 Definition of the blocking system

There are two types of blocking systems on the work centres:

1. Clamps blocking
2. Suction cups blocking

3.10.1 Clamps blocking

To use the clamps one or more blocking profiles must be defined to use the automatic setup of the clamps in every phase. The “blocking profile” is optional for the manual positioning but compulsory for the automatic positioning.

3.10.1.1 Clamps positioning on the workpiece

3.10.1.1.1 Manual clamps positioning

To configure the blocking bar position use the SetBarPosition instruction. The configuration is associated to the current main workplan. The input parameters for the instruction are described in the table.

Name	Type	Description
id	int	Position bar id (value between 1 and the max number of bars).
xPosition	double	Bar position along axis X (referred to the selected reference system for the workpiece being machined).

Table 87: SetBarPosition parameters

To configure the position of a support base along a bar use the SetSupportBasePosition instruction. The configuration is for the current main workplan. The support base is associated to the bar for which the position has been set in the last previous instruction of the script. The input parameters for the instruction are described in the table.

Name	Type	Description
id	int	Support base id (value between 1 and the max number of support bases).
yPosition	double	Support base position along axis Y (referred to the selected reference system for the workpiece being machined).

Table 88: SetSupportBasePosition parameters

Note: These two instructions can also be used for the suction cups positioning.

To set the clamp position along a bar, use the `SetClampPosition` instruction. The configuration is for the current main workplan. The clamp is associated to the bar for which the position has been set in the last previous instruction of the script. The input parameters for the instruction are described in the table.

The `SetSupportBasePosition` does not have to be used too.

Name	Type	Description
supportId	int	Clamp id (if the table is motor driven: identifies the support base where the clamp must be fitted or moved if the table is manual: identifies if there is the clamp or it creates it).
yPosition	double	Clamp position along axis Y (referred to the selected reference system for the workpiece being machined).
status	int	Blocking status: 0 = closed; 1 = open; 2 = closed on the workpiece.
code	string	Clamp code

Table 89: SetClampPosition parameters

To set the position of a horizontal clamp along a bar use the `SetHorizontalClampPosition` instruction. The configuration refers to the current main workplan. The horizontal clamp is associated to the bar for which the position has been set in the last previous instruction of the script. The input parameters for the instruction are described in the table.

The `SetSupportBasePosition` does not have to be used too.

Name	Type	Description
supportId	int	Clamp id (if the table is motor driven: identifies the support base where the clamp must be fitted or moved if the table is manual: identifies if there is the clamp or it creates it).
yPosition	double	Clamp position along axis Y (referred to the selected reference system for the workpiece being machined).
status	int	Blocking status: 0 = closed; 1 = open; 2 = closed on the workpiece.
code	string	Clamp code

Table 90: SetHorizontalClampPosition parameters

See below for an example with a workpiece blockage.

The examples below show the application of some of the instruction described above.

The example shows a workpiece blockage with a motor driven table machine and clamps blocking. The position of the bars and the clamps has been configured based on the operations being performed.

In particular, the operation process consists of two workplans. "Setup" and "Swap". A different clamps blocking has been configured for each one, similarly to the example in chapter 3.10.1.1.2. In the passage between workplans the position of the clamps has been changed for the requirements of the next workplan. Please not that the use of the workpiece blocking instructions requires the user to pay the utmost attention to avoid possible collisions caused by the movements of the bars and clamps.

```

SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);

int xPos = 10;
int yPos = 435;

SetWorkpieceSetupPosition(xPos,yPos,0, 0);

//WORKPIECE BLOCKING

//Position on the workpiece
// bar 4: out, open
SetBarPosition(4, 2300-xPos);
SetClampPosition(1, 1360-yPos, 1, "H110-A-145");
SetClampPosition(2, 1522-yPos, 1, "H110-A-145");
SetClampPosition(3, 1685-yPos, 1, "H110-A-145");

// bar 3:
SetBarPosition(3, 1935-xPos);
SetClampPosition(1, 1360-yPos, 1, "H110-A-145");
SetClampPosition(2, 1522-yPos, 1, "H110-A-145");
SetClampPosition(3, 1685-yPos, 1, "H110-A-145");

// bar 2:
SetBarPosition(2, 800);
SetClampPosition(1, -2-yPos, 1, "H110-A-145");
SetClampPosition(2, 527-yPos, 1, "H110-A-145");
SetClampPosition(3, 1685-yPos, 1, "H110-A-145");

// bar 1:
SetBarPosition(1, 400);
SetClampPosition(1, -2-yPos, 1, "H110-A-145");
SetClampPosition(2, 527-yPos, 1, "H110-A-145");
SetClampPosition(3, 1685-yPos, 1, "H110-A-145");

//Clamps closure
// bar 2:
SetBarPosition(2, 800);
SetClampPosition(2, 527-yPos, 2, "H110-A-145");
// bar 1:
SetBarPosition(1, 400);
SetClampPosition(2, 527-yPos, 2, "H110-A-145");

... (continue)

CreateRoughFinish("Door Housing Routing_5",63,"Door Housing Routing",
TypeOfProcess.GeneralRouting ,"E152","3",2,-1,-1,6,0);

CreateWorkplan("Swap");
SetWorkpieceSetupPosition(10,435,0, 0);

//WORKPIECE BLOCKING
//Clamps opening
// bar 2:
SetBarPosition(2, 800);
SetClampPosition(2, 527-yPos, 1, "H110-A-145");
// bar 1:

```

```

SetBarPosition(1, 400);
SetClampPosition(2, 527-yPos, 1, "H110-A-145");
//Position on the workpiece
// bar 4: out, open
SetBarPosition(4, 2300-xPos);
SetClampPosition(1, -2-yPos, 1, "H110-A-145");
SetClampPosition(2, 160-yPos, 1, "H110-A-145");
SetClampPosition(3, 322-yPos, 1, "H110-A-145");
// bar 3:
SetBarPosition(3, 1935-xPos);
SetClampPosition(1, -2-yPos, 1, "H110-A-145");
SetClampPosition(2, 160-yPos, 1, "H110-A-145");
SetClampPosition(3, 322-yPos, 1, "H110-A-145");

// bar 2:
SetBarPosition(2, 800);
SetClampPosition(1, 402-yPos, 1, "H110-A-145");
SetClampPosition(2, 1522-yPos, 1, "H110-A-145");
SetClampPosition(3, 1685-yPos, 1, "H110-A-145");
// bar 1:

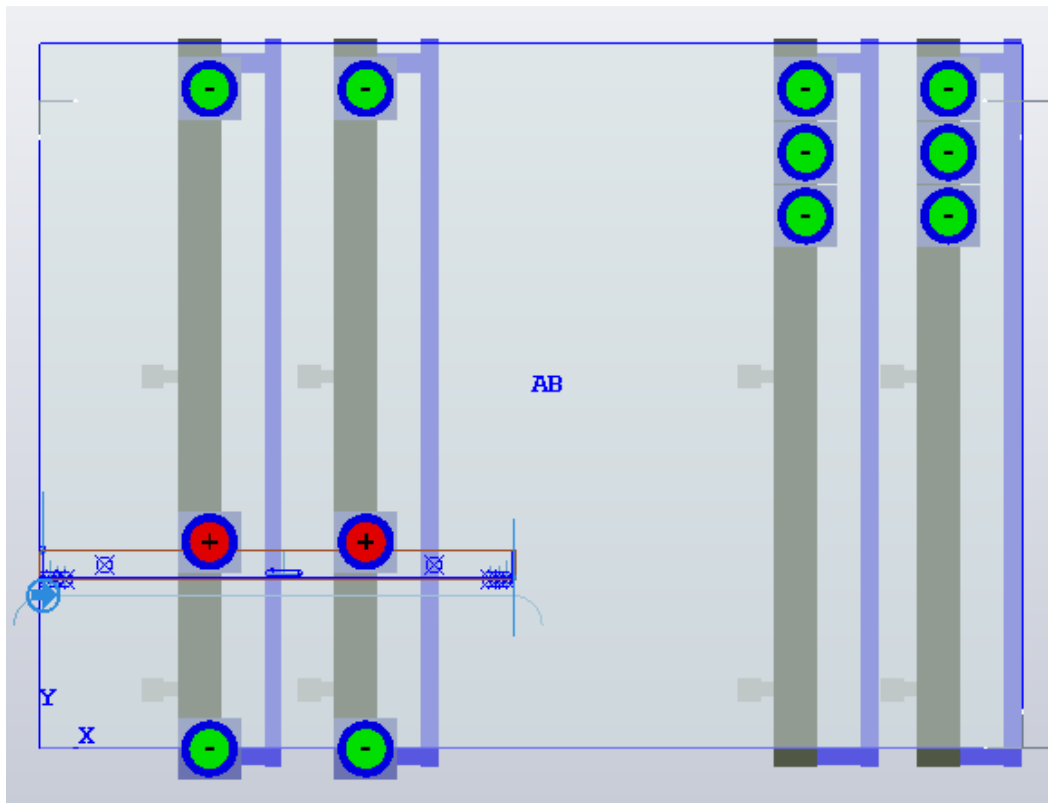
SetBarPosition(1, 400);
SetClampPosition(1, 402-yPos, 1, "H110-A-145");
SetClampPosition(2, 1522-yPos, 1, "H110-A-145");
SetClampPosition(3, 1685-yPos, 1, "H110-A-145");

//Clamps closure
// bar 2:
SetBarPosition(2, 800);
SetClampPosition(1, 402-yPos, 2, "H110-A-145");
// bar 1:
SetBarPosition(1, 400);
SetClampPosition(1, 402-yPos, 2, "H110-A-145");

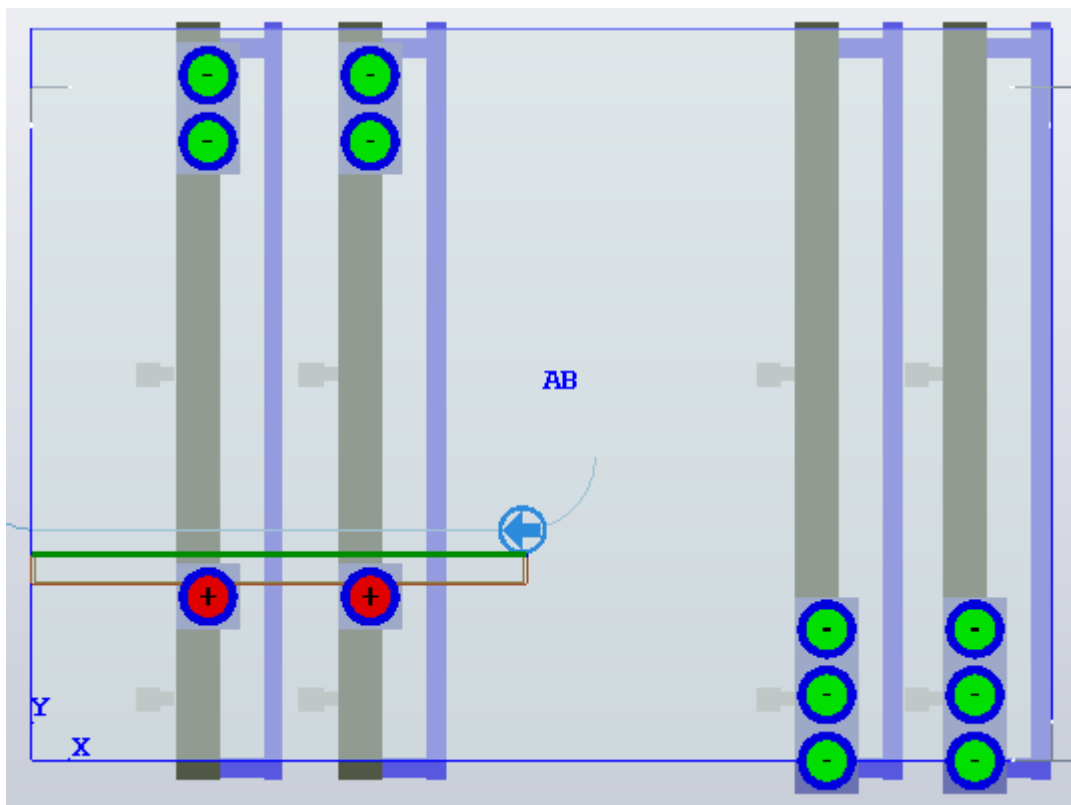
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 3);
SetDustpanOffset(10, 1, false);
CreatePolyline("Frame External Routing_32",1200,70);
AddSegmentToPolyline(0.70);
SetRetractStrategy(false, true, 3, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing_33",65,"Frame External Routing",
TypeOfProcess.GeneralRouting ,"E200","3",2,-1,-1,6,0);

```

Image related to a blockage realised for the “Setup” phase.



See below for the image showing the type of blocking used for the “Swap”.



3.10.1.1.2 Position clamps with Blocking profile

The blocking profile defines which side or sides are useful to block a specific workpiece with the clamps system.

The following must be defined to create a blocking profile:

1. the blocking profile geometry;
2. minimum clamping, that is the minimum distance of the external edge of the clamp from the blocking profile (defines the minimum workpiece section that will be blocked with the clamp).

The maximum clamping is not considered a descriptive specification of the blocking profile, as it depends exclusively on the clamp geometry: it is not considered a property of the blocking profile, but a specification of the type of clamps used (configuration parameter). Its value is used with the minimum clamping value in the clamps setup on the workpiece operations.

MSL considers the creation of a blocking geometry like a polyline geometry (for more information on how to construct a generic polyline, see the relative chapter) and provides the following instructions:

- `CreateBlockingProfile`: creates a new blocking profile, defining the profile start point;
- `AddSegmentToBlockingProfile`: adds a segment to the blocking profile;
- `AddArc3PointsToBlockingProfile`: adds an arc to the blocking profile, with three points of the arc;
- `AddArc2PointCenterToBlockingProfile`: adds an arc to the blocking profile with two points of the arc and the centre point
- `AddArc2PointRadiusToBlockingProfile`: adds an arc to the blocking profile with two points of the arc and the radius;
- `AddArcCenterAngleToBlockingProfile`: adds an arc to the blocking profile with the centre point and the arc angle;
- `AddSegmentTanToBlockingProfile`: adds a tangent segment to the blocking profile;
- `AddArcTanToBlockingProfile`: adds a tangent arc to the blocking profile;
- `CloseBlockingProfile`: closes the blocking profile with a segment that joins the start point of the profile with the final point.

See below for the description tables of the input parameters for the instructions.

Name	Type	Description
name	string	Name of the blocking profile
startX	double	Coordinate X of the first blocking profile point.
startY	double	Coordinate Y of the first blocking profile point.
offset	double	Minimum clamping (def. 0).

Table 91: CreateBlockingProfile parameters

Name	Type	Description
endX	double	Coordinate X of the final blocking profile point.
endY	double	Coordinate Y of the final blocking profile point.
nameElement	string	Name of added element

Table 92: AddSegmentToBlockingProfile parameters

Name	Type	Description
p2X	double	Coordinate X of point 2
p2Y	double	Coordinate Y of point 2
p3X	double	Coordinate X of point 3
p3Y	double	Coordinate Y of point 3
nameElement	string	Name of added element

Table 93: AddArc3PointsToBlockingProfile parameters

Name	Type	Description
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
isClockwise	bool	Clockwise direction
nameElement	string	Name of added element

Table 94: AddArc2PointCenterToBlockingProfile parameters

Name	Type	Description
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
radius	double	Arc radius
isClockwise	bool	Clockwise direction
isOver180	bool	Arc angle greater than 180°
nameElement	string	Name of added element

Table 95: AddArc2PointRadiusToBlockingProfile parameters

Name	Type	Description
centerX	double	Coordinate X of centre point
centerY	double	Coordinate Y of centre point
angle	double	Arc angle
nameElement	string	Name of added element

Table 96: AddArcCenterAngleToBlockingProfile parameters

Name	Type	Description
length	double	Segment length
nameElement	string	Name of added element

Table 97: AddSegmentTanToBlockingProfile parameters

Name	Type	Description
endX	double	Coordinate X of final point
endY	double	Coordinate Y of final point
option	int	The following values are allowed: 0 = direct tangent; 1 = inverse tangent; 2 = clockwise arc; 3 = counterclockwise arc.
nameElement	string	Name of added element

Table 98: AddArcTanToBlockingProfile parameters

Name	Type	Description
nameElement	string	Name of added element

Table 99: CloseBlockingProfile parameters

The example shows a code extract from a script used to produce a frame on which two blocking profiles are created, both with segment geometry, but with different minimum clamping values:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Frame_Left",1200,70,63);
CreateRawWorkpiece("Frame_Left",10,10,2,0,0,0);
//external blocking profile
CreateBlockingProfile("Frame_Left_2",1210,70,-5);
AddSegmentToBlockingProfile(-10.70, "");
SetWorkpieceSetupPosition(10,435,0, 0);

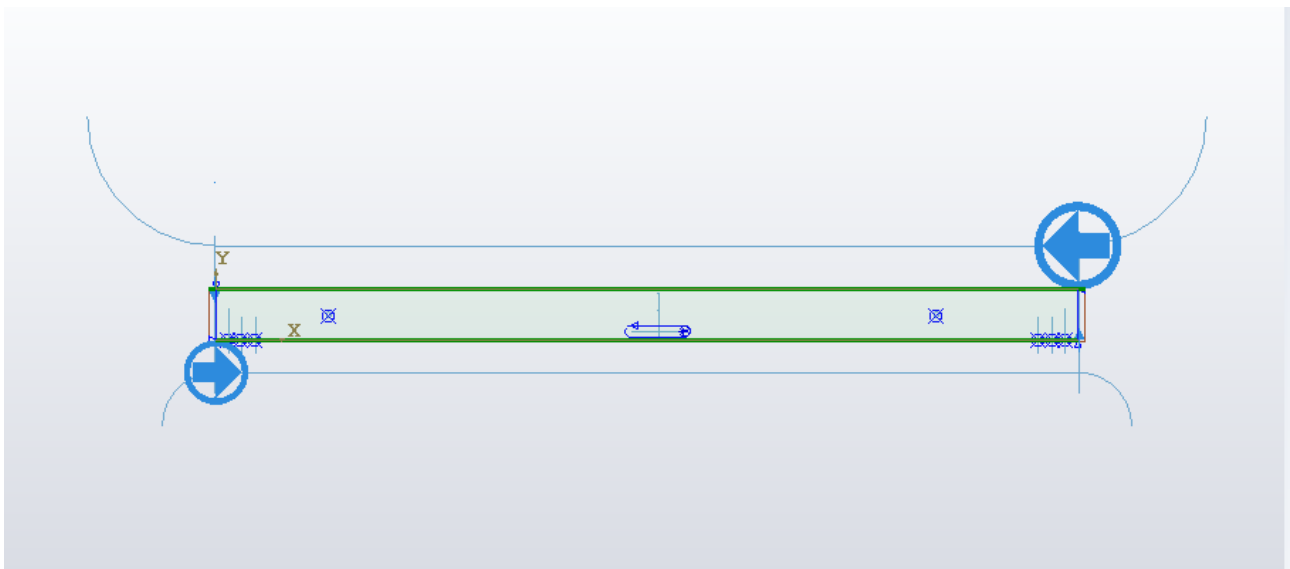
... (continue)

CreateWorkplan("Swap");
SetWorkpieceSetupPosition(10,435,0, 0);
//internal blocking profile
CreateBlockingProfile("Frame_Left_3",0,0, -17);
AddSegmentToBlockingProfile(1200.0, "");

... (continue)
```

Please note that the blocking profiles are associated to two different workplans. the first for the “Setup”, whilst the second for the “Swap”. Two different profiles have been defined because the side used for the workpiece blockage in the first workplan is the side being machined in the second phase.

The profiles created can be seen in Maestro, with the green colour of the profile line. The automatic clamps setup rules will be applied to this line.



In order to configure the clamps positioning, without using the automatic setup rules, the blocking profile definition no longer applies. In this case the user must configure the clamps positioning with the instructions described in chapter **Errore. L'origine riferimento non è stata trovata.**

3.10.2 Workpiece blocking with suction cups

The suction cups blocking system, unlike the clamp system, does not require the blocking definition. This system can be used with a blocking bars system and with a “multi-function” table. In the first case the suction cups must be fitted on the bars support bases (similarly to the clamps), in the second case on the multifunction table.

In order to configure the suction cups positioning, without the automatic setup rules, the position of each suction cups in relation to the workpiece being blocked must be defined for both solutions.

For the suction cups positioning you must firstly perform the bars and bases positioning with the `SetBarPosition` and `SetSupportBasePosition` instructions (See chapter 3.10.1.1.1)

To set the suction cup position along a bar, use the `SetSuctionCupPosition` instruction. The configuration is for the current main workplan. The suction cup is associated to the bar for which the position has been set in the last previous instruction of the script. The input parameters for the instruction are described in the table.

The `SetSupportBasePosition` does not have to be used too.

Name	Type	Description
supportId	int	Suction cup id (if the table is motor driven: identifies the support base where the suction cup must be fitted or moved if the table is manual: identifies the suction cup, if it exists or it creates it).
yPosition	double	Suction cup position along axis Y (referred to the selected reference system for the workpiece being machined).
angle	double	Suction cup rotation angle (if it supports rotation).
code	string	Suction cup code

Table 100: SetSuctionCupPosition parameters

```

Example of workpiece blocked with suction cups :
SetMachiningParameters("AB", 1, 12, 1114112, false);
CreateFinishedWorkpieceBox("Telaio_Left",600,70,63);
CreateRawWorkpiece("Telaio_Left",10,10,2,0,0,0);
//profilo bloccaggio esterno
CreateBlockingProfile("Telaio_Left_2",610,70,-5);
AddSegmentToBlockingProfile(-10,70, "");
SetWorkpieceSetupPosition(10,825,0, 0);

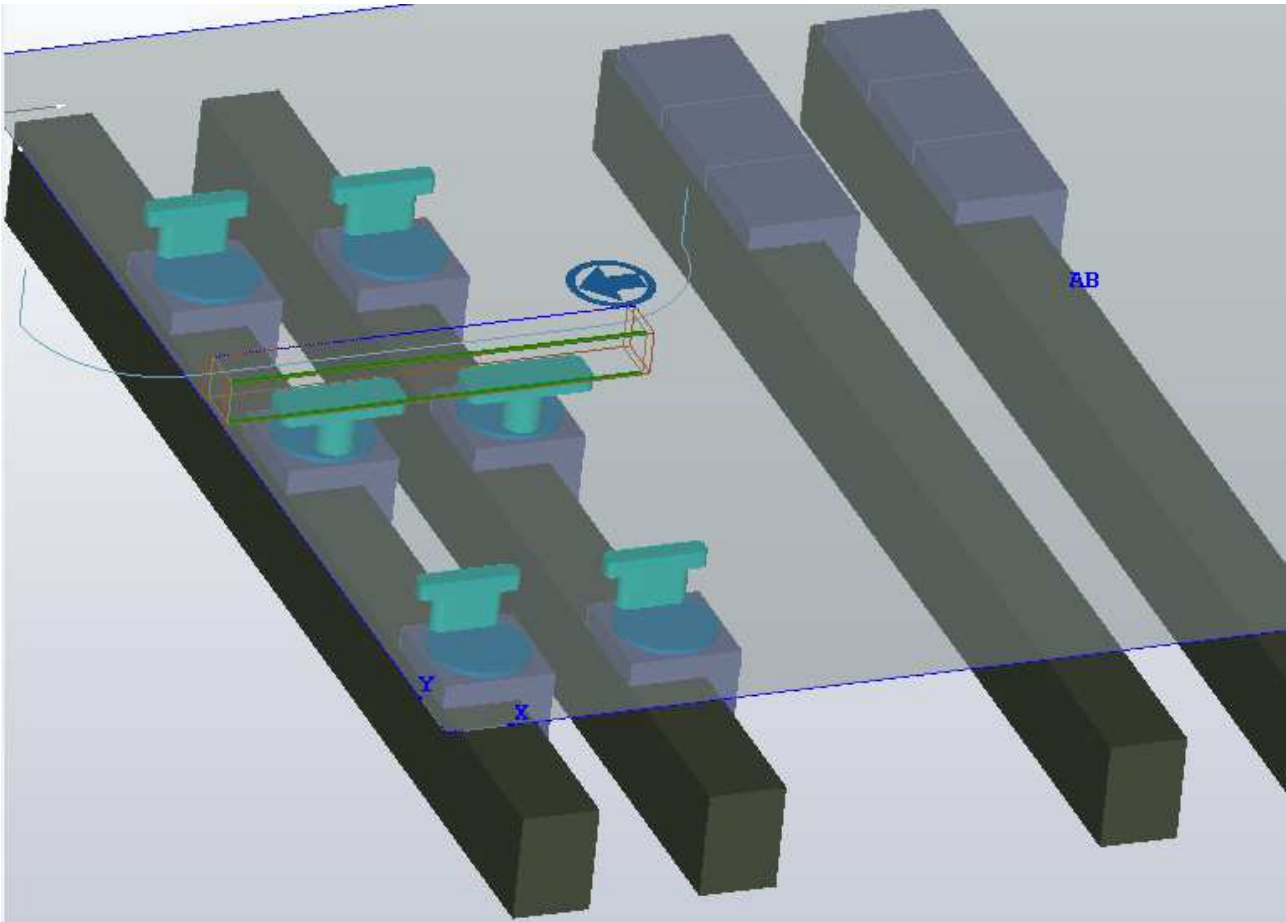
```

```

SetBarPosition(1, 150);
SetSupportBasePosition(2,100);
SetSuctionCupPosition(3,500,0,"H110-A-145x30");
SetSuctionCupPosition(2,74,180,"H110-A-180x65");
SetSuctionCupPosition(1,-500,0,"H110-A-145x30");
SetBarPosition(2, 420);
SetSupportBasePosition(2,100);
SetSuctionCupPosition(3,500,0,"H110-A-145x30");
SetSuctionCupPosition(2,74,180,"H110-A-180x65");
SetSuctionCupPosition(1,-500,0,"H110-A-145x30");
SetBarPosition(3, 1000);
SetBarPosition(4, 1300);

```

Image related to the example:



To set the suction cup position on a MLTF (multifunction) table, use the SetMLTFSuctionCupPosition instruction. The configuration is for the current main workplan. The instruction does not apply to bar tables. The input parameters for the instruction are described in the table.

Name	Type	Description
xPosition	double	Suction cup position along axis X (referred to the selected reference system for the workpiece being machined).
yPosition	double	Suction cup position along axis Y (referred to the selected reference system for the workpiece being machined).
angle	double	Suction cup rotation angle (if it supports rotation).
code	string	Suction cup code

Table 101: SetMLTFSuctionCupPosition parameters

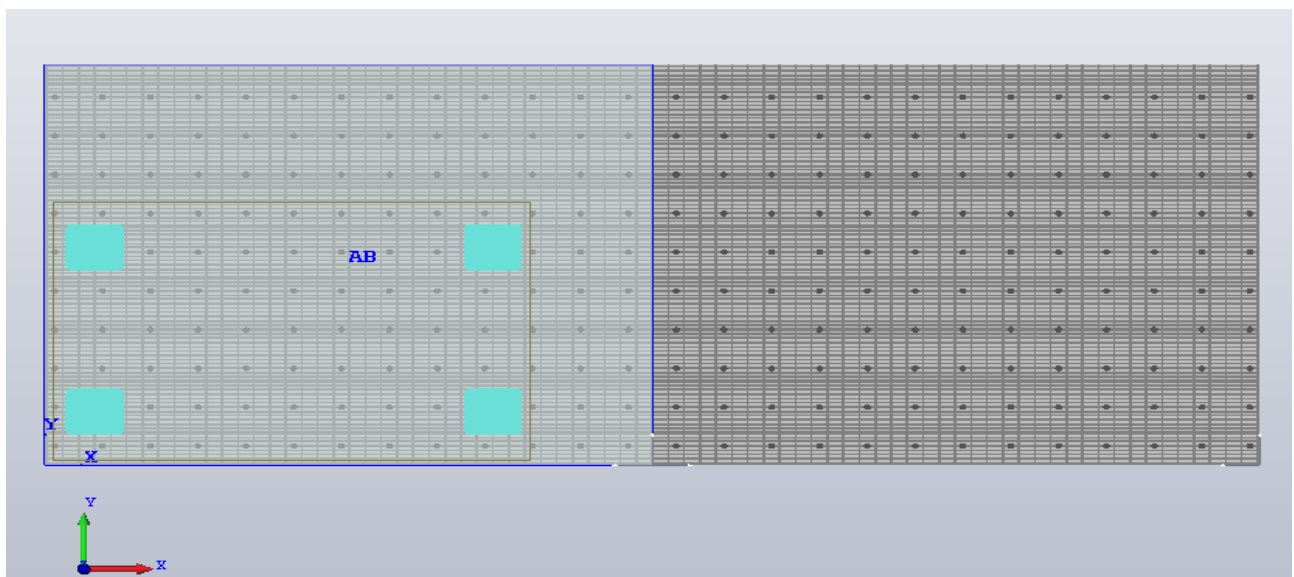
In order to reproduce the workpiece blocking, by way of an example, with a multifunction table machine and with suction cups, use the SetMLTFSuctionCupPosition instruction:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("example",1200,800,63);

SetWorkpieceSetupPosition(20,15,0, 0);

//WORKPIECE BLOCKING (MLTF table)
SetMLTFSuctionCupPosition(100 , 150, 0, "H50-scan-compatible");
SetMLTFSuctionCupPosition(100 , 650, 0, "H50-scan-compatible");
SetMLTFSuctionCupPosition(1100 , 150, 0, "H50-scan-compatible");
SetMLTFSuctionCupPosition(1100 , 650, 0, "H50-scan-compatible");
```

The image below shows the type of blocking performed.



3.11 Definition of workpieces machined contemporaneously

In order to machine various workpieces define the following:

1. Number of workpieces to machine contemporaneously on the table
2. Whether to minimize the tool changes in the program

3.11.1 Number of workpieces machined contemporaneously

The number of workpieces to machine contemporaneously on the table depends on the dimension of the workpieces and the dimension of the machine table.

A workpieces setup layout in the machine is required so that the various workpieces can be laid out without colliding with each other and to ensure that the machining of one workpiece does not machine the others.

After having decided, for example, that four workpieces can be machined contemporaneously, the script for each workpiece must have the correct positioning coordinates in the "SetWorkpieceSetupPosition" instruction.

See below for the instructions to lay out 4 workpieces on the AB work area:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);
CreateFinishedWorkpieceBox("Workpiece1",800,400,63);
CreateRawWorkpiece("Workpiece1",10,10,2,0,0,0);
SetWorkpieceSetupPosition(10,10,0, 0);
```

...(workpiece 1 machining)...

```
CreateFinishedWorkpieceBox("Workpiece2",800,400,63);
CreateRawWorkpiece("Workpiece2",10,10,2,0,0,0);
SetWorkpieceSetupPosition(1600,10,0, 0);
```

...(workpiece 2 machining)...

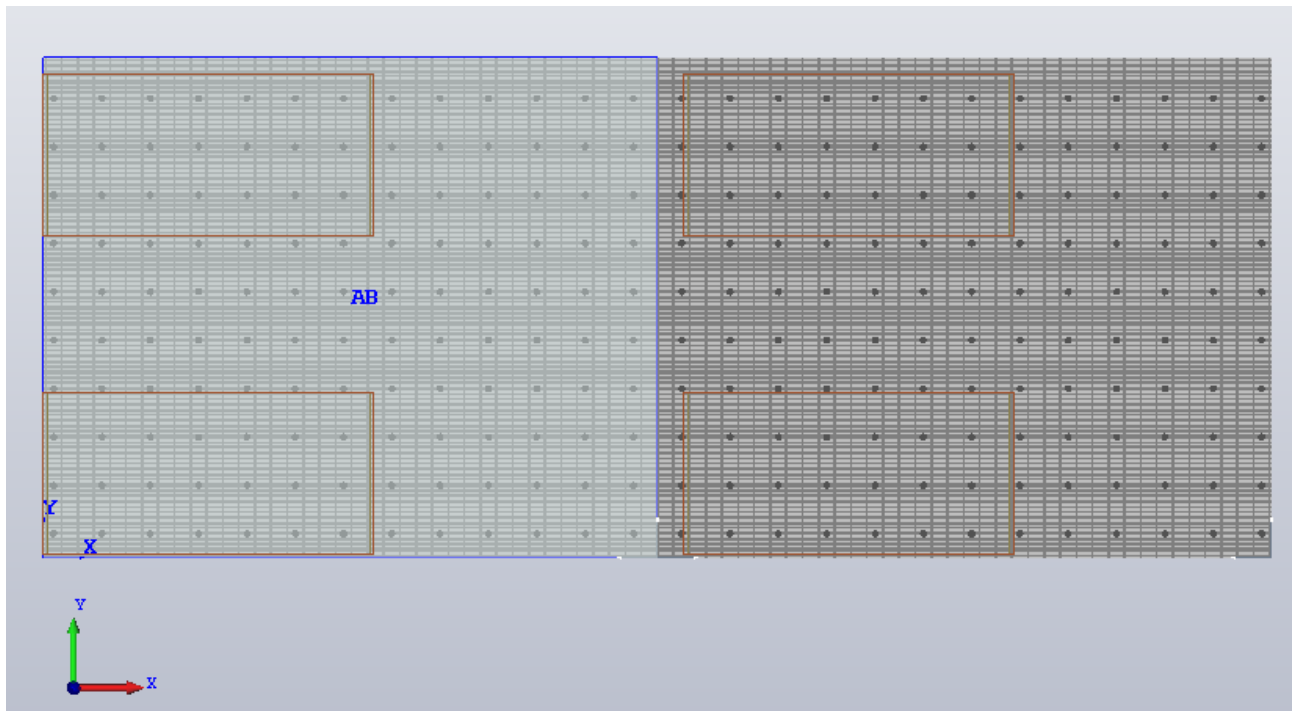
```
CreateFinishedWorkpieceBox("Workpiece3",800,400,63);
CreateRawWorkpiece("Workpiece3",10,10,2,0,0,0);
SetWorkpieceSetupPosition(10,800,0, 0);
```

...(workpiece 3 machining)...

```
CreateFinishedWorkpieceBox("Workpiece4",800,400,63);
CreateRawWorkpiece("Workpiece4",10,10,2,0,0,0);
SetWorkpieceSetupPosition(1600,10,0, 0);
```

...(workpiece 4 machining)...

The setup can be seen in the figure. The edges of the workpieces are in brown.



After having arranged the workpieces on the table, they must be machined in sequence, machining the first completely and then the second, third and fourth. In this way a lot of time would be lost due to an excessive number of tool changes.

To minimize the tool changes, use the Maestro "Tool Changes Optimization" function.

3.12 Optimization project

3.12.1 Creating pgmx with various optimized workpieces

To create a program (pgrmx) with various workpieces whose machining is performed with the least number of tool changes, starting from one or more scripts containing the definition of the machining processes of the single workpieces, use the “Xconverter” command available in the Maestro installation package .

The Xconverter command is used to define which workpieces to insert in a single pgmx and decide whether to optimize the machining.

For the syntax to use see chapter 3.1.2.3

3.13 Mathematical functions

The table below lists the main mathematical functions available in the scripting.

Function	Description and type of return
<code>double abs(double value)</code> <code>double ABS(double value)</code>	Returns the absolute value of a specific value.
<code>double acos(double value)</code> <code>double ACOS(double value)</code>	Returns the arc-cosine of a specific value.
<code>double asin(double value)</code> <code>double ASIN(double value)</code>	Returns the arc-sine of a specific value.
<code>double atan(double value)</code> <code>double ATAN(double value)</code>	Returns the arc-tangent of a specific value.
<code>double sin(double value)</code> <code>double SIN(double value)</code>	Returns the sine of a specific value.
<code>double cos(double value)</code> <code>double COS(double value)</code>	Returns the cosine of a specific value.
<code>double tan(double value)</code> <code>double TAN(double value)</code>	Returns the tangent of a specific value.
<code>double ln(double value)</code> <code>double LN(double value)</code>	Returns the logarithm of a specific value.
<code>double log10(double value)</code> <code>double LOG10(double value)</code>	Returns the logarithm to base ten of a specific value.
<code>double exp(double value)</code> <code>double EXP(double value)</code>	Returns the exponential of a specific value.
<code>double sqrt(double value)</code> <code>double Sqrt(double value)</code>	Returns the square root of a specific value.
<code>double pow(double x, double y)</code> <code>double POW(double x, double y)</code>	Returns the elevation to base power X and exponent Y.
<code>double rd(double value)</code> <code>double RD(double value)</code>	Returns the rounding off by defect for a specific decimal value.
<code>double ru(double value)</code> <code>double RU(double value)</code>	Returns the rounding off by excess for a specific decimal value.

3.14 Accessorial functions

This section describes certain functions that can help the user in creating a script.

Function	Parameters	Description and type of return
<code>string</code> GetVersion()	None	Reads the Maestro version in execution and returns it as string
<code>Int</code> GetProgrammingOrigine()	No uno	Returns an integer value that identified the programming origin: 0 = SCM 1 = Morbidelli
<code>void</code> Print(<code>string</code> message)	Message to print string. (if the string starts with “@” the parameter is the identification key of the message to print).	Used to print an error message.
<code>bool</code> NDEF(<code>double?</code> value)	Double type test variable.	Used to check if the variable contains a valid value. Returns true if the variable does not contain a valid value.
<code>bool</code> DEF(<code>double?</code> value)	Double type test variable.	Used to check if the variable contains a valid value. Returns true if the variable contains a valid value.
<code>bool</code> NDEF(<code>string</code> value)	String type test variable.	Used to check if the variable contains a valid value. Returns true if the variable does not contain a valid value.
<code>bool</code> DEF(<code>string</code> value)	String type test variable.	Used to check if the variable contains a valid value. Returns true if the variable contains a valid value.
<code>double</code> V(<code>double?</code> par)	Double type variable	Returns the value in the variable (if the variable is not defined it returns zero).
<code>double</code> VALUE(<code>double?</code> par)	Double type variable	Returns the value in the variable (if the variable is not defined it returns zero).

4 Nesting Project Management

4.1 Create new Nesting project

4.1.1 Nesting parts

Parts to be entered in a Nesting project must be .pgmx type programs that must contain just one workpiece and one phase.

A list of parts to be inserted in a Nesting project with a .CSV format file.

4.1.1.1 CSV file structure (for Nesting)

The .csv file for Nesting describes all the single programs (.pgmx) that must be inserted as parts in the creation of a Nesting project.

Each program is described by a line of the file consisting of a sequence of parameters separated by a semicolon (;) or comma (,), as specified in the options.

The parameters are:

Workpiece Name	Name of the workpiece contained in the pgmx file to nest (obligatory)
Workpiece Type:	Type of workpiece (empty field or text).
Description	Workpiece description (empty field or text).
Quantity:	Quantity to nest (value greater than 0)
ExtraQuantity:	Additional quantity to nest if possible (value greater than or equal to 0).
Length:	Workpiece length (along axis X) (OBLIGATORY).
Width:	Workpiece width (along axis Y) (OBLIGATORY).
Thickness:	Workpiece thickness (along axis Z) (OBLIGATORY).
Material:	Name of the workpiece material (empty field or material name)
Grain:	Grain (1 horizontal, 2 vertical)
File.pgmx :	File name with extension (relative name of file with extension)

Example:

```
Part200x300;F1;Routing;5;3;200.000;300.000;18;NOCE;0;Part200x300.pgmx
Part400x400;F1;Routing;10;6;400.000;400.000;18;WALNUT;0;Part400x400.pgmx
```

5 .mixx project creation from command line

A mix type project (.mixx) can be created from a command line with the XConverter command and a .csv type of file.

The XConverter command is launched from the command line with the PC "commands prompt" function.

The most effective way is to create a batch file (.bat) where the complete instruction to be executed is written.

5.1 .CSV file structure (for Mix)

The .csv file describes the single programs (.pgmx) of the Mixx and the information used to execute them in the machine, such as dimensions, blockages, repetitions, etc.

The lines are contained in a file, consisting of a series of elements composed of a key in brackets "[TAG]" separated with the symbol "=" from the associated value. The field must end with the character ";".

The allowed keys are listed below:

[PRG]=string; : pgmx. file name
[DX]=decimal value; : Workpiece length (single)
[DY]=decimal value; : Workpiece width (single)
[DZ]=decimal value; : Workpiece thickness (single)
[BX]=decimal value; : Coordinate X of workpiece positioning
[BY]=decimal value; : Coordinate Y of workpiece positioning
[BZ]=decimal value; : Coordinate Z of workpiece positioning
[R]=integer value; : Number of program repetitions
[FLD]=string; : Work area on which to execute the program
[MES]=string; : Measuring system ("MM"=millimetres - "IN"=inches)
[OPM]=integer value; : Mechanical options (Xilog T field)
[BLK]=integer value; : Type of workpiece blockage (Xilog V field)

The type of representation in which the header parameters value is assigned is:
"[TAG]=Value" of this type (example of MIX step):

[PRG]=string; [DX]=value; [DY]=value; [DZ]=value; [BX]=value; [BY]=value; [BZ]=value;
[R]=value; [FLD]=value; [MES]=string; [OPM]=value; [BLK]=value;

Only "[PRG]", the PGMX name, is obligatory, whilst all the other fields are optional. If they are not present, they must not be entered in the MIXX step; when MIXX is launched the default of the equivalent value set in the PGMX or at the system level will apply (type of measuring system [MES]).

The sequence with which the fields appear is not preset, as they are processed as nominal "tag".

5.2 Create .mixx project from .csv file

This mode is used to create a .mixx file from a .csv file.
To do this use the Xconverter command with the m 11 option.

Example:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^  
-s -m 11 ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame>List for mix.csv" ^  
-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx" ^  
-o "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\mix.mixx"
```

With this example the following file is created: mix.mixx

Using the input file: List for mix.csv

That contained the following lines :

```
[PRG]=1_MSX.pgm  
[PRG]=1_MDX.pgm
```

Example 2:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^  
-s -m 11 ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame>List with folder.csv" ^  
-t "D:\Documents\SCM\SW\Maestro\Tlgx\SerrSCM_R1.tlgx" ^  
-o "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\mix_folder.mixx"
```

Using the input file: List with folder.csv

That contained the following lines :

```
[PRG]=folder\1_MSX.pgm  
[PRG]=folder\1_MDX.pgm
```

6 Creation of nesting project from the command line

You can create a nesting project type (.nstx) from the command line (nstx) using the csv command XConverter and using a csv type file

The XConverter command is launched from the command line using the "command prompt" on the PC.

The most effective way is to create a batch file (.bat) on which the complete instruction to execute is written.

6.1 Nesting project creation execution from the command line

You can create a nesting project .nstx from the command line starting from:

- a .csv file with the nesting options
- a .csv file describing the sheets in stock
- a .csv file with the list of parts to nest

The XConverter command is launched from the command line using the "command prompt" on the PC.

The most effective way is to create a batch file (.bat) on which the complete instruction to execute is written.

To do this you use the Xconverter with option m 15

Utilisation example with individual program:

```
Call "D:\Progetti\Xcam\UI00\bin\Debug\XConverter.exe" ^
(*\Projects\Xcam\UI00\bin\Debug\XConverter.exe" ^)
-s -m 15 ^
-the "D:\Documenti\SCM\SW\Maestro\Progetti\options.csv" ^
(*\Documents\SCM\SW\Master\Projects\options.csv" ^)
"D:\Documenti\SCM\SW\Maestro\Progetti\sheets.csv" ^
(*\Documents\SCM\SW\Master\Projects\sheets.csv" ^)
"D:\Documenti\SCM\SW\Maestro\Progetti\parts.csv"
(*\Documents\SCM\SW\Master\Projects\parts.csv")
-t "D:\Documents\SCM\SW\Master\Tlgx\tools.tlgx" ^
-or "D:\Documents\SCM\SW\Master\Projects\results.nstx" ^
-report
```

The order in which you specify the .csx input must be the following:

- .csv file with the nesting options
- .csv file describing the sheets that can be used for nesting
- .csv file with the list of parts to nest

6.2 File structure ,CSV of the options

Describes the options to run the nesting algorithm.

The lines are contained in the file are characterised by a series of elements consisting of a key in brackets "[TAG]" separated by the symbol "=" by the associated value. The field must end with the character ";" or "." as set in the options to Master.

Below are the keys accepted:

[TYPE]

Type: string; : Accepted values RECTANGULAR, FREEFORM, TUBE. Default: FREEFORM
Defines the type of Nesting.

[PARTSEQUENCING]

Type: string; Accepted values: AREA, PERIMETER. Default: AREA
Defines whether the Nesting order depends on the area of the parts or from the perimeter.

[MINIMUMPARTDISTANCE]

Type: decimal; Values accepted: greater than or equal to 0. Default: 2.
Defines the minimum distance between two nested parts.

[SPOILBOARDTHICKNESS]

Type: decimal; Values accepted: greater than or equal to 0. Default: 18.
Defines the thickness of the panel measured by the cover panel.

[ENABLEOUTSIDE CUT]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines the ability to pass with the tool on the outside of the sheet.

[ENABLEAUTOMATIC CUT]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 1.
Defines the automatic cutting of parts.

[ENABLEPARTHOLESNESTING]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines the possibility of using the holes of parts to be nested.

[ENABLEPARTHOLES CUT]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines the cutting of holes of the parts if the parameter ENABLEAUTOMATIC CUT is enabled.

[CUTTING TOOL]

Type: string; Accepted values: tool name shown in setup or empty string to use the default cutting tool for the nesting cut selected in the Master options. Default: "".

[CUT DEPTH]

Type: decimal; Accepted values: all. Default: 0.
Defines the extra depth of the cut if the parameter ENABLEAUTOMATIC CUT is enabled.

[MAX CUT WIDTH]

Type: decimal; Values accepted: greater than or equal to 0. Default: 0. The value is used if the automatic cutting of parts ENABLEAUTOMATIC CUT is not enabled and represents the maximum cut to apply between the parts.

[PREFEREPARTHOLESNESTING]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines the preference of Nesting in holes of the parts if ENABLEPARTHOLESNESTING parameter is enabled.

[RESOLUTION]

Type: decimal; Accepted values: greater than or equal to 0. Default: 1

Defines the Nesting resolution.

[LABELNAME]

Type: string; Accepted values: name of the label template to be applied to the parts or empty string for not applying the label. Default: ""

Defines the name of the label for labelling of Nesting parts.

[FINALCUTFEEDSPEED]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0

Defines the feed speed of the final pass of the cutting of the parts.

[FINALCUTMATERIALDEPTH]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0

Defines the depth of material that needs to be left out of the cut depth of the engraving pass.

[ENABLEINCISION]

Type: integer; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines the enabling of the incision cut of the first part of the final cut.

[MAXIMUMINCISIONPARTAREA]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.

Defines the maximum area of the part for which you can perform the incision before cutting.

0 value means that it is always carried out, values greater than 0 indicates the threshold below which it is executed.

[ENABLECADCOMPENSATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines whether the management of the tool compensation is calculated by Cad or by Cn.

[CUTDIRECTION]

Type: Type; Accepted values: GEOMETRY, CLOCKWISE, COUNTERCLOCKWISE Default: GEOMETRY.

Defines the direction of the cut compared to the profile.

[ENABLEPRESSUREROLLERS]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines the enabling of pressure rollers if available in configuration.

[ENABLETOOLBLOWER]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines the enabling of the machine function of the tool blower.

[TOOLBLOWERMODE]

Type: whole; Accepted values: 0 (= air), 1 (= Air and Oil), 2 (= Blower head), 3 (= Air Tank), 4 (= Air and Oil Tank). Default: 0.

Defines the enabling of the machine function of the tool blower.

[ENABLEMECHANICALPRESSER]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines the enabling of the machines mechanical presser function.

[ENABLEEXECUTABLESOPTIMIZATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

Defines whether to optimise machining.

[EXECUTABLESOPTIMIZATIONSTEPWIDTH]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.
Defines the step width with which to optimise machining.

[EXECUTABLESOPTIMIZATIONSTEPLength]

Type: decimal; Accepted values: greater than or equal to 0. Default: 1.
Defines the step length with which to optimise machining.

[ENABLESMALLPARTOPTIMIZATIONFIRST]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to optimise the cuts of smaller parts first.

[ENABLEDRILLINGOPTIMIZATIONFIRST]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to optimise the drilling of other machining first.

[ENABLEMILLINGINVERSIONOPTIMIZATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to invert the machining during the optimisation stage.

[CUTSTARTPOSITION]

Type: string; Accepted values: NONE, MIDDLEOFFIRSTELEMENT, MIDDLEOFMOSTLENGTHELEMENT, VERTEXOFFIRSTELEMENT Default: NONE:
Defines the starting position of the cut.

[ENABLEREMNANTGENERATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to enable the generation of the remnants.

[ENABLEREMNANTBYAREA]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to treat the remnants by area or size

[REMNANTREMNANTMINAREA]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.
Defines the minimum sheet area to be regarded as remnants.

[REMNANTREMNANTMINLENGTH]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.
Defines the minimum length of the sheet area to be regarded as remnants.

[REMNANTREMNANTMINWIDTH]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.
Defines the minimum width of the sheet area to be regarded as remnants.

[REMNANTTYPE]

Type: string; Accepted values: TRUESHAPE, RECTANGULAR, STEPPED, CLEAN CUT. Default: RECTANGULAR.
Defines the type of remnants to be generated.

[ENABLEREMNANTONLYONLASTSHEET]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to enable the generation only on the last sheet.

[REMNANTCUTTINGTOOL]

Type: string; Accepted values: tool name shown in setup or empty string to use the default tool for cutting remnants. Default: "" in this case the cutting parts tool is taken.

[ENABLESCRAPGENERATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to enable the generation of the scrap.

[ENABLESCRAPPOCKETING]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to enable the extraction of the scrap.

[SCRAPCUTMINAREA]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.
Defines the minimum area for scrap to be cut below this value the scrap is particle board.

[SCRAPPOCKETPASSOVERLAP]

Type: decimal; Accepted values: greater than or equal to 0. Default: 5.
Defines the percentage of overlap of the machining passes of the extraction of the scrap.

[SCRAPPOCKETALLOWANCESIDE]

Type: decimal; Accepted values: greater than or equal to 0. Default: 5.
Defines the extraction of the scrap of the quantity of scrap material to leave between the profile and the last pass.

[ENABLESCRAPCUTIFPOCKETINGIMPOSSIBLE]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to enable the cutting of the scarp if the extraction of the scrap is not possible.

[ENABLESCRAPPRECUT]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.
Defines whether to enable the pre cut scraps.

[SCRAPPRECUTDISTANCE]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0.
Define the distance from the edge of the scrap profile to which pre-cut must be carried out.

[SCRAPCUTTINGTOOL]

Type: string; Accepted values: tool name shown in setup or empty string to use the default tool for cutting scrap. Default: "" in this case the parts cutting tool is taken.

Example .csv file with the specified options (for unselected options the default values planned are used):

[TYPE] = RECTANGULAR;
[MINIMUMPARTDISTANCE] = 3;
[SPOILBOARDTHICKNESS] = 0;

6.3 File structure ,CSV of the sheets

[CODE]

Type: string; : Accepted values string not anything with code. Default: required field

[WIDTH]

Type: decimal; Accepted values: greater than 0. Default: 1600.

[HEIGHT]

Type: decimal; Accepted values: greater than 0. Default: 1200.

[DEPTH]

Type: decimal; Accepted values: greater than 0. Default: 18.

[MATERIAL]

Type: string; Accepted values: all. Default: "".

[QUANTITY]

Type: whole; Accepted values: greater than 0. Default: 1.

[NESTINGDIRECTION]

Type: string; Accepted values: X, Y. Default: X.

[NESTINGSTARTCORNER]

Type: string; Accepted values: LOWERLEFT, LOWERRIGHT, UPPERLEFT, UPPERRIGHT. Default: LOWERLEFT.

[GRAIN]

Type: string; Accepted values: X, Y, empty string. Default: X.

[LEFTMARGIN]

Type: decimal; Values accepted: greater than or equal to 0. Default: 0.

[TOPMARGIN]

Type: decimal; Values accepted: greater than or equal to 0. Default: 0.

[RIGHTMARGIN]

Type: decimal; Values accepted: greater than or equal to 0. Default: 0.

[BOTTOMMARGIN]

Type: decimal; Values accepted: greater than or equal to 0. Default: 0.

[ENABLEROTATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

File example .csv file with the list of parts to nest:

```
[CODE]=sheet1; [WIDTH] = 3000; [HEIGHT] = 1500; [DEPTH] = 22; [MATERIAL] = MDF; [QUANTITY] = 3;  
[CODE]=sheet2; [WIDTH] = 2000; [HEIGHT] = 1500; [DEPTH] = 18; [MATERIAL] = MDF; [QUANTITY] = 10;
```

6.4 File structure ,.CSV of the parts

[PARTPROGRAMNAME]

Type: string; : Accepted values: filename pgmx of the program piece. Default: required field

[CODE]

Type: string; : Accepted values string not anything with code. Default: if not specified is considered the name of the first piece present in the pgmx.

[WIDTH]

Type: decimal; Accepted values: greater than 0. Default: if not specified the pgmx one is used.

[HEIGHT]

Type: decimal; Accepted values: greater than 0. Default: if not specified the pgmx one is used.

[DEPTH]

Type: decimal; Accepted values: greater than 0. Default: if not specified the pgmx one is used.

[MATERIAL]

Type: string; Accepted values: all. Default: "".

[QUANTITY]

Type: whole; Accepted values: greater than 0. Default: 1.

[EXTRAQUANTITY]

Type: whole; Accepted values: greater than 0. Default: 0.

[ENABLEROTATION]

Type: whole; Accepted values: 0 (= disabled), 1 (= enabled). Default: 0.

[GRAIN]

Type: string; Accepted values: X, Y, empty string. Default: X.

[ROTATIONSTEPANGLE]

Type: decimal; Accepted values: greater than or equal to 0. Default: 15 (valid only for freeform nesting).

[PRIORITY]

Type: whole; Accepted values: greater than or equal to 0. Default: 0.

[EDGEMINDISTANCE]

Type: decimal; Accepted values: greater than or equal to 0. Default: 0 (valid only for freeform nesting).

LABELFIELD label type fields for labelling. These fields allow you to assign the information for the label of the part and you can have a field number from 1 to N where N is defined in the Master options.

[LABELFIELD1]

Type: string; Accepted values: all.

[LABELFIELDN]

Type: string; Accepted values: all.

File example .csv file with the list of parts available for nesting:

[PARTPROGRAMNAME]=part1.pgm;[MATERIAL]=MDF;[QUANTITY]=1;[EXTRAQUANTITY]=0

[PARTPROGRAMNAME]=part2.pgm;[MATERIAL]=MDF;[QUANTITY]=4;[EXTRAQUANTITY]=0

6.5 Nesting report

In addition to the output nstx file in the same folder, you can generate a report file with the Nesting results in xml format (. nstsrxx) in which the cutting drawings are represented and the parts included in them. To do this you must enter the key in the command line:

-report

which enables the generation of the cutting diagram.

Below is an extract from the xml type file that defines the cutting diagram.

```
<NestingSheetsResult xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.Nesting">
  <SheetResults>
    <SheetResult>
      <Key xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
        <ID>1739377</ID>
        <ObjectType>ScmGroup.XCam.Nesting.SheetResult</ObjectType>
      </Key>
      <Name
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">Foglio1</N
ame>
      <IsRotated>false</IsRotated>
      <NetSheetArea>5000000</NetSheetArea>
      <NumberOfSheets>1</NumberOfSheets>
      <PartResults>
        <PartResult>
          <Key xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
            <ID>0</ID>
            <ObjectType i:nil="true" />
          </Key>
          <Name
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">Pezzo500x
500</Name>
          <Angle>0</Angle>
          <IsLabelEnabled>false</IsLabelEnabled>
          <IsMirrored>false</IsMirrored>
          <LabelXCoordinate>0</LabelXCoordinate>
          <LabelYCoordinate>0</LabelYCoordinate>
          <PartID
xmlns:d6p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
            <d6p1:ID>1739369</d6p1:ID>
            <d6p1:ObjectType>ScmGroup.XCam.Nesting.PartInfo</d6p1:ObjectType>
          </PartID>
          <PartName>Pezzo500x500</PartName>
          <XCoord>7</XCoord>
          <YCoord>7</YCoord>
        </PartResult>
        <PartResult>
          <Key xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
            <ID>0</ID>
            <ObjectType i:nil="true" />
          </Key>
          <Name
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">Pezzo500x
500</Name>
```

```

<Angle>0</Angle>
<IsLabelEnabled>>false</IsLabelEnabled>
<IsMirrored>>false</IsMirrored>
<LabelXCoordinate>0</LabelXCoordinate>
<LabelYCoordinate>0</LabelYCoordinate>
<PartID
xmlns:d6p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d6p1:ID>1739369</d6p1:ID>
<d6p1:ObjectType>ScmGroup.XCam.Nesting.PartInfo</d6p1:ObjectType>
</PartID>
<PartName>Pezzo500x500</PartName>
<XCoord>7</XCoord>
<YCoord>519</YCoord>
</PartResult>
</PartResults>
<SheetID
xmlns:d4p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d4p1:ID>1739322</d4p1:ID>
<d4p1:ObjectType>ScmGroup.XCam.Nesting.SheetInfo</d4p1:ObjectType>
</SheetID>
<SheetName>Foglio1</SheetName>
<TotalCutArea>0</TotalCutArea>
<TotalPartArea>0</TotalPartArea>
<Utilisation>89.4</Utilisation>
<WorkpieceID
xmlns:d4p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d4p1:ID>1739407</d4p1:ID>
<d4p1:ObjectType>ScmGroup.XCam.MachiningDataModel.ProjectModule.WorkPiece</d4p1:ObjectType>
</WorkpieceID>
</SheetResult>
<SheetResult>
<Key xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<ID>1739378</ID>
<ObjectType>ScmGroup.XCam.Nesting.SheetResult</ObjectType>
</Key>
<Name
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">Foglio1(1)<
/Name>
<IsRotated>>false</IsRotated>
<NetSheetArea>5000000</NetSheetArea>
<NumberOfSheets>1</NumberOfSheets>
<PartResults>
<PartResult>
<Key xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<ID>0</ID>
<ObjectType i:nil="true" />
</Key>
<Name
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">Pezzo500x
500</Name>
<Angle>0</Angle>
<IsLabelEnabled>>false</IsLabelEnabled>
<IsMirrored>>false</IsMirrored>
<LabelXCoordinate>0</LabelXCoordinate>
<LabelYCoordinate>0</LabelYCoordinate>
<PartID
xmlns:d6p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d6p1:ID>1739369</d6p1:ID>
<d6p1:ObjectType>ScmGroup.XCam.Nesting.PartInfo</d6p1:ObjectType>
</PartID>
<PartName>Pezzo500x500</PartName>
<XCoord>7</XCoord>

```

```

<YCoord>7</YCoord>
</PartResult>
<PartResult>
<Key xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<ID>0</ID>
<ObjectType i:nil="true" />
</Key>
<Name
xmlns="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">Pezzo500x
500</Name>
<Angle>0</Angle>
<IsLabelEnabled>>false</IsLabelEnabled>
<IsMirrored>>false</IsMirrored>
<LabelXCoordinate>0</LabelXCoordinate>
<LabelYCoordinate>0</LabelYCoordinate>
<PartID
xmlns:d6p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d6p1:ID>1739369</d6p1:ID>
<d6p1:ObjectType>ScmGroup.XCam.Nesting.PartInfo</d6p1:ObjectType>
</PartID>
<PartName>Pezzo500x500</PartName>
<XCoord>7</XCoord>
<YCoord>519</YCoord>
</PartResult>
</PartResults>
<SheetID
xmlns:d4p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d4p1:ID>1739322</d4p1:ID>
<d4p1:ObjectType>ScmGroup.XCam.Nesting.SheetInfo</d4p1:ObjectType>
</SheetID>
<SheetName>Foglio1(1)</SheetName>
<TotalCutArea>0</TotalCutArea>
<TotalPartArea>0</TotalPartArea>
<Utilisation>89.2</Utilisation>
<WorkpieceID
xmlns:d4p1="http://schemas.datacontract.org/2004/07/ScmGroup.XCam.MachiningDataModel.Utility">
<d4p1:ID>1740261</d4p1:ID>
<d4p1:ObjectType>ScmGroup.XCam.MachiningDataModel.ProjectModule.WorkPiece</d4p1:ObjectType>
</WorkpieceID>
</SheetResult>
</SheetResults>
</NestingSheetsResult>

```

In addition to the cutting diagram, you can generate a file that describes the geometry of each part used in the Nesting.

By enabling the geo-option in the same Nesting folder many files as there are parts used will be generated, and each file will have a name coinciding with the part and extension (.iso).

Example:

Nesting parts

Part1

Part2

Part3

Exit:

Part1.iso

Part2.iso
Part3.iso

Each file will have a layout:

G0 X = Y Value = Value: Definition of the starting point of the X, Y geometry coordinates.
G1 X= Y Value =Value : Definition of the end point of a linear stretch X, Y coordinates.
G2 X= Y Value= I Value= J Value= Value : Definition of the end point of a linear arc time of X, Y coordinates.
and in the Centre, I, J.
G3 X= Y Value= I Value= J Value= Value : Definition of the endpoint of an arc counterclockwise direction of X, Y coordinates and Centre in I, J.

7 Execution of the cycle time estimate from command line

You can run the estimate of project cycle times (.pgmx) or an entire mix of projects (.mixx) through the XConverter command. The estimated results of the cycle times are exported to a csv format file.

The XConverter command is launched from the command line using the "command prompt" on the PC.

The most effective way is to create a batch file (.bat) on which the complete instruction to execute is written.

To do this you use the Xconverter command with option m 16

Utilisation example with individual program:

```
Call "D:\Progetti\Xcam\UI00\bin\Debug\XConverter.exe" ^
(*\Projects\Xcam\UI00\bin\Debug\XConverter.exe" ^)
-s -m 16 ^
-the "D:\Documenti\SCM\SW\Maestro\Progetti\test.pgm" ^
(*\Documents\SCM\SW\Master\Projects\test.pgm" ^)
-or "D:\Documents\SCM\SW\Master\Projects\estimationTimesResults.csv"
```

Here is an example of the CSV file generated for a single program.

```
;Total;test;
Number of repetitions; 1; 1;
Execution time [s]; 498.9; 498.9;
Total cost;0;0;
Milling' process execution time [s]; 143.7; 143.7;
Drilling' process execution time [s];9.2 9.2;
Facing' process execution time [s]; 0;0;
Edging' process execution time [s]; 34.5;34.5;
Edge trimming' process execution time [s]; 77.4;77.4;
Group operator time '4' [s];131.3; 131.3;
Group operator time '3' [s]; 133.6; 133.6;
Vertical drilling process execution time [s];0;0;
Number of vertical drilling downstrokes;0;0;
Average vertical drilling execution time [s];0;0;
Vertical drilling execution time [s];9,2;9,2;
Number of horizontal drilling downstrokes;6;6;
Average horizontal drilling execution time[s];1.5;1.5;
Drilling' execution time [s];9,2;9,2;
Milling' execution time [s];131,3;131,3;
Execution time is 'tilted milling' [s];0; 0;
3D milling' execution time [s];0; 0;
Contouring' execution time [s];0;0 ;
Emptying' execution time [s];0;0 ;
Blade cuts' execution time [s];0;0;
Channels' execution time [s];0;0;
Edging' execution time [s];34.5;34.5;
Trimming' execution time [s];16.3;16.3;
```

NOTE: the field names are localised in the language selected in the interface Master. For example, if the selected language is English, the csv file obtained as a result would be the following.

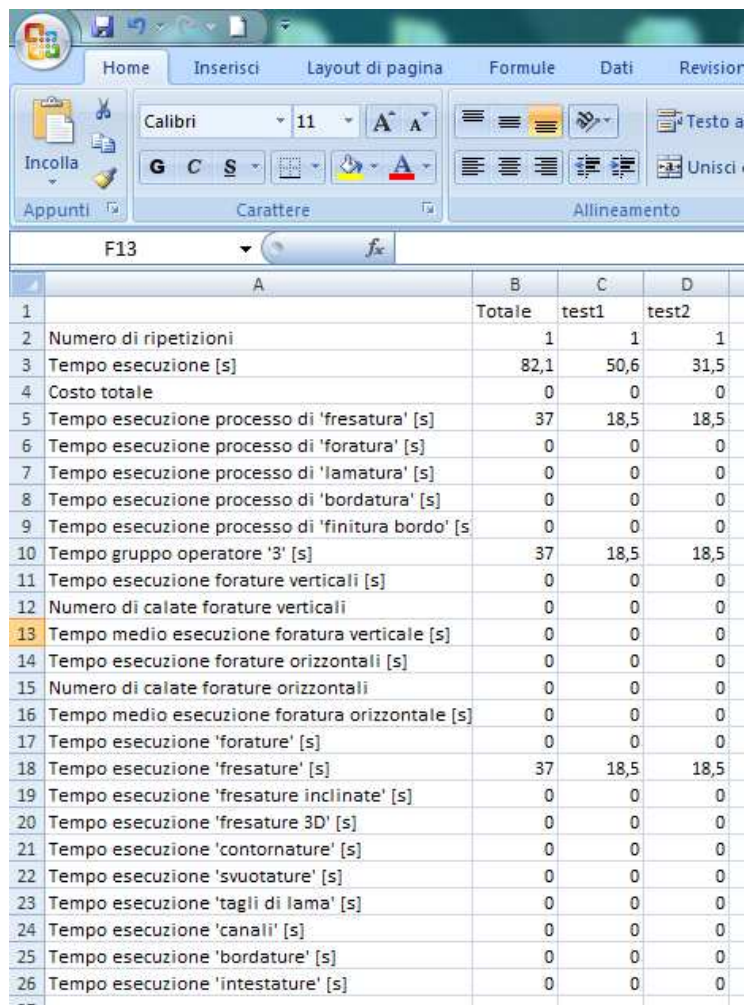
```
;Total;test;
Number of repetitions;1;1;
Execution time [s];50.6;50.6;
Total cost;0;0;
Milling execution time [s];18.5;18.5;
Drilling execution time [s];0;0;
Blading execution time [s];0;0;
Edge banding execution time [s];0;0;
Finishing execution time [s];0;0;
Machining head time '3' [s];18.5;18.5;
Vertical drilling execution time [s];0;0;
Number of vertical drilling downstrokes;0;0;
Vertical drilling average execution time [s];0;0;
Horizontal drilling execution time [s];0;0;
Number of horizontal drilling downstrokes;0;0;
Horizontal drilling average execution time [s];0;0;
Drilling execution time [s];0;0;
Milling execution time [s];18.5;18.5;
Tilted milling execution time [s];0;0;
3D milling execution time [s];0;0;
Contouring execution time [s];0;0;
Pocketing execution time [s];0;0;
Blade cuts execution time [s];0;0;
Slots execution time [s];0;0;
Edging execution time [s];0;0;
End trimming execution time [s];0;0;
```


Example of using a mix of two programs test1.pgm and test2.pgm:

```
Call "D:\Progetti\Xcam\UI00\bin\Debug\XConverter.exe" ^
(*\Projects\Xcam\UI00\bin\Debug\XConverter.exe" ^)
-s -m 16 ^
-the "D:\Documenti\SCM\SW\Maestro\Progetti\test.mixx" ^ (*
Document\SCM\SW\Master\Projects\test.mixx" ^)
-or "D:\Documenti\SCM\SW\Maestro\Progetti\estimationTimesResults.csv"
(*\Documents\SCM\SW\Master\Projects\estimationTimesResults.csv")
```

The following is an example of the CSV file generated for a mix of programs.

```
;Total;test1;test2;
Number of repetitions; 1;1;1;
Execution time [s]; 82.1;50.6;31.5;
Total cost;0;0;0;
Milling' process execution time [s]; 37; 18.5;18.5;
Drilling' process execution time [s];0;0;0;
Facing' process execution time [s]; 0;0;0;
Edging' process execution time [s];0;0;0;
Edge trimming' process execution time [s];0;0;0;
Group operator time '3' [s]; 37; 18.5;18.5;
Vertical drilling execution time [s];0;0;0;
Number of vertical drilling downstrokes;0;0;0;
Average vertical drilling execution time [s];0;0;0;
Vertical drilling execution time [s];0;0;0;
Number of horizontal drilling downstrokes;0;0;0;
Average horizontal drilling execution time[s];0;0;0;
Drilling' execution time [s];0;0;0;
Milling' execution time [s];37;18.5;18.5;
Execution time is 'tilted milling' [s];0; 0;0;
3D milling' execution time [s];0; 0;0;
Contouring' execution time [s];0;0;0;
Emptying' execution time [s];0;0;0;
Blade cuts' execution time [s];0;0;0;
Channels' execution time [s];0;0;0;
Edging' execution time [s];0;0;0;
'Trimming' execution time [s];0;0;0;
```



	A	B	C	D
1		Totale	test1	test2
2	Numero di ripetizioni	1	1	1
3	Tempo esecuzione [s]	82,1	50,6	31,5
4	Costo totale	0	0	0
5	Tempo esecuzione processo di 'fresatura' [s]	37	18,5	18,5
6	Tempo esecuzione processo di 'foratura' [s]	0	0	0
7	Tempo esecuzione processo di 'lamatura' [s]	0	0	0
8	Tempo esecuzione processo di 'bordatura' [s]	0	0	0
9	Tempo esecuzione processo di 'finitura bordo' [s]	0	0	0
10	Tempo gruppo operatore '3' [s]	37	18,5	18,5
11	Tempo esecuzione forature verticali [s]	0	0	0
12	Numero di calate forature verticali	0	0	0
13	Tempo medio esecuzione foratura verticale [s]	0	0	0
14	Tempo esecuzione forature orizzontali [s]	0	0	0
15	Numero di calate forature orizzontali	0	0	0
16	Tempo medio esecuzione foratura orizzontale [s]	0	0	0
17	Tempo esecuzione 'forature' [s]	0	0	0
18	Tempo esecuzione 'fresature' [s]	37	18,5	18,5
19	Tempo esecuzione 'fresature inclinate' [s]	0	0	0
20	Tempo esecuzione 'fresature 3D' [s]	0	0	0
21	Tempo esecuzione 'contornature' [s]	0	0	0
22	Tempo esecuzione 'svuotature' [s]	0	0	0
23	Tempo esecuzione 'tagli di lama' [s]	0	0	0
24	Tempo esecuzione 'canali' [s]	0	0	0
25	Tempo esecuzione 'bordature' [s]	0	0	0
26	Tempo esecuzione 'intestature' [s]	0	0	0

8 Tools Data-Base

When executing a script you can consult the tools database to retrieve specific information on a specific tool identified with its alphanumeric code (the same found in the tools database). The primitive that allows access to the information is the `GetTool`.

The `GetTool` primitive provides, if found, the `Tool` object for the specified code.

Name	Type	Description
name	string	Tool identification name

Table 102: GetTool parameters

With the `Tool` object the main geometric and technological properties of the tool can be consulted:

1. Description (`Description`)
2. Total tool length (`TotalLength`)
3. Total length (`PilotLength`)
4. Tool cutter length, useful length (`CutterLength`)
5. Total tool diameter, that is dimension in the normal axis rotation direction (`TotalDiameter`)
6. Tool cutting diameter, that is dimension in the normal axis rotation direction (`CuttingDiameter`)
7. Tool radius (`Radius`)
8. Blade type tool thickness (`BladeThickness`)
9. Tool rotation axis angle in relation to the vertical (`Angle`)
10. Max tool rotation speed (`MaxSpindleSpeed`)
11. Max tool feed speed (`MaxFeedSpeed`)

The example below shows the consultation of the tool radius with ID "E101".

```
double toolRadius;

// Retrieve tool object
Tool tool = GetTool("E101");
// Check if tool is found (tool object must not be null)
if (tool != null)
{
    // Initialize a variable with the tool radius
    toolRadius = tool.Radius;
}

... (continue)
```

9 Post-processor from command line

A program for the machine panel (.xml) can be created from a command line with the XConverter command.

The XConverter command is launched from the command line with the PC "commands prompt" function.

The most effective way is to create a batch file (.bat) where the complete instruction to be executed is written.

9.1 Create .xml file from a .pgmx file

This mode is used to convert one or more .PGMX files in .xml files.
To do this use the Xconverter command with the m 10 option.

Example:

```
Call "D:\projects\XCam\UI00\bin\Debug\XConverter.exe" ^  
-s -m 10 ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MSX.pgm" ^  
-i "D:\Documents\SCM\SW\Maestro\Projects\Post MaestroWD\0\Frame\MDX.pgm" ^
```

With this example the following files are created:

MSX_Output.xml
MDX_Output.xml

Using the input files:

MSX.pgm
MDX.pgm